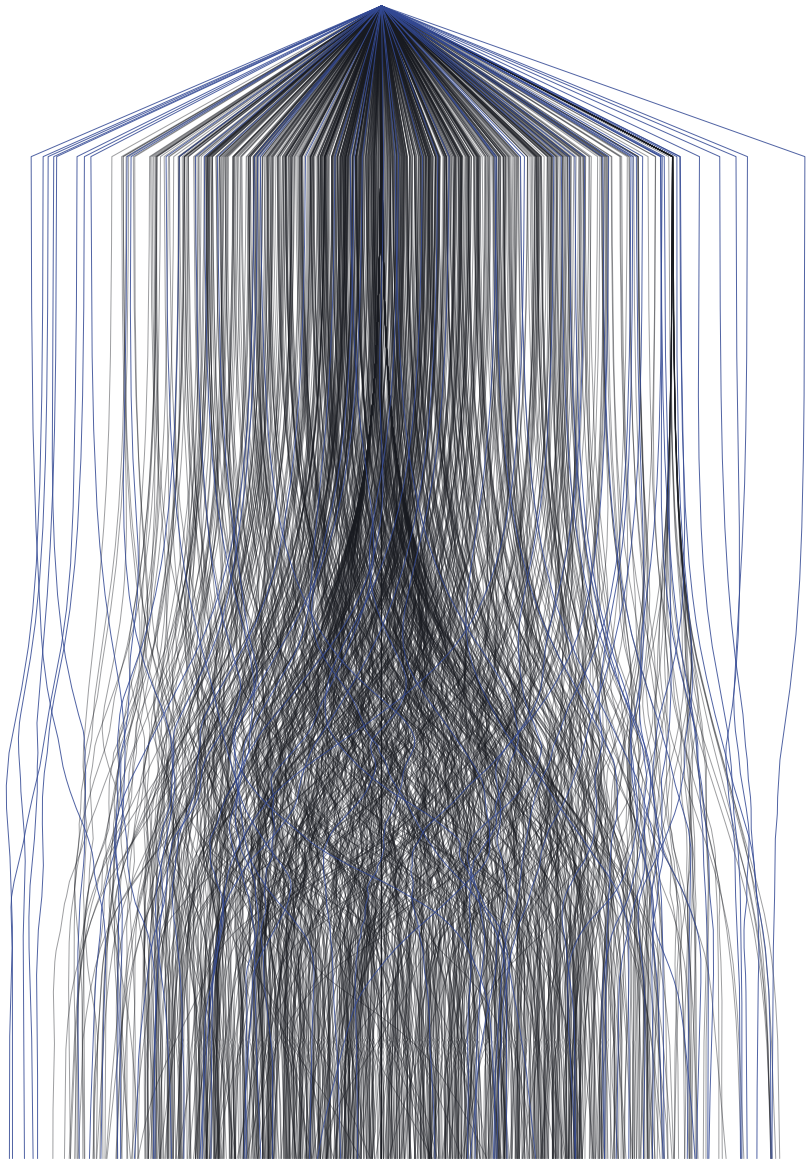
PICOGPT



Frontispiece. *Every one of the specimen's 1007 parameters, followed through training.* Time runs down the plate on a logarithmic scale (200 steps; AdamW, η 0.02 → 0.002 (warmup 20, cosine), β_2 0.95, weight decay 0.01 on matrices, clip 1; seed 1). Each path is one parameter's displacement $\Delta = \theta_i(t) - \theta_i(0)$ from its random starting value, placed horizontally at $\operatorname{asinh}(\Delta/\delta)$ with δ = 0.2608 the median final displacement (linear near 0, logarithmic far out); every path starts on the centre line. The straight roof at the top is Adam's first update, which is sign descent: nearly every parameter moves by the same distance η_1 = 0.001 (weight decay and ε cause the small differences). In value ink: the 56 coordinates of the embedding E. Eight paths do not visibly leave the centre line: the key bias b_k, whose gradient is identically zero.

PicoGPT

A *transformer small enough to see every number*

E .33 .71 −.62 −.21 **1.00** **−1.83** −.04 .81 **−2.95** **−1.28** .29 **−1.04** .94 −.47 **1.31** **−1.79** **−1.14** −.71 **−1.62** 1.02 **1.29** −.82 **−2.29** **−1.05** **1.29** **1.88** **1.60** **−1.46** −.11 .45 .61 **−1.31** **1.02** **−1.53** **−2.51** −.86 −.16 **1.18** **−.69** .41 **−.76** −.19 .64 .38 **1.60** **−.66** −.48 −.06 **.99** .30 .24 −.21 **−.95** −.33 **−.85** **1.10** γ_1 .87 **.98** **.86** **.96** **.95** **.76** **1.15** **1.18** β_1 .09 .08 −.08 −.05 −.01 −.05 .17 −.25 W_q .54 **−.55** −.21 .29 −.05 −.16 **−.70** .00 −.07 .32 **.97** −.03 **.69** .09 .11 **1.18** .05 .36 **1.13** **−.31** −.13 −.06 .14 **−.65** .21 .39 **−.42** .12 .08 **.48** −.11 .23 .30 −.07 −.13 **.57** .13 .18 .31 .12 **−.35** .24 .28 **−.32** .34 **−1.06** .03 .05 **−1.16** **−.39** **−.37** **−.49** **−.60** .01 **1.05** **−.26** **.58** .36 **−.48** **.50** .41 **−.29** −.05 .49 b_q −.08 .15 −.31 **.32** .09 −.08 −.30 .03 W_k .15 .43 .02 −.13 .09 .06 **1.53** **−.42** −.03 **−.27** −.14 **−.41** **−.80** .23 −.07 **−.90** .13 .26 −.15 .35 **.62** .06 −.02 **.49** **−.29** **−.86** **.61** −.08 .06 **−.42** .42 −.18 **−.59** .00 **.57** .39 −.15 **.52** **.54** .26 .37 .47 **−.76** **.42** .09 −.19 **−.47** −.21 **−.69** .24 **.71** **−.75** .32 **.57** −.28 .28 .12 −.07 **1.00** .59 −.03 **−.72** **−.47** −.27 b_k .00 .00 .00 .00 .00 .00 .00 .00 W_v .07 −.35 **−1.06** .10 −.14 **−.33** .04 **−.42** −.07 **−.48** −.31 −.21 −.05 −.27 .07 −.22 −.05 −.08 .05 **.68** **−.50** **−.57** .30 .37 −.19 −.11 **−.77** **−1.07** −.45 .28 −.05 .25 **−.38** −.31 −.26 .38 −.24 **.72** **−.48** **−.68** **−.54** −.02 −.12 .08 **−.36** **−.62** −.30 .20 .10 **.67** .26 .02 −.18 .35 .10 .34 **−.35** .18 **−.43** .10 **.59** **−.49** **.63** .40 b_v .06 .17 .13 −.09 −.24 −.18 −.09 −.19 W_o .29 .14 .13 −.02 −.11 .35 −.04 .25 **.51** .35 −.32 −.04 .38 −.23 −.12 −.31 **.52** −.16 −.04 **−.40** **.60** .12 −.09 .05 .34 −.24 −.32 **−.53** .06 **−.51** **−.55** **.58** −.05 .55 −.20 .09 −.30 −.14 .04 −.23 −.06 −.05 **−.51** .06 −.08 .18 .32 .18 **.50** **.51** .13 −.40 −.31 −.38 −.02 −.33 −.10 .24 **.57** .02 −.41 −.20 .14 −.13 b_o −.03 .16 .08 .01 .13 .02 −.06 −.31 γ_2 **.82** **.98** **1.38** **1.10** **1.17** **1.28** **1.31** **1.13** β_2 .26 .12 −.15 −.06 .15 .04 .00 −.24 W_1 −.45 −.49 −.18 .28 **.79** .43 −.23 **1.10** **.76** −.08 .06 .15 .11 −.18 **.56** **.76** .21 .16 −.20 −.33 .00 .23 **−.67** **.70** .12 .37 −.24 .22 **−1.00** −.28 −.06 **.65** .15 .29 **.68** −.11 **−.52** **−1.03** −.43 .30 −.25 .15 .10 −.10 −.04 −.14 −.30 **.91** .33 **1.06** **.64** −.44 −.18 .04 −.10 **−.90** −.01 .19 **−.54** .22 .17 .17 −.27 .01 .25 −.29 .37 **−.68** .36 −.12 .33 −.28 .15 **.58** **−.93** **.59** **−.71** **.96** −.16 .03 .04 −.27 .30 **−.69** **.66** .04 .11 **−.77** .17 **−.77** **−.92** −.15 **−.73** −.48 .40 −.15 **.94** **.64** −.30 −.15 **−.59** −.39 **−.65** .12 −.04 −.19 .29 **−1.01** −.32 **−.72** −.16 −.03 .36 .02 .09 .24 **−.52** .22 **.67** .35 .45 .38 −.06 .27 −.47 .35 −.07 .26 **.54** **.59** −.49 −.29 −.24 **.44** **1.29** −.26 −.27 −.14 .28 −.39 .16 .19 .24 **−.89** .39 −.08 .13 −.02 −.32 −.01 .23 .10 .33 −.10 .13 **−.83** **−.69** .26 .42 −.15 .42 −.03 .30 −.09 .32 .47 **−.71** **.87** −.13 −.10 **−.55** .25 −.20 .31 −.01 .05 **−1.08** **−.69** **−1.03** .10 .43 **−1.00** **.63** −.41 −.36 −.24 −.10 −.12 **−.98** −.22 −.46 **.78** .37 −.48 −.33 **.51** **−.82** **.70** −.20 .06 −.46 −.10 **.91** −.04 .20 **.48** **−.62** −.45 −.23 .26 −.01 **1.07** .13 **−.65** −.03 **.63** **−.50** **.67** **.44** **−.79** **.52** **−.68** −.25 −.17 **−1.04** −.11 **−.67** .31 **.73** −.12 .17 **1.01** .23 **−1.12** −.31 −.11 **.64** −.28 .00 .21 **.56** .18 .25 −.20 −.24 **.51** .13 **.92** .38 .16 **.62** −.18 −.17 −.33 −.15 .31 b_1 −.05 −.18 .19 .11 −.16 .21 .04 .06 −.17 .18 .30 −.24 −.14 −.03 .01 .10 −.11 −.29 −.16 −.09 **−.37** .17 −.06 .17 −.02 .04 −.02 −.12 −.18 .07 −.30 .06 W_2 **−.52** −.19 .26 .16 .34 −.17 .13 .15 −.09 **−.79** −.04 **.64** .08 −.07 .21 .02 −.21 .02 .22 .08 −.42 .00 −.16 .11 **.44** .23 .31 −.40 **−.58** .07 −.28 −.01 −.37 −.14 .00 **.55** .27 −.12 .20 .22 .26 **.69** .18 **−.57** **.54** .12 −.33 −.23 −.09 −.42 −.25 **−.74** .36 **.66** .12 −.10 −.14 −.32 −.36 .08 .23 .11 .10 −.19 −.12 −.31 −.45 .09 .14 .00 .14 .13 −.01 **.55** **.49** −.39 −.22 .07 −.30 **.72** −.17 **.63** .47 −.23 −.26 −.43 −.05 −.31 −.15 .32 .21 .27 **−.53** .00 .01 −.14 .34 −.28 −.38 .16 .02 −.02 −.06 **.51** .33 **.52** .02 −.11 −.05 −.31 −.48 −.15 −.30 −.32 −.07 .32 .34 −.15 .27 .33 −.36 .01 .17 .33 .33 −.32 .18 −.21 **.53** −.41 −.52 −.32 .33 .28 .01 .33 .44 .25 −.23 .09 .09 −.14 **−.55** .20 **.65** −.06 −.34 −.21 −.01 .09 **−.45** −.02 −.07 .31 .41 −.34 −.29 −.08 .09 −.43 −.31 .25 .08 −.03 −.29 −.04 .02 .27 .14 −.33 **−.62** −.01 .44 **.45** .19 −.27 −.10 −.07 .16 .16 .02 −.05 .13 −.47 .28 .26 .23 −.25 −.48 .36 −.25 .35 −.06 **−.57** −.24 −.22 .04 .38 .19 −.11 .05 **.49** .16 −.35 −.34 −.36 −.01 −.31 .38 −.04 .21 −.24 **−.48** .04 .09 .01 **−.51** −.18 .25 .00 .42 −.17 **.36** **−.63** .25 **.61** −.02 −.05 −.28 −.26 −.27 .10 −.31 −.38 −.33 **.50** **.52** −.22 .21 .20 −.06 −.03 −.15 −.02 −.14 .23 .29 .09 .08 **−.56** −.16 .06 .09 .09 .43 −.04 b_2 −.12 .01 .11 .06 −.02 −.06 .04 −.16 γ_f **2.13** **2.11** **1.98** **1.87** **2.17** **2.06** **1.56** **1.77** β_f −.28 −.22 .32 .21 −.23 −.28 .19 .13 W_{out} **−.84** **.92** −.20 **1.03** **−.73** .44 **.59** **−.81** .44 .51 **.70** **.88** **−.36** **−1.06** .06 **−1.05** .53 −.19 **.98** −.14 **−.95** **.70** .27 **−1.06** −.63 −.01 −.40 −.27 .48 **.73** **1.02** **−.98** **−.67** **−.61** .54 −.28 **−.80** .26 **−.70** −.52 **.94** **.78** .35 **−.85** .12 −.05 **−.50** **.89** −.24 −.19 .17 −.29 .48 .31 **−.85** **.95** b_{out} .33 −.38 −.35 −.29 .36 −.49 .10

$\theta \in \mathbb{R}^{1007}$: every parameter of the trained specimen, in the order the code keeps them. The weight of the type is $|\theta_i|$.

TOBIAS J. OSBORNE

The specimen	a transformer with 1007 parameters: $V = 7$, $D = 8$, $H = 2$, $L = 1$, $T = 8$
trained on	“to be or not to be”
its loss	2.004 → 0.059 nats in 200 steps; the text’s own ambiguity allows no less than 0.059
asked to continue “to be or”	“to be or not to be”

Every number in every figure comes from the specimen, as exported by the Julia code to artifacts/specimen.json. The code is at github.com/tobiasosborne/PicoGPT.jl.

Two spot inks carry one meaning each throughout the book: value ink for quantities computed going forward, parameters and activations; covector ink for adjoints $\bar{x}$ pulled back from the loss. Printed in one colour, value marks are solid and covector marks are barred or hatched.

Set in Literata, with mathematics in XCharter Math, labels in Archivo and code in JuliaMono, all under the SIL Open Font License. Typeset from the same HTML as the web edition, by Paged.js in Chromium.

Proof of 2026-11-04.

CONTENTS

0 Prologue 7

What this is. The specimen. How to read it: three names, two inks.

What this is 7 · The specimen 8 · How to read this book 10 · Two editions 10 · Running the code 11 · Exercises 12

1 The task 15

Text as a finite sequence. The chain rule of probability makes the model autoregressive: a language model is a map from contexts to distributions.

Text as a finite sequence 15 · The chain rule 16 · A language model is a map 16 · The probability of a text 18 · The training set 19 · What no model can learn 20 · In the code 21 · Exercises 22

2 Symbols become vectors 25

One-hot vectors, the embedding matrix, positions as Fourier features.

One-hot vectors 25 · The embedding matrix 26 · Positions as Fourier features 27 · A shift is a rotation 29 · The first state 31 · The pullback, in one line 32 · In the code 32 · Exercises 33

3 Energies and probabilities 35

Logits are negative energies; softmax is the Gibbs distribution. Temperature. Shift invariance and log-sum-exp.

The last map 36 · Softmax is the Gibbs distribution 36 · Temperature 37 · Only differences matter 39 · Log-sum-exp and the free energy 41 · In the code 44 · Exercises 44

4 Measuring surprise 47

Cross-entropy, KL divergence, maximum likelihood. The baseline ln V. The gradient: predicted minus observed.

The price of a prediction 48 · The loss 48 · Maximum likelihood 50 · Cross-entropy and the KL divergence 50 · The floor 51 · The gradient 52 · In the code 53 · Exercises 55

5 Attention 59

A row-stochastic mixing of columns; query, key, value; the $1/\sqrt{d}$ scaling; the causal mask as lower-triangularity; heads as a direct sum.

Where columns meet 59 · An average of earlier columns 60 · Scores, and why a softmax 60 · Why $\sqrt{d}$ 62 · The causal mask 63 · Heads: a direct sum 63 · In the code 65 · Exercises 66

6 The block 69

The residual stream as an Euler step. LayerNorm as projection to a sphere. The MLP. The whole model as one composition.

The stream 69 · LayerNorm 71 · The MLP 74 · One composition 76 · In the code 76 · Exercises 80

7 Pulling back 83

The chain rule as a composition of transposes. Each pullback derived. The gradient check.

One number, many parameters 83 · The pullback 84 · Fan-out: adjoints add 87 · The pullbacks, one by one 88 · The gradient check 92 · The parameter that never learns 94 · In the code 95 · Exercises 96

8 Learning 99

Gradient flow and its discretisation; minibatch noise; Adam as diagonal preconditioning; schedules; clipping; initial variances.

Watching it learn 100 · Gradient flow and its Euler steps 100 · Minibatches 104 · Adam 104 · Schedules and clipping 108 · The slingshot 110 · Where to start 112 · In the code 115 · Exercises 115

9 Speaking 117

Sampling, temperature, and what the Shakespeare model learned, shown honestly.

One character at a time 117 · Temperature in practice 119 · The specimen speaks 119 · A real model 121 · Inside the real model 123 · Exercises 124

Epilogue 127

What we left out — BPE, scale, GPUs — and a dictionary of jargon.

What we left out 127 · A dictionary of jargon 130

CHAPTER ZERO

Prologue

0

A transformer is a composition of maps between named spaces. Running it pushes values forward; training pulls one number, the loss, back through the transposed derivatives. This book shows it on a model small enough to see every number.

MATHS	PHYSICS	ML
a point $\theta \in \mathbb{R}^p$	*the coordinates of a configuration*	"parameters, weights"
$p = \dim \theta$	*the number of degrees of freedom*	"parameter count, model size"
a map f_θ from inputs to outputs	*a response function*	"model, network, architecture"
evaluating f_θ, value by value	*integrating forward*	"forward pass, inference"
$\ell : \mathbb{R}^p \to \mathbb{R}$, to be minimised	*an energy, a cost*	"loss, objective"
the covector $\bar{x} = \partial \ell / \partial x$	*the linear response, the conjugate variable*	"gradient"
applying $Df^\top$ in reverse order	*the adjoint equation*	"backward pass, backpropagation"

0.1 What this is

THIS IS A BOOK ABOUT ONE FUNCTION. IT TAKES A PIECE OF text and returns, for every position in it, a probability distribution over the character that comes next. It is built from matrix products, a softmax, a normalisation and one elementwise nonlinearity, composed in a fixed order, and it is the function at the

centre of every current large language model. The book derives it, writes it in Julia from scratch, and trains it.

The reader we have in mind has a degree's worth of linear algebra, analysis and probability. For that reader the mathematics of this subject is easy. The obstacle is the language: a field that grew up quickly has named familiar objects in unfamiliar ways, and has given several names to some of them. Softmax is a Gibbs distribution. Cross-entropy is a relative entropy plus a constant. Backpropagation is the chain rule, applied in the cheap order. We say so every time, and then use the words of the field, because they are the words of its literature.

0.2 The specimen

The protagonist of this book is a small transformer that we call *the specimen*. It is trained on a single line,

to *be* or *not* to *be*

eighteen characters over an alphabet of seven. It has the shape of a real model, with every part present: an embedding, one block of attention with two heads followed by a small network, a final normalisation and a readout. Only the sizes are small: a context of $T = 8$ characters, a width of $D = 8$, and so on (the shapes are fixed in Chapter 1 and Chapter 2). With L blocks, a width D, an alphabet of V characters and an inner width F, the number of parameters is

$$p = 2DV + V + 2D + L\,(4D^2 + 2DF + 9D + F),$$

and for the specimen's $V = 7$, $D = 8$, $F = 32$ and $L = 1$, that is $p = 1007$. Here they all are.

One group of these 1007 numbers can never learn. Look closely at the plate. By the end of Chapter 5 you can say which group, and prove it.

Every figure in this book shows this model, or the larger one of Chapter 9, and every number in every figure is computed from it as the page is drawn: in your browser if you are reading on screen, and when the book was set if you are reading on paper. Nothing is illustrative: when a figure prints a weight to two decimals, the model computed it.

The specimen learns its line almost perfectly, and the "almost" is instructive, not a defect. Chapter 1 shows why no model could do better.

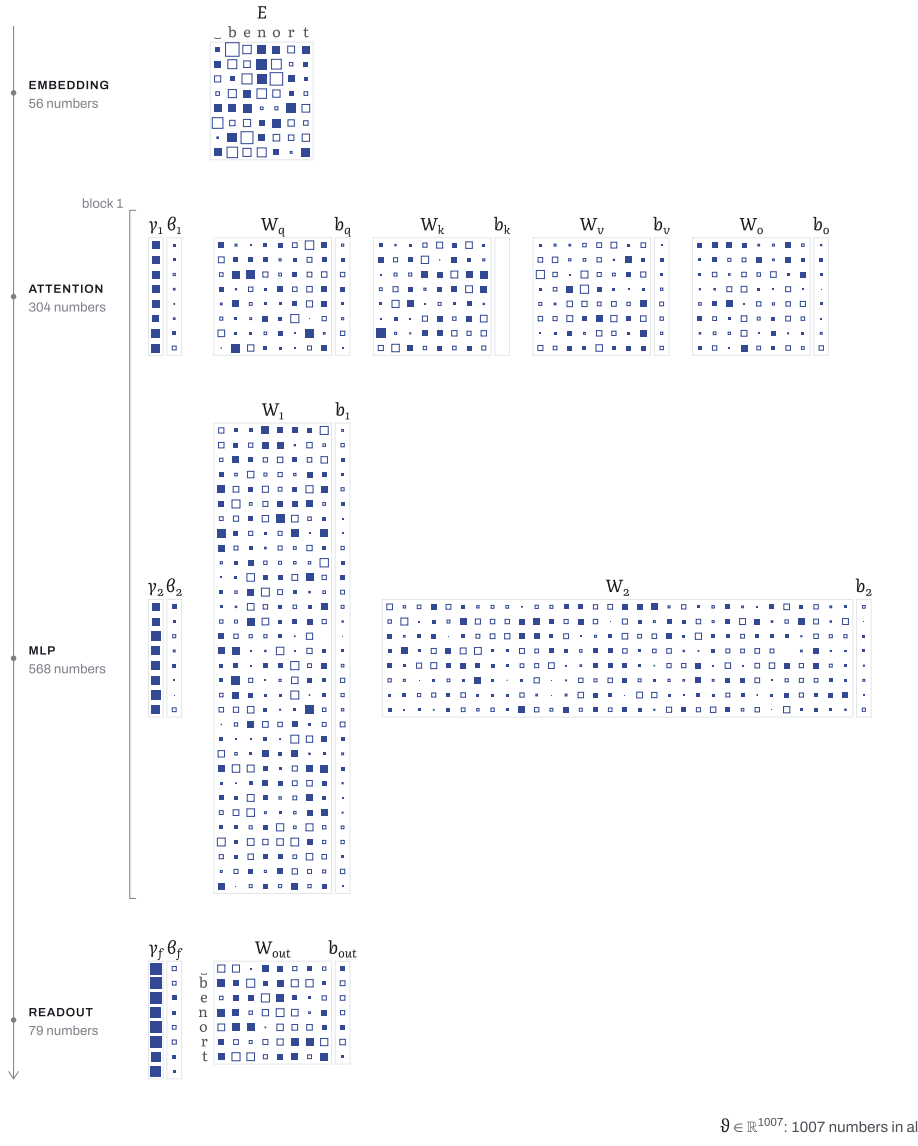
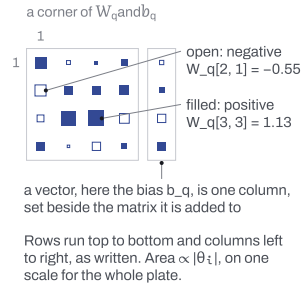


Figure 0.1 The whole specimen: all of its parameters, after training, in the order of the computation from top to bottom. Each matrix is drawn as written, rows top to bottom and columns left to right, with one square per entry whose area is the entry's magnitude, filled when it is positive and open when it is negative; vectors are single columns. One scale serves the whole plate. Point at a block to learn its name and job, or at a square to read its value.

HOW TO READ



0.3 How to read this book

Three habits are used throughout.

Three names. Every object is introduced as a map between named spaces, with its index form and matrix form side by side, and it is given three names: the mathematician's, the physicist's and the one used in machine learning. The names table at the start of each chapter lists the objects it defines. In it the mathematical name is set in roman, the physical one in italic, and the field's name in this face, between quotation marks: jargon is shown as jargon. The epilogue collects all of them into one dictionary.

Two inks. Besides black, this book is printed in two colours, and each has exactly one meaning. *Value ink*, an indigo, marks the quantities computed going forward: parameters and activations. *Covector ink*, a cinnabar red, marks the adjoints pulled back from the loss, the gradients, written with a bar, $\bar{x}$. The red pen marks the corrections, and gradients are corrections. Forward computation runs down the page, the backward computation runs up, and position in a text runs from left to right. Nothing else is ever coloured. Printed in one colour, the two inks stay apart by shape: value marks are solid, covector marks are barred or hatched.

Figures are numbered by chapter. "Figure 5.2" is the second figure of Chapter 5. Sections are numbered the same way, §5.3, and the margin of each section gives its number.

Keys. A glyph is a picture of numbers in which every mark stands for one number. The first time a glyph appears, its figure carries a key in the margin, headed "How to read": one instance of the glyph, enlarged and drawn from the figure's own data, with hairline leaders to plain notes. A key explains how to read the picture, never what it shows. After its first appearance a glyph is used without one.

0.4 Two editions

The book exists in two forms made from one source. The web edition is interactive: its figures run the specimen in the page, and they respond to the hand. You can drag a window along the text, click a character, move a temperature, or scrub through a computation with the playhead under the figure. The print edition is the same text and the same figures, typeset for paper; each figure appears in a static state that is complete on its own, and computations that unfold in time are printed as small multiples of their

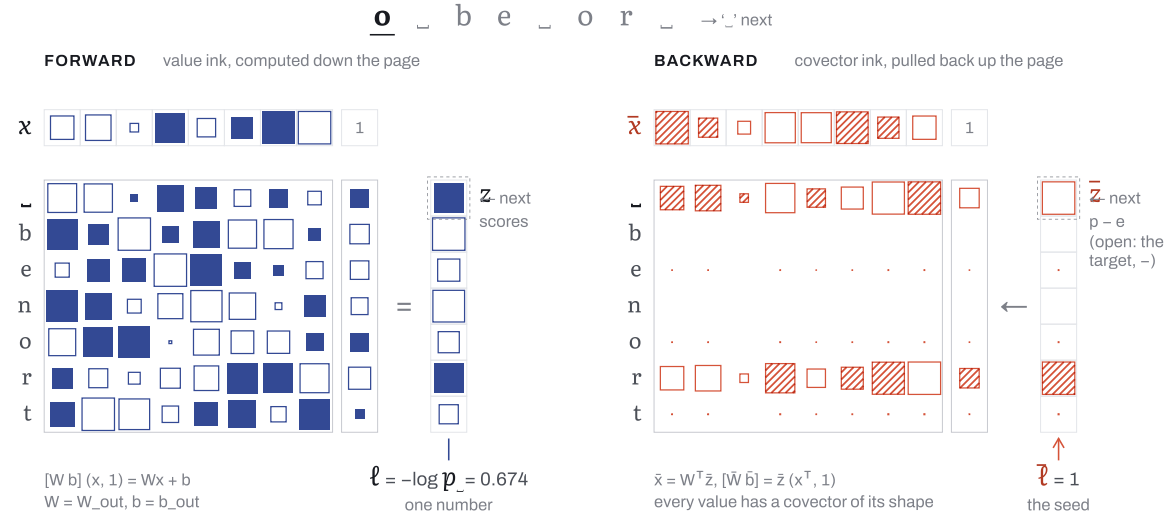
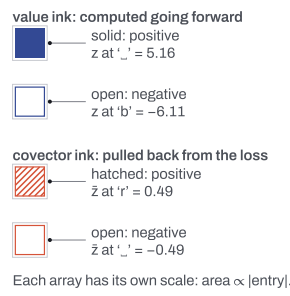


Figure 0.2 The two inks on the last map of the model, $z = W_{\text{out}}x + b_{\text{out}}$, applied to one column. Forward, in value ink: the column x enters at the top, and the matrix turns it into seven scores, one per character, from which the loss of this one prediction is computed. Backward, in covector ink: the loss returns its adjoint $\bar{z}$, and the transpose sends it back to $\bar{x} = W_{\text{out}}^{\mathsf{T}}\bar{z}$; the matrix receives $\bar{W}_{\text{out}} = \bar{z}\,x^{\mathsf{T}}$. Scrub the computation, or click another character of the window.



stages. Nothing is held back from print: interaction is only for asking the next question.

0.5 Running the code

The code is a Julia package of a few hundred lines that uses nothing but the standard library. Each file belongs to a chapter, in the order of the book, and every operation returns its value together with its pullback. No automatic differentiation is used anywhere; Chapter 7 derives each pullback by hand, and the tests check every one of them against finite differences.

The code shown in this book is cut from the source files when the book is built, never copied by hand, and the line numbers in its margin are the real line numbers of the file. Here is the parameter count above, as the code computes it:

src/parameters.jl nparams · lines 34–43

```
"""
    nparams(c) = 2VD + V + 2D + L(4D² + 2DF + 9D + F)

The number of parameters p. Embedding and readout give 2VD + V, the final LayerNorm 2D;
each block has four D×D attention matrices, F×D and D×F MLP matrices, and 9D + F gains and biases.
"""
function nparams(c::Config)
    (; V, D, F, L) = c
    2V*D + V + 2D + L * (4D^2 + 2D*F + 9D + F)
end
```

To run it, install Julia (1.10 or later) and, from the root of the repository,

```
julia --project -e 'using Pkg; Pkg.test()'     # every pullback against finite differ
julia --project scripts/export_specimen.jl     # train the specimen; write artifacts/
```

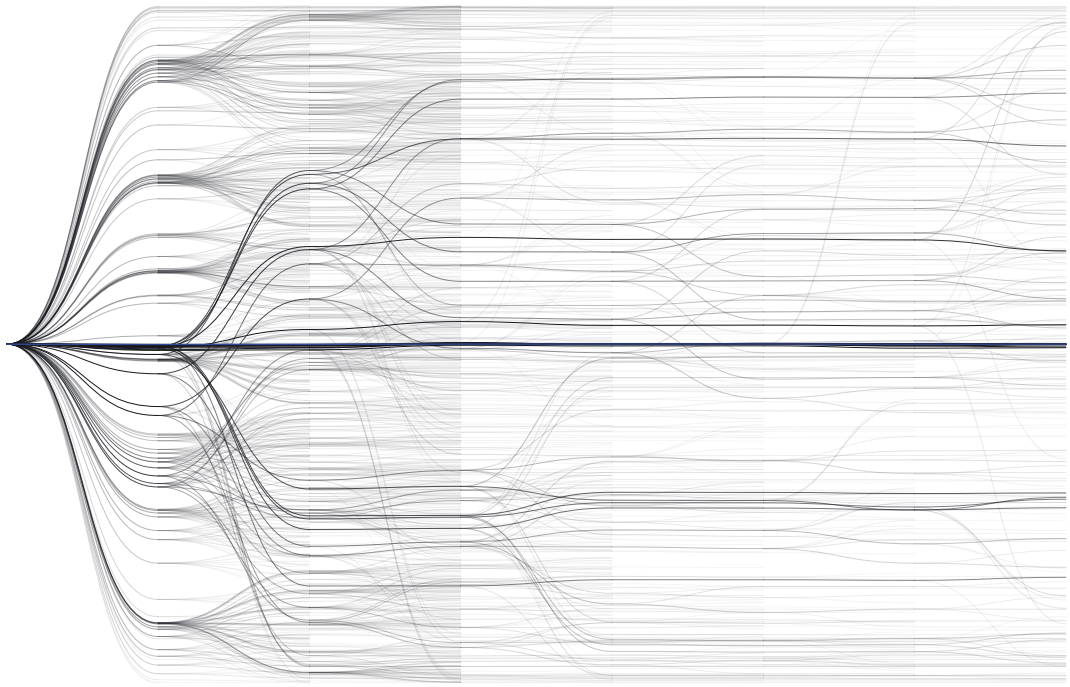
The second command trains the specimen exactly as this book shows it, and writes every number the figures use. The browser has its own implementation of the same model, in JavaScript, which reproduces the Julia numbers to twelve decimal places; it is what makes the web figures live.

Exercises

EXERCISE 0.1 Check the parameter count. Attribute each term of $p = 2DV + V + 2D + L(4D^2 + 2DF + 9D + F)$ to the blocks of Figure 0.1 (p. 9), and verify $p = 1007$ for the specimen. Which terms grow fastest with D?

EXERCISE 0.2 Try it in Figure 0.1 (p. 9). One block of the plate is blank, although the model has been trained. Which is it? Write down a guess for why, and check it against Chapter 5.

EXERCISE 0.3 Try it in Figure 0.2 (p. 11). Click through the characters of the window. For most of them the covector marks almost vanish. What does a vanishing $\bar{z}$ say about the prediction, and at which positions is it large?



Chapter 1. *Every continuation of "t" the specimen considers, from untrained to trained.* Columns are positions 1 to 8. Each string's probability $P(x_2 \ldots x_k \mid \text{"t"}) = \prod P(\text{next} \mid \text{prefix})$ is the length of its own interval of the vertical axis, nested inside its prefix's interval; edges join each string to its one-character extensions, inked in proportion to probability^0.6 (edges too faint to see are omitted). 38 moments of training (AdamW, η 0.02 → 0.002 (warmup 20, cosine), β_2 0.95, weight decay 0.01 on matrices, clip 1) are overlaid. Value ink: the final model, which has learned "to be or".

CHAPTER ONE

1 The task

A text is a finite sequence over a finite alphabet. By the chain rule of probability, a distribution on texts is the same thing as a rule that sends every context to a distribution over the next symbol, and a language model is such a rule.

MATHS	PHYSICS	ML
a finite set $\mathscr{A}$ with $V = \lvert\mathscr{A}\rvert$ elements	*the single-site state space*	"vocabulary"
$a \in \{1, \dots, V\}$, the index of a symbol	*a label of a state*	"token, token id"
$x = (x_1, \dots, x_m) \in \mathscr{A}^m$	*a configuration, a history*	"sequence, text"
the prefix $x_{\le t} = (x_1, \dots, x_t)$	*the past*	"context, prompt"
$q(\,\cdot \mid x_{\le t}) \in \Delta^{V-1}$	*a conditional law, a transition kernel*	"next-token prediction"
$P(x) = \prod_t q(x_{t+1} \mid x_{\le t})$	*a chain of conditional experiments*	"autoregressive model"
the largest t the map accepts, T	*the length of the memory*	"context length, context window"
$-\log q(x_{t+1} \mid x_{\le t})$	*surprisal*	"per-token loss"
the pairs $(x_{\le t}, x_{t+1})$ we fit	*the sample*	"training set"

1.1 Text as a finite sequence

COLLECT THE DISTINCT CHARACTERS OF A TEXT AND SORT them. That is the *alphabet* $\mathscr{A}$, a finite set with V elements, and

from here on its elements are known only by their position in the sorted list, $a \in \{1, \dots, V\}$. A text of length m is then a point $x \in \{1, \dots, V\}^m$: a word of m letters over V symbols, nothing more. The model never sees a character, only the integer that stands for it.

The space is a symbol like any other. Figures draw it as ␣ so that it can be seen.

The book's specimen is trained on one line of $m = 18$ characters, `to be or not to be`. Its alphabet has $V = 7$ symbols, the space first because it sorts first: ␣ b e n o r t. As a word over $\{1, \dots, 7\}$ the line begins $(7,5,1,2,3,1,5,6, \dots)$. The numbering is arbitrary and means nothing: the model will have to learn that r may follow o, and the fact that they are numbered 5 and 6 does not help it. Chapter 2 makes sure that the numbers cannot even suggest it.

1.2 The chain rule

Let P be any probability distribution on $\mathscr{A}^m$. Applying the definition of conditional probability $m - 1$ times gives

$$P(x_1, \dots, x_m) = P(x_1) \prod_{t=1}^{m-1} P(x_{t+1} \mid x_1, \dots, x_t).$$

This is the *chain rule of probability*. It assumes nothing: no independence, no Markov property, no stationarity. It only says that one may draw a text one symbol at a time, each from its distribution given everything before it.

The converse is just as true, and it is the one we use. Take *any* family of distributions $q(\,\cdot \mid c)$ on $\mathscr{A}$, one for every context c. The product $P(x) = q(x_1) \prod_t q(x_{t+1} \mid x_{\le t})$ is non-negative, and it sums to one: summing over x_m removes the last factor, then summing over x_{m-1} removes the next, and so on down to x_1. So a distribution on texts and a rule "context $\mapsto$ distribution of the next symbol" are the same object, written in two ways. A model of the second kind is called *autoregressive*: it writes by feeding its own output back in as the next context.

1.3 A language model is a map

Write $\Delta^{V-1} = \{p \in \mathbb{R}^V : p_a \ge 0,\ \sum_a p_a = 1\}$ for the probability simplex: the distributions on the alphabet. A *language model* with context length T is a map

$$q_\theta : \bigcup_{t=1}^{T} \mathscr{A}^t \longrightarrow \Delta^{V-1}, \qquad x_{\le t} \longmapsto q_\theta(\,\cdot \mid x_{\le t}),$$

from contexts of up to T symbols to distributions over the next one. The subscript $\theta \in \mathbb{R}^p$ is the list of numbers that fixes which map we mean: the *parameters*. Everything in the rest of the book is about how q_θ is computed from θ (Chapter 2 to Chapter 6), and how θ is chosen (Chapter 4 to Chapter 8).

Cutting the context at T symbols is a modelling decision, not a consequence of the chain rule; for the specimen, $T = 8$. A context of t symbols is a point of a finite set with V^t elements, so in principle q_θ is a table. The table is big. A transformer is a way of writing it down with far fewer numbers than it has entries, in a form that can be adjusted a little at a time.

Figure 1.1 shows the trained specimen's map on the eight prefixes of one window. Each column is a context, growing by one character, and below it the distribution the model assigns to the character that comes next.

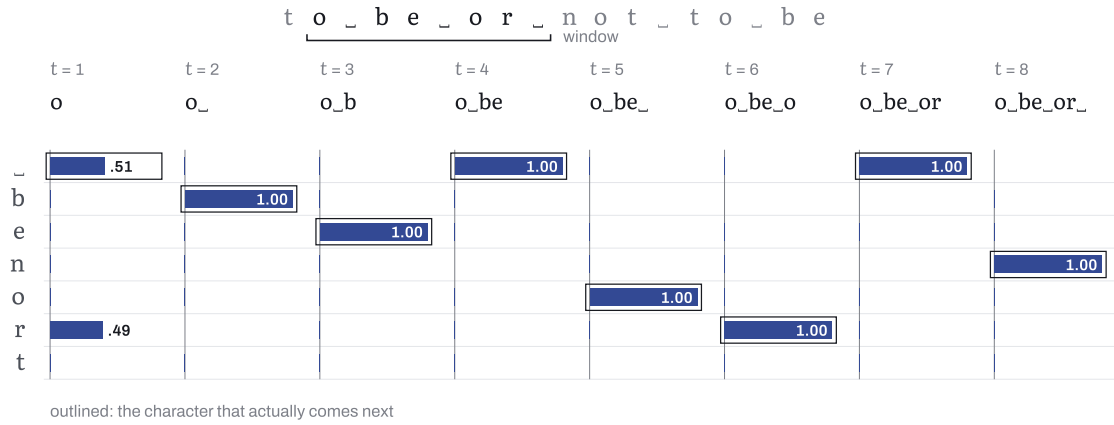


Figure 1.1 The specimen as a map from contexts to distributions. Column t is the context $x_{\leq t}$, the first t characters of the window; below it are the seven probabilities $q_\theta(a \mid x_{\leq t})$, one bar per character a, in the order of the alphabet. The character that actually comes next is marked. Drag the window along the line.

Most columns are a single bar of height one: after `o be o`, the specimen is sure that `r` comes next, and it is right. The first column is different. Given only `o`, the specimen splits its mass almost evenly between ␣ and `r`. That is not a mistake, and we come back to it in §4.5 (p. 51).

1.4 The probability of a text

Given a model, the chain rule assigns a probability to every text. For a window of T characters followed by one more, the model gives the last T of them, each conditioned on what precedes it:

$$q_\theta(x_2, \ldots, x_{T+1} \mid x_1) = \prod_{t=1}^{T} q_\theta(x_{t+1} \mid x_{\le t}).$$

The first character is given, never predicted: there is nothing yet to predict it from. Products of many numbers below one are awkward, so we take logarithms. The *surprise* of one prediction is

$$s_t = -\log q_\theta(x_{t+1} \mid x_{\le t}) \in [0, \infty],$$

measured in *nats* because the logarithm is natural. A certain and correct prediction costs nothing; a prediction that gave the truth probability $1/V$, as uniform guessing does, costs $\log V$; a prediction that ruled the truth out costs infinitely much. The surprises of a text add up, and the probability of the text is the exponential of minus their sum.

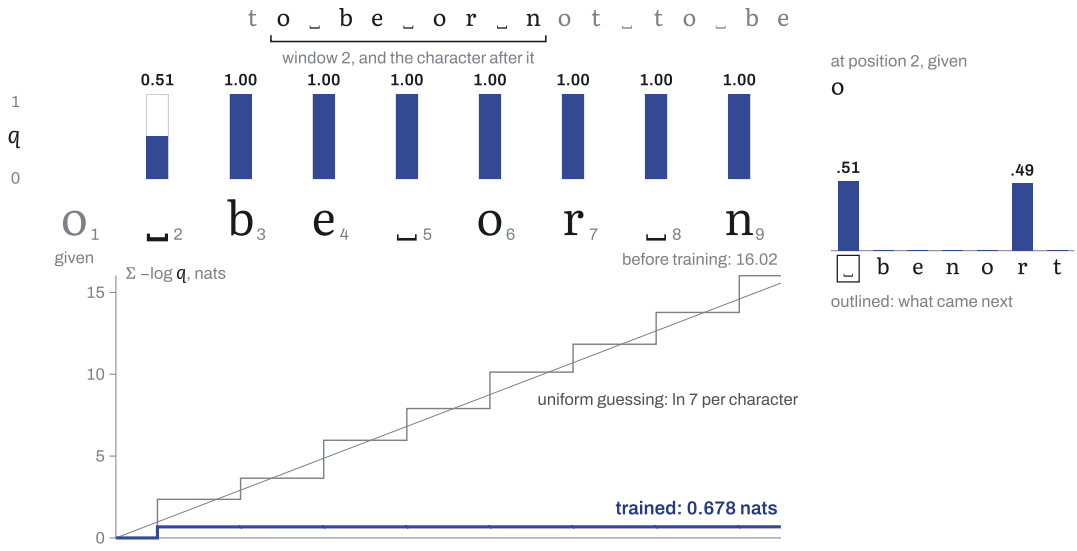


Figure 1.2 The chain rule as a staircase. Above each character x_{t+1} of the window is the probability the model gave it in its context, $q_\theta(x_{t+1} \mid x_{\le t})$; the staircase below accumulates the surprises $-\log q_\theta$, and its height at the end is minus the log-probability of the whole text. A hairline marks what uniform guessing would cost. Click a character to see the whole distribution it was drawn from; drag the window.

The mean surprise per prediction, averaged over all the predictions we have, is the number that training will make small. Chapter 4 derives it properly, as a cross-entropy, and shows that making it small is maximum likelihood.

The unit matters little: in bits, using $\log_2$, every surprise is multiplied by $1/\ln 2$. Nats keep the derivatives clean.

1.5 The training set

The model accepts at most T symbols, so we cut the corpus into *windows*: every run of T consecutive characters that has one more character after it. For a window starting at s, the *input* is $x_s, \ldots, x_{s+T-1}$ and the *target* is the same run shifted by one, $x_{s+1}, \ldots, x_{s+T}$. Position t of the window is asked for the character that follows its prefix. A corpus of m characters has $m - T$ windows; the specimen's line has ten.

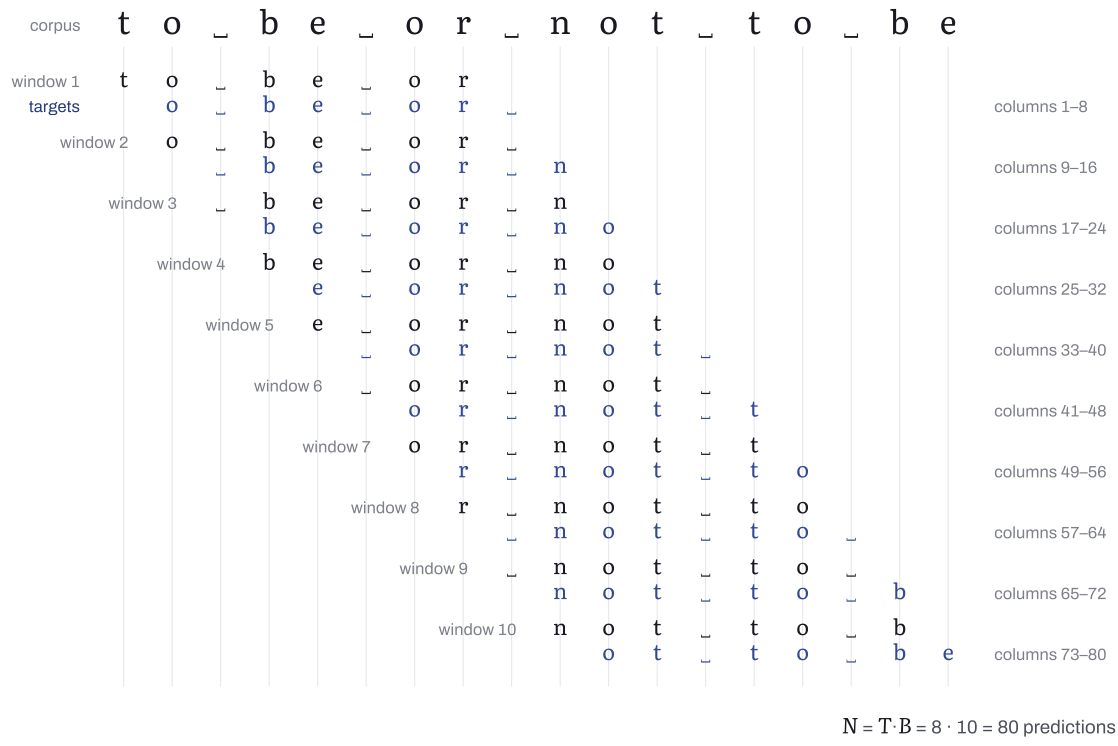


Figure 1.3 The whole training set. The corpus is at the top; below it, each row is one window, its inputs in ink and, offset by one place, its targets in value ink. Every target is one prediction, and the ten windows of eight make the $N = T \cdot B$ columns that the model processes at once.

One window yields T predictions, not one. A single pass of the model computes all T distributions of a window together, each conditioned only on its own prefix; Chapter 5 shows how the model keeps position t from reading positions after t, so that the targets are not visible to the predictions they test. Stacking the B windows side by side gives the $N = T \cdot B$ columns of the arrays in every later chapter. For the specimen, $B = 10$ and $N = 80$.

The windows overlap, so the same stretch of text is predicted several times, from contexts of different lengths. That is deliberate. It also means that the objective is an average over these 80 conditional predictions, not the probability of the line counted once.

1.6 What no model can learn

Could a model make every surprise zero? Only if every context determines its next character. Look at the first position of each window, where the context is a single character. Two windows begin with `o`: in one the next character is ␣ (from `to be`), in the other it is `r` (from `or`). Three begin with ␣, followed by `b`, `o` and `n`. A model sees the same context in each case, so it must give the same distribution, and no distribution puts probability one on two different characters.

The floor depends on the cut as well as on the text: windows of another length see other contexts, and a longer line has other ambiguities. Exercise 1.3 asks for one.

The best a model can do with a context c that occurs n_c times, followed $n_{c,a}$ times by a, is to predict the observed frequencies $q(a \mid c) = n_{c,a}/n_c$. (That this is optimal is Gibbs' inequality, proved in Chapter 4.) The mean surprise is then the entropy of the text's own next-character statistics,

$$H_{\min} = \frac{1}{N} \sum_{c} \sum_{a} n_{c,a} \log \frac{n_c}{n_{c,a}}.$$

Every other context of the specimen's windows is followed by one character only, and contributes nothing. The two windows that start with `o` each cost at least $\log 2$, the three that start with ␣ each cost at least $\log 3$, and so

$$H_{\min} = \frac{2 \log 2 + 3 \log 3}{80} \text{ nats}.$$

It is a property of the text and of the way we cut it, not of any model. A perfect specimen would sit exactly on this floor, and Chapter 8 shows how close training gets.

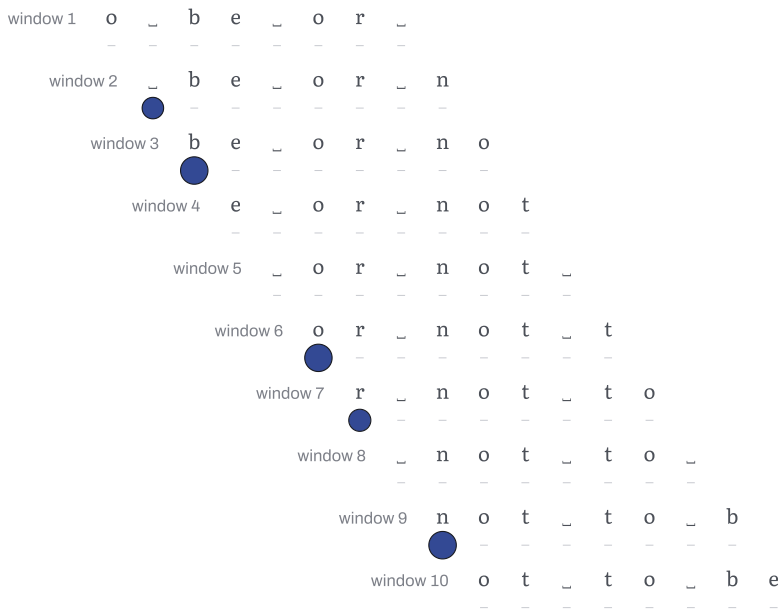


Figure 1.4 The floor under the loss. The 80 predictions of the training set, laid out as in Figure 1.3 (p. 19); each is drawn at the smallest surprise any model can achieve on it. Only five are not free, all at the first position of a window. Point at one to see which characters share its context.

1.7 In the code

The alphabet and the change of names between characters and integers are three short functions:

src/text.jl alphabet · line 11

```
alphabet(text) = sort(unique(text))
```

src/text.jl encode · lines 18–21

```
function encode(text, alphabet)
    number = Dict(ch => a for (a, ch) in enumerate(alphabet))
    [number[ch] for ch in text]
end
```

and the windows, with their targets shifted by one, are one line:

src/train.jl windows · lines 15–16

```
windows(data, T, starts = 1:length(data) - T) =
    ([data[i + t - 1] for t in 1:T, i in starts], [data[i + t] for t in 1:T, i in starts])
```

Column b of `x` is window b, so a batch of B windows is a $T \times B$ array of integers, read down its columns, as every array in this book is.

Exercises

EXERCISE 1.1

A table that lists $q(\,\cdot \mid c)$ for every context c of length 1 to T has $V - 1$ free numbers per context. Show that it has $V^{T+1} - V$ free numbers in all, and evaluate this for the specimen. How does it compare with the specimen's parameter count in Chapter 0?

EXERCISE 1.2

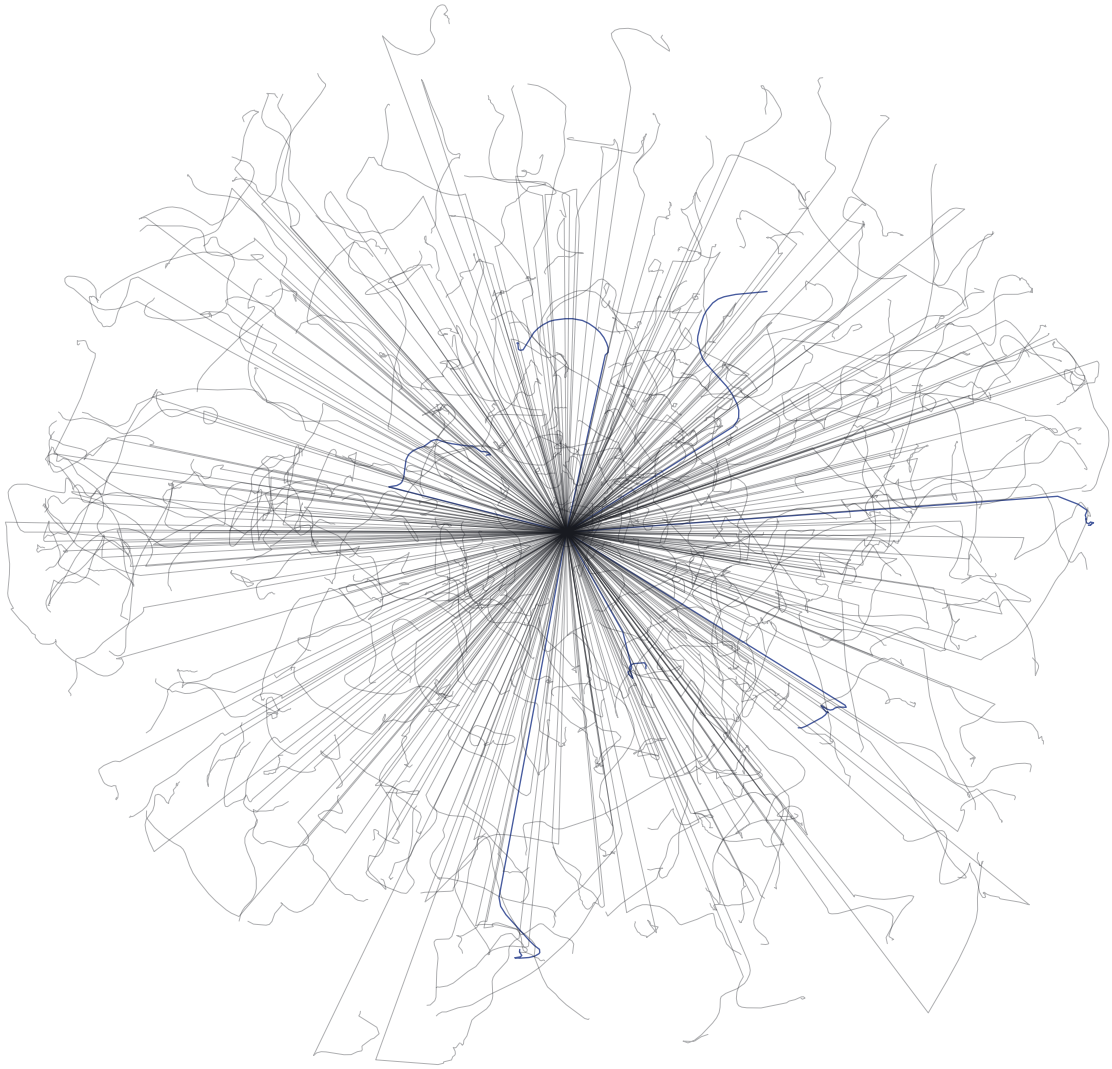
A *bigram* model is a language model whose prediction depends on the last symbol only, $q(\,\cdot \mid x_{\le t}) = q(\,\cdot \mid x_t)$. How many free numbers does it have? Find the bigram model with the smallest mean surprise on the specimen's 80 predictions, and show that its mean surprise is larger than $H_{\min}$. Which contexts does a bigram model confuse that the specimen does not?

EXERCISE 1.3

Show that $H_{\min} = 0$ exactly when every context in the training set is followed by one character only. Then compute $H_{\min}$ for the line `to be or not to be or` with $T = 8$. Which contexts are now ambiguous?

EXERCISE 1.4

Try it in Figure 1.2 (p. 18). Find a window in which the trained model's staircase is not flat. At which position does it rise, and by how much, compared with the floor of Figure 1.4 (p. 21)?



Chapter 2. *The seven embedding vectors, followed through training, in 64 independent runs.* Each path is the displacement E e_a(t) − E e_a(0) ∈ $\mathbb{R}^8$ of one character's embedding over 200 steps (AdamW, η 0.02 → 0.002 (warmup 20, cosine), β_2 0.95, weight decay 0.01 on matrices, clip 1; seeds 1–64), projected onto the plane of the two leading singular vectors of that run's final displacement and scaled so its longest final displacement reaches the rim. In value ink: one run (seed 1).

CHAPTER TWO

2 Symbols become vectors

Character a becomes the column Ee_a of a learned matrix $E \in \mathbb{R}^{D \times V}$, position t becomes a fixed vector of sines and cosines on which a shift acts as a rotation, and their sum is the first state of the model: one column of $\mathbb{R}^D$ per token.

MATHS	PHYSICS	ML
a standard basis vector $e_a \in \mathbb{R}^V$	*a pure state, an occupation-number basis vector*	"one-hot vector"
$E \in \mathbb{R}^{D \times V}$, column a is Ee_a	*a coupling of each symbol to D modes*	"embedding matrix, token embedding"
$x \mapsto E[:, x] = EC$, with C one-hot	*evaluating a linear map on basis states*	"embedding lookup"
the dimension D of the columns	*the number of modes*	"model dimension, d_{model}, width"
$P_t = \left(\sin \omega_r t, \cos \omega_r t\right)_r$, fixed	*phases of $D/2$ oscillators*	"sinusoidal positional encoding"
$X_0 = E[:, x] + P \in \mathbb{R}^{D \times T}$	*the initial condition*	"input embeddings, residual stream at layer 0"

2.1 One-hot vectors

THE INTEGERS $1, \ldots, V$ THAT NAME THE SYMBOLS ARE labels, not quantities. Nothing about the model should depend

on the fact that o is numbered 5 and r is numbered 6, or on any arithmetic of the labels. So we send symbol a to the standard basis vector

$$e_a \in \mathbb{R}^V, \qquad (e_a)_b = \delta_{ab}.$$

Every symbol is a unit vector and any two are orthogonal: $e_a \cdot e_b = \delta_{ab}$, so $\|e_a - e_b\| = \sqrt{2}$ for every pair $a \neq b$. No symbol starts closer to one than to another. Relabelling the alphabet permutes the basis and changes nothing else. In the language of the field, e_a is a *one-hot* vector: one coordinate is hot, the others cold.

A one-hot vector is a terrible representation to compute with, and a perfect one to compute *from*. A linear map $E : \mathbb{R}^V \to \mathbb{R}^D$ is determined by what it does to the basis, and on e_a it simply returns its column a.

2.2 The embedding matrix

The model's first parameters are a matrix $E \in \mathbb{R}^{D \times V}$, whose columns are indexed by the alphabet. Symbol a is sent to

$$Ee_a = E_{:,a} \in \mathbb{R}^D, \qquad (Ee_a)_i = \sum_b E_{ib}\,\delta_{ba} = E_{ia}.$$

That is the whole of the *embedding*: column a of E is what the model means by the character a. For a sequence $x_1, \ldots, x_T$, collect the one-hot vectors as the columns of $C = (e_{x_1}, \ldots, e_{x_T}) \in \{0,1\}^{V \times T}$; then

$$EC = \left(E_{:,x_1}, \ldots, E_{:,x_T} \right) = E[:, x] \in \mathbb{R}^{D \times T}.$$

In matrix form the lookup is a product with a one-hot matrix; in code it is indexing, and C is never formed. The specimen has $D = 8$ and $V = 7$, so E holds 56 of its parameters. Training adjusts them like all the others.

Why should a character be a point in $\mathbb{R}^8$, when $\mathbb{R}^7$ would hold seven orthogonal vectors exactly? Because every later map is linear in these columns, or close to it, and the model needs room: in D dimensions it can place the characters where its later maps find them useful, and D is chosen once for the whole model, not for the alphabet.

The vectors in this book are drawn as spoke glyphs: component k is a spoke at angle $2\pi(k-1)/D$ from twelve o'clock, clockwise, with length proportional to $|x_k|$ and a filled tip for a positive component, an open one for a negative. The key in the margin is drawn from the figure's own vector. All the glyphs of one figure share one scale, so lengths can be compared across a figure but not between figures.

What does training do to E? Figure 2.2 (p. 28) shows the matrix before and after. At the start its entries are drawn

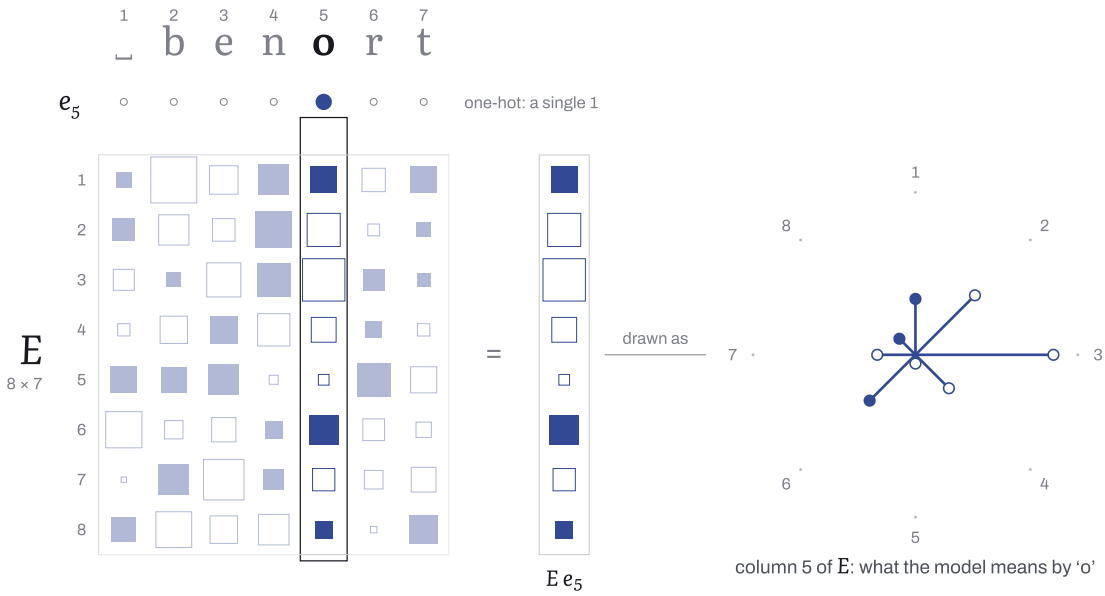
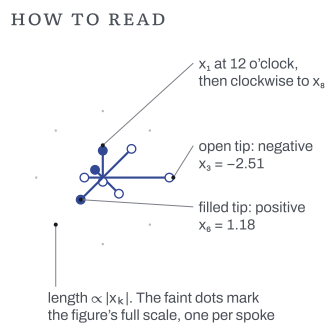


Figure 2.1 A character becomes a vector. The alphabet is a row of seven slots, and the one-hot vector e_a fills one of them; the embedding returns column a of E, drawn as a spoke glyph with one spoke for each of its eight components. Click another character.



independently from a standard normal distribution: seven random directions in $\mathbb{R}^8$, which are already far from one another. Training moves them, but not far. The specimen learns its text mostly in the maps that come later, which see the columns of E only through the linear maps they apply to them.

2.3 Positions as Fourier features

The column Ee_{x_t} does not know where it stands. The maps of Chapter 6 act on each column separately, and attention, the only map in which columns interact, weights them by scores between columns; without more information it would treat the past as an

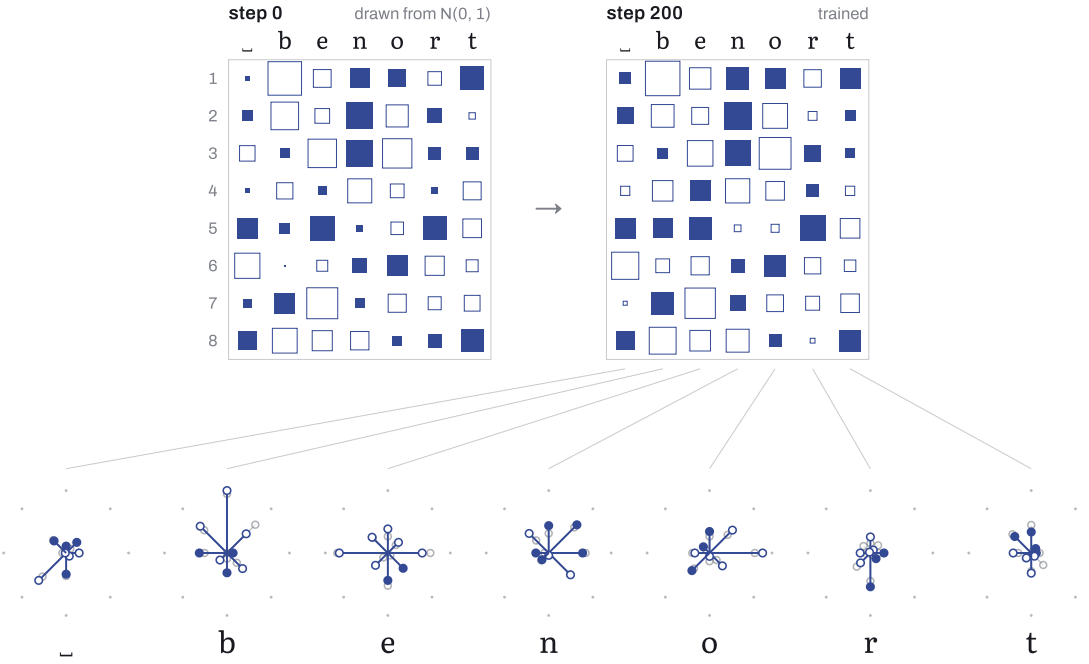


Figure 2.2 The embedding matrix $E \in \mathbb{R}^{8\times 7}$ during training. Each column belongs to a character, drawn as squares whose area is $|E_{ia}|$, filled where $E_{ia} > 0$, and again as a spoke glyph. Scrub through the 200 training steps; the faint glyphs are where the columns started.

unordered set. Chapter 5 makes this precise in an exercise: without positions, a character cannot tell the order of what came before it, and `bot` read from its `t` looks exactly like `obt`. So the model adds to column t a fixed vector P_t that encodes t.

The code is the one of the original transformer paper. Take $D/2$ frequencies in geometric progression and, for each, a sine and a cosine of the position:

$$P_{2r+1,\,t} = \sin\big(\,\omega_r(t-1)\,\big), \qquad P_{2r+2,\,t} = \cos\big(\,\omega_r(t-1)\,\big),$$

$$\omega_r = 10000^{-2r/D}, \qquad r = 0, \ldots, D/2-1, \quad t = 1, \ldots, T.$$

Position 1 is phase zero. Each pair of rows is a clock hand turning at angular velocity ω_r per step; the first turns by one radian per step, the last by about $10^{-4\cdot(D-2)/D}$. For the specimen, $D = 8$ gives four clocks, with $\omega_r = 1,\ 10^{-1},\ 10^{-2},\ 10^{-3}$. Every column has the same length, $\|P_t\|^2 = \sum_r (\sin^2 + \cos^2) = D/2$: the positions lie on a product of $D/2$ circles, a torus inside the sphere of radius $\sqrt{D/2}$.

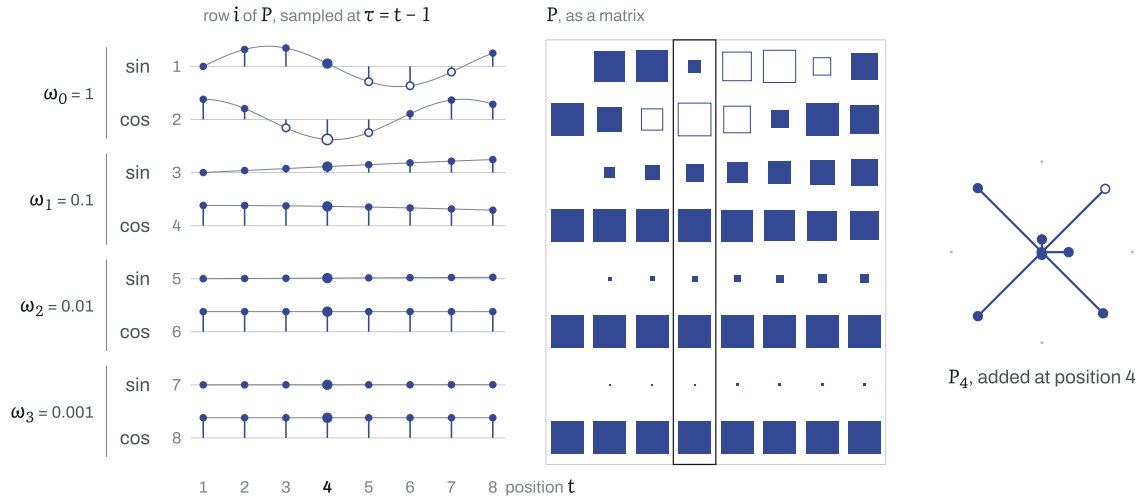


Figure 2.3 The positional code $P \in \mathbb{R}^{8\times 8}$. Each row is a sampled sinusoid: the hairline is $\sin(\omega_r \tau)$ or $\cos(\omega_r \tau)$ for continuous τ, and the stems are its values at the eight positions, the entries of P. On the right are the same numbers as a matrix, with area $|P_{it}|$, filled where positive, and column t, the vector added at position t, as a spoke glyph. Point at a position.

The fast clock tells neighbours apart; the slow ones barely move over eight positions. They are there for longer contexts: with $T = 256$, as in the Shakespeare model of Chapter 9, the slowest of its clocks still turns by less than a tenth of a radian over the whole window, and in between the clocks cover every scale of distance.

2.4 A shift is a rotation

Why sines and cosines, rather than writing t itself into a coordinate? Because a shift of position then acts linearly, and by the same map at every position. For each frequency, the angle-addition formulas give

$$\begin{pmatrix} \sin \omega_r (t+k) \\ \cos \omega_r (t+k) \end{pmatrix} = \begin{pmatrix} \cos \omega_r k & \sin \omega_r k \\ -\sin \omega_r k & \cos \omega_r k \end{pmatrix} \begin{pmatrix} \sin \omega_r t \\ \cos \omega_r t \end{pmatrix},$$

and stacking the pairs,

$$P_{t+k} = R(k)\, P_t, \qquad R(k) = \bigoplus_{r=0}^{D/2-1} \begin{pmatrix} \cos \omega_r k & \sin \omega_r k \\ -\sin \omega_r k & \cos \omega_r k \end{pmatrix}.$$

Each block is a rotation, by $-\omega_r k$ because the coordinates are ordered sine before cosine, so $R(k)$ is orthogonal, $R(0) = I$ and $R(k+\ell) = R(k)R(\ell)$: the shifts act on $\mathbb{R}^D$ as a group of rotations. The map $R(k)$ does not depend on t. A linear map applied to the code can therefore compute "k places earlier" in the same way wherever it stands.

A second consequence follows from $\sin a \sin b + \cos a \cos b = \cos(a-b)$:

$$P_s \cdot P_t = \sum_{r=0}^{D/2-1} \cos\big(\,\omega_r (t-s)\,\big).$$

Rotary position embeddings (Su et al., 2021), now used by most large language models whose design is published, take the rotation literally: instead of adding P_t, they rotate the query and key vectors of Chapter 5 by $R(t)$ before the inner product, so that a score depends on positions only through $t - s$.

The inner product of two positions depends only on their distance. The Gram matrix $P^\top P$ is constant along its diagonals: it is a Toeplitz matrix, with $D/2$ on the diagonal. It does not simply fall off with distance. The fast clock has period 2π, a little more than six steps, so on that clock positions six apart are nearly in phase again, and only the slower clocks keep them apart.

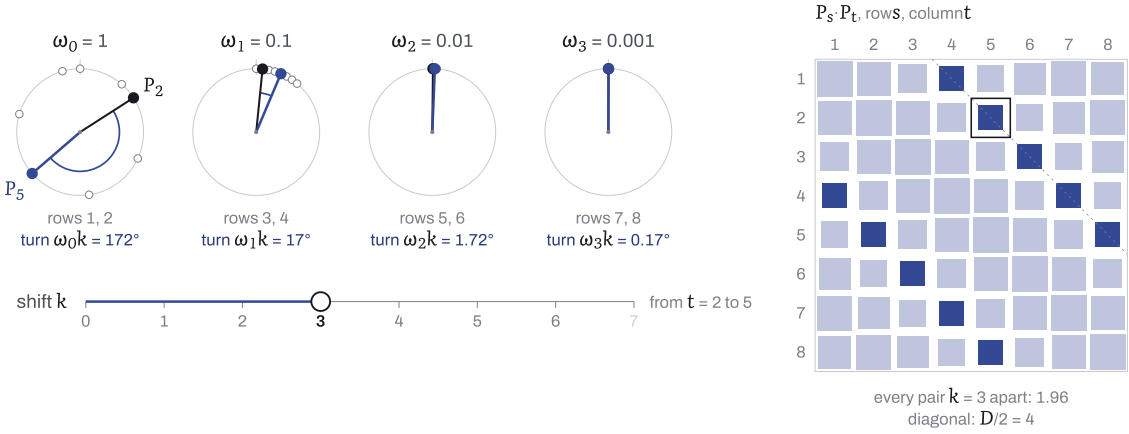


Figure 2.4 A shift is a rotation. Each circle is one frequency ω_r: the dots are the eight positions, at angle $\omega_r(t-1)$, and the hands are P_t and P_{t+k}. Shifting by k turns every hand by $\omega_r k$, whatever t is. Right: the inner products $P_s \cdot P_t$, constant along each diagonal. Drag a hand round its circle to change t, or the shift k along its scale.

None of this makes the trained model translation invariant. The model never sees P alone; it sees $E[:, x] + P$, through learned maps that can use absolute position as freely as relative position. The rotation property guarantees only that relative position is *available* as a linear function of the code.

2.5 The first state

The input to the first block is the sum of the two codes,

$$X_0 = E[:, x] + P \in \mathbb{R}^{D \times T}, \qquad (X_0)_{it} = E_{i, x_t} + P_{it}.$$

In matrix form, $X_0 = EC + P$; for a batch of windows, the same P is added to each, column by column. Addition rather than concatenation keeps the width at D: the character and the position share the same coordinates, superposed, and the learned maps that follow must separate them. The same character at two positions is now two different vectors, and the same position holds a different vector for each character.

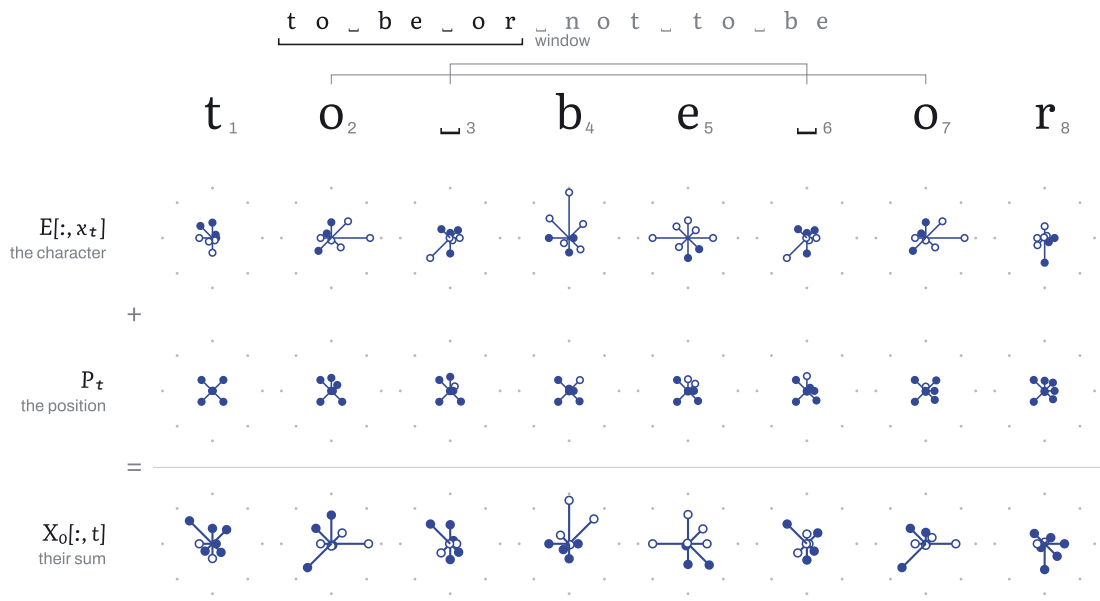


Figure 2.5 What enters the first block. For each position of the window, top to bottom: the column $E_{:, x_t}$ of the character, the positional code P_t, and their sum $X_0[:, t]$, all on one scale. Repeated characters, joined by a hairline above, share their top glyph and differ below. Drag the window along the line.

From here on, $X \in \mathbb{R}^{D \times N}$, one column per token, is called the *residual stream*, for reasons that Chapter 6 explains. Every block reads it and adds to it; nothing else ever writes to it.

2.6 The pullback, in one line

How does the loss reach E? Hold the characters x fixed: then $dX_{\mathrm{o}} = dE\,C$, and for any covector $\bar{X}_{\mathrm{o}}$,

$$\langle \bar{X}_{\mathrm{o}}, dX_{\mathrm{o}} \rangle = \operatorname{tr}\left(\bar{X}_{\mathrm{o}}^{\mathsf{T}} dE\, C \right) = \langle \bar{X}_{\mathrm{o}} C^{\mathsf{T}}, dE \rangle, \qquad \text{so} \qquad \bar{E} = \bar{X}_{\mathrm{o}} C^{\mathsf{T}}, \quad \bar{E}_{:,a} = \sum_{n\,:\,x_n = a} \bar{X}_{\mathrm{o},\,:,\,n}.$$

Column a of $\bar{E}$ collects the adjoints of every token that is the character a. The adjoint of a lookup is a *scatter-add*. The positional code is fixed, so it receives no adjoint, and the characters themselves are integers, so they have none. Chapter 7 derives every other pullback in the same way.

2.7 In the code

The positional code is a comprehension over rows and positions:

src/embedding.jl positions · lines 19–23

```
function positions(D, T, R::Type{<:AbstractFloat} = Float64)
    D > 0 && iseven(D) && T ≥ 0 || throw(ArgumentError("positions needs positive even D and nonnegative T"))
    ω(k) = R(10000)^(-R((k - 1) ÷ 2) / (D ÷ 2))    # rows 2i-1 and 2i share ωᵢ
    [isodd(k) ? sin(ω(k) * (t - 1)) : cos(ω(k) * (t - 1)) for k in 1:D, t in 1:T]
end
```

and the embedding returns its value and its pullback, the scatter-add above:

src/embedding.jl embed · lines 37–50

```
function embed(x, E)
    D, T = size(E, 1), size(x, 1)
    X = E[:, vec(x)]                                # look up one column per token
    T > 0 || throw(ArgumentError("sequences must contain at least one token"))
    reshape(X, D, T, size(x, 2)) .+= positions(D, T, eltype(E))  # the same P for every sequence
    function back(X̄)
        Ē = zero(E)
        for (n, a) in enumerate(x)
            @views Ē[:, a] .+= X̄[:, n]
        end
        Ē
    end
    X, back
end
```

Exercises

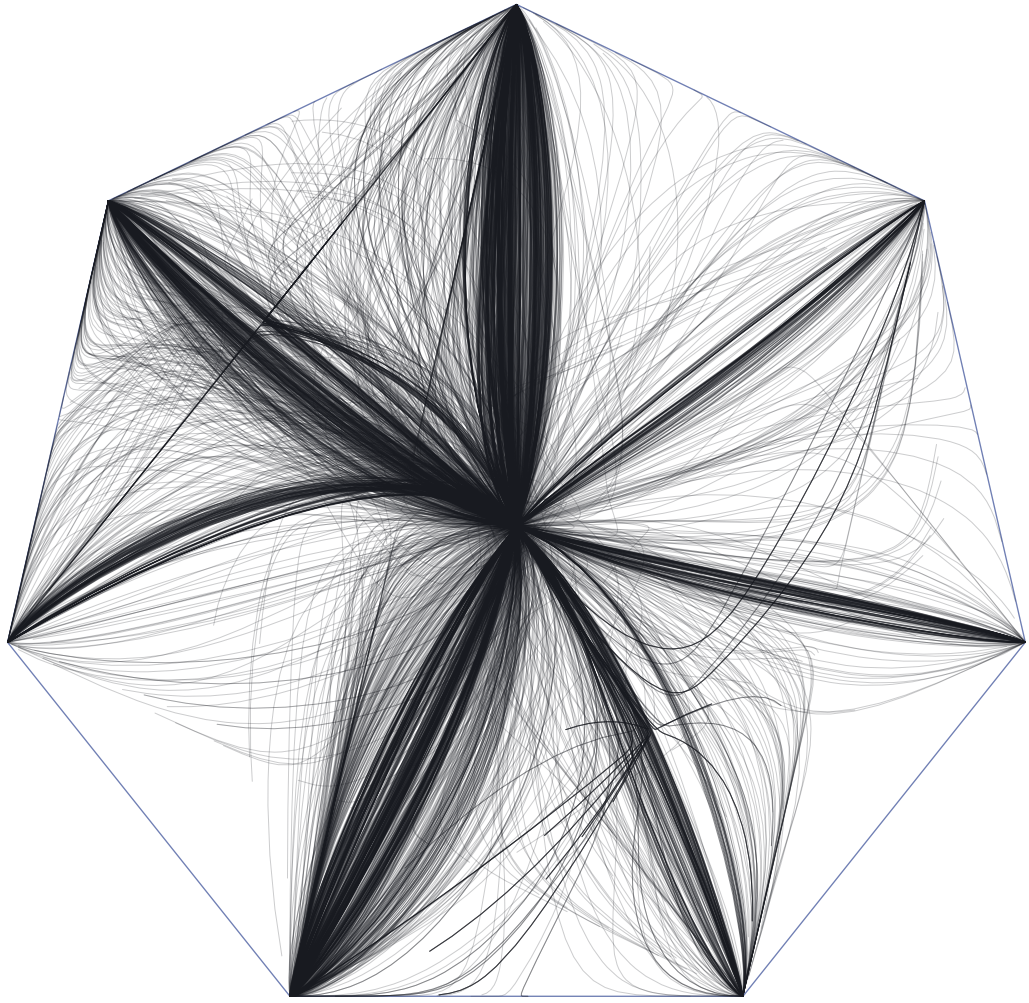
EXERCISE 2.1 Show that $\|e_a - e_b\| = \sqrt{2}$ for all $a \neq b$, and that any permutation of the alphabet acts on $\mathbb{R}^V$ as an orthogonal map. If the alphabet were instead encoded as the single number $a \in \mathbb{R}$, which pairs of characters would a linear map find closest?

EXERCISE 2.2 Prove $P_{t+k} = R(k)P_t$ and $R(k)^\top R(k) = I$. Show that $P_s \cdot P_t = \sum_r \cos(\omega_r(t-s))$, and compute $\|P_{t+1} - P_t\|^2$ for the specimen's four frequencies. Which clock contributes most?

EXERCISE 2.3 Try it in Figure 2.3 (p. 29). Over the eight positions of the specimen, the slowest clock turns by $7\omega_3$. How large is that angle, and what do its two rows of P look like? What would you lose by deleting them?

EXERCISE 2.4 Try it in Figure 2.5 (p. 31). Choose a window in which a character appears twice and compare the two columns of X_0. Then use Figure 2.4 (p. 30) to find the two distinct positions of the specimen whose codes have the largest inner product, and explain the answer with the period of the fast clock. Why can $P_s \cdot P_t$ never be smaller than $-D/2$?

EXERCISE 2.5 The embedding has DV parameters. A *learned* positional code, as in GPT-2, would replace P by a parameter matrix of the same shape. How many parameters would that add to the specimen, and what property of this chapter would it give up?



Chapter 3. *The Gibbs distribution of every logit vector the specimen produced, as the temperature falls.* 3040 logit columns $z \in \mathbb{R}^7$ (80 per moment, 40 moments of training: AdamW, η 0.02 → 0.002 (warmup 20, cosine), β_2 0.95, weight decay 0.01 on matrices, clip 1). Each path is $p_\beta = \mathrm{softmax}(\beta z)$ for β from 0.01 to 25, projected linearly onto the plane with the seven characters _ b e n o r t at the corners of the heptagon (value ink, clockwise from the top). Every path starts at the uniform centre; it ends at a corner when one character wins, and on an edge or face when the text is genuinely ambiguous.

CHAPTER THREE

3 Energies and probabilities

The model ends every column with V unconstrained real numbers, the logits; read as negative energies, softmax turns them into the Gibbs distribution of statistical mechanics, so temperature, the indifference to a common shift, and log-sum-exp are all facts about a finite system in equilibrium.

MATHS	PHYSICS	ML
$z \in \mathbb{R}^V$	*negative energies* $-E_a$	"logits"
$\mathrm{softmax}(z) \in \Delta^{V-1}$	*Gibbs distribution*	"softmax output, probabilities"
$\sum_a e^{z_a}$	*partition function* $\mathcal{Z}$	"normaliser"
$\mathrm{lse}(z) = \log \sum_a e^{z_a}$	$\log \mathcal{Z} = -F$ *at* $\tau = 1$	"log-sum-exp"
$\tau = 1/\beta$	*temperature*	"temperature"
$\arg\max_a z_a$	*ground state*	"greedy choice"
$z \mapsto z + c\mathbf{1}$	*choice of the zero of energy*	"shift invariance"

3.1 The last map

THE MODEL'S FINAL MAP TAKES THE RESIDUAL STREAM, normalises each column, and applies one affine map:

$$Z = W_{\text{out}}\,\mathrm{LN}(X;\gamma_f,\beta_f) + b_{\text{out}}\mathbf{1}^{\top} \in \mathbb{R}^{V\times N}, \qquad W_{\text{out}} \in \mathbb{R}^{V\times D}.$$

Column n of Z, which we call $z \in \mathbb{R}^V$, holds one real number for each character of the vocabulary. For the specimen that is seven numbers, one each for `␣ b e n o r t`. The model's job at this column is to state a probability distribution over the next character, that is, a point of the *probability simplex*

$$\Delta^{V-1} = \left\{ p \in \mathbb{R}^V : \; p_a \geq 0, \;\; \textstyle\sum_a p_a = 1 \right\}.$$

The entries of z obey no such constraint. They may be negative, and they need not sum to anything. We need a map $\mathbb{R}^V \to \Delta^{V-1}$, and we want it to be positive, normalised, increasing in each z_a, and smooth, because Chapter 7 will differentiate it. The map the model uses is

$$p_a = \frac{e^{z_a}}{\sum_{b=1}^{V} e^{z_b}}, \qquad \text{in vector form} \qquad p = \mathrm{softmax}(z) = \frac{e^{z}}{\mathbf{1}^{\top} e^{z}},$$

with the exponential taken entry by entry. The entries of z are called *logits*.

"Logit" began as the name of the log-odds function $\mathrm{logit}(q) = \log \frac{q}{1-q}$ of a probability q (Berkson, 1944). In machine learning it has drifted to mean any real number that becomes a probability after an exponential and a normalisation. Here: an entry of z.

Why this map? One could normalise any positive function of the logits. The answer is that this particular one is an old acquaintance, and nearly everything we need to know about it was worked out in the nineteenth century.

3.2 Softmax is the Gibbs distribution

Consider a physical system with finitely many states $a \in \{1, \ldots, V\}$, where state a has energy E_a. In equilibrium at temperature τ (in units where Boltzmann's constant is 1), the probability of finding it in state a is

$$p_a = \frac{1}{\mathcal{Z}}\, e^{-E_a/\tau}, \qquad \mathcal{Z} = \sum_{a=1}^{V} e^{-E_a/\tau}.$$

This is the Gibbs (or Boltzmann) distribution, and $\mathcal{Z}$ is the *partition function*. Set

$$E_a = -z_a, \qquad \tau = 1,$$

and it is softmax, symbol for symbol. A logit is a negative energy. A large logit is a deep energy level, and a deep level is heavily occupied. The model's last layer computes the energies of seven levels, and the prediction is the thermal occupation of those levels at temperature one.

Figure 3.1 (p. 38) draws exactly this, for one context of the trained specimen. Each character is a rung of a ladder at the height of its energy, and the bar along the rung is its probability. Every rung can be dragged, which changes one logit: lowering a rung fills it, and since the bars must sum to one, every other bar shrinks.

Look at the three lowest rungs in the figure. Three of the specimen's ten training windows begin with a space, and in them the space is followed by `b`, by `o` and by `n`. At the first position of a window the model sees nothing but that space, so it cannot tell the three windows apart, and it has put the three characters at almost exactly the same energy, far below the other four. A physicist would call this a threefold (nearly) degenerate ground state. In Chapter 4 we will see that this is not a failure of the model but the best it can possibly do.

The physics also tells us what p optimises. For any distribution r on the V states, write its mean energy $U(r) = \sum_a r_a E_a$ and its entropy $H(r) = -\sum_a r_a \log r_a$. A two-line computation (§3.5, p. 41) shows that the Gibbs distribution is the unique minimiser of

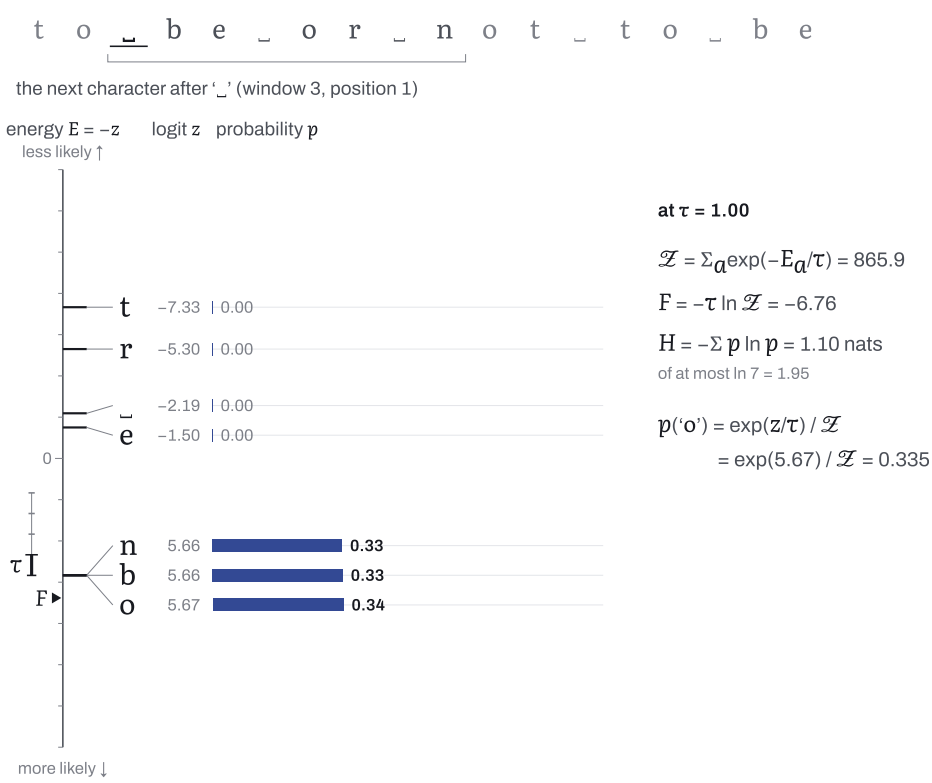
$$U(r) - \tau H(r)$$

over the whole simplex. At low temperature the minimum is dominated by the energy, so all mass goes to the lowest levels. At high temperature it is dominated by the entropy, so the mass spreads out. Softmax is the compromise between "put the probability where the logits are large" and "do not commit more than you must".

3.3 Temperature

Dividing the logits by $\tau > 0$ before the softmax gives the Gibbs distribution at temperature τ. Physicists often prefer the inverse temperature $\beta = 1/\tau$:

$$p_a^{(\tau)} = \frac{e^{z_a/\tau}}{\sum_b e^{z_b/\tau}} = \operatorname{softmax}(\beta z)_a .$$



HOW TO READ

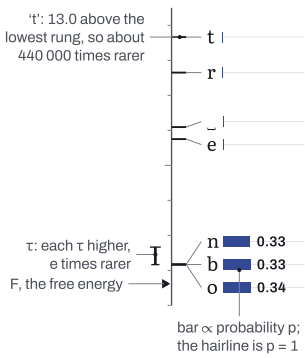


Figure 3.1 The trained specimen's prediction for the character after "␣", at the start of the window "␣be␣or␣n", drawn as an energy ladder. Each character is a rung at its energy $E_a = -z_a$, and its bar is its probability p_a. The ruler on the left stands on the lowest rung and is τ long. Drag a rung, drag the axis to shift the whole ladder, or drag the end of the ruler to change the temperature.

The ratio of the occupations of two levels is $p_a/p_b = e^{-(E_a - E_b)/\tau}$. So temperature is the unit in which energy differences are measured: a rung $k\tau$ above another is e^k times less occupied. That is what the ruler in Figure 3.1 shows. Its length is τ, and its faint ticks mark 2τ, 3τ, … above the lowest rung. Rungs far above the end of the ruler are empty.

The two limits are worth stating exactly. As $\tau \to \infty$, every weight $e^{z_a/\tau}$ tends to 1, and

$$\lim_{\tau \to \infty} p_a^{(\tau)} = \frac{1}{V},$$

the uniform distribution: the system melts. As $\tau \to 0$, let $G = \arg\max_a z_a$ be the set of lowest levels, the ground states. Dividing numerator and denominator by $e^{\max z/\tau}$ leaves weights $e^{-(E_a - E_{\min})/\tau}$, which tend to 1 on G and to 0 elsewhere, so

$$\lim_{\tau \to 0} p_a^{(\tau)} = \begin{cases} 1/|G|, & a \in G, \\ 0, & a \notin G. \end{cases}$$

The frozen limit is uniform on the ground states. It is not an arbitrary choice of one of them: tied levels share the probability equally, at every temperature.

The bottom row of Figure 3.2 (p. 40) shows what "nearly" degenerate means in practice. The three low rungs after "␣" are not exactly level. At $\tau = 1$ they share the probability almost equally, and they still do at $\tau = \frac{1}{4}$, because the gaps between them are tiny compared with τ. Only when τ falls far below those gaps does the lowest of the three take everything. The limit $\tau \to 0$ is honest: it picks the true ground state. But at any temperature you are likely to use, the three behave as one degenerate level.

The model is trained at $\tau = 1$, and inside the model the softmax always runs at the temperature it was trained at. Temperature becomes a free parameter in two other places: in attention, where the scores are divided by $\sqrt{d}$ (Chapter 5), and when the model writes text, where it is the most important dial there is (Chapter 9).

The frontispiece. The drawing that opens this chapter collects such paths for every logit column the specimen produced while it was being trained. The seven characters sit at the corners of a heptagon, in vocabulary order, clockwise from the top. Each hairline is one column z, followed from $\tau = 100$ (the centre) down to $\tau = 0.04$. A path that reaches a corner is a context with one clear successor; a path that stops on an edge or inside a face is one the text leaves open. The map from the simplex to the page is linear, so straight lines stay straight; but it squashes six dimensions into two, so it is a shadow of the simplex, not a chart of it.

3.4 Only differences matter

For every constant $c \in \mathbb{R}$,

$$\operatorname{softmax}(z + c\mathbf{1}) = \operatorname{softmax}(z),$$

because numerator and denominator both gain the factor e^c. In the physics this is the statement that energy has no absolute zero: shifting every level by the same amount is a change of convention, not of the system. Drag the axis in Figure 3.1 and the whole ladder slides, the partition function and the free energy change, and not one probability moves.

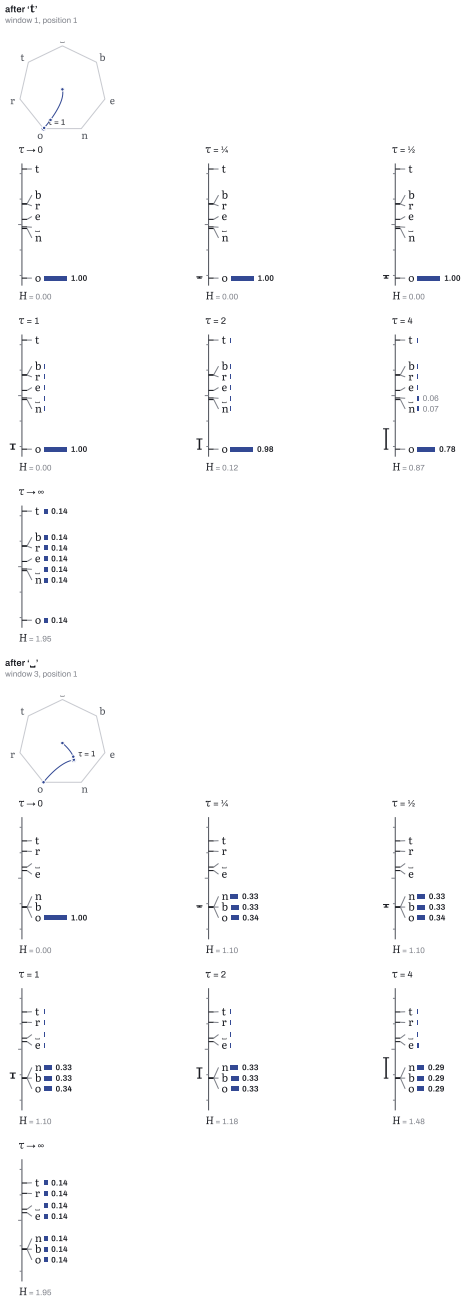


Figure 3.2 Two of the specimen's predictions, each at seven temperatures. Top: after "t", where the corpus is unambiguous; bottom: after "␣", where it is not. Each ladder is the same set of energies; only the temperature changes, and with it the occupations and the entropy H printed beneath. On the left, the same distributions as points of the simplex: the path runs from the uniform centre ($\tau = \infty$) towards the ground states ($\tau \to 0$), with the seven temperatures marked.

So the map $z \mapsto \mathrm{softmax}(z)$ is not injective. It is constant on every line $z + \mathbb{R}\mathbf{1}$, and its derivative must annihilate $\mathbf{1}$. Differentiating the quotient gives the Jacobian, in index form and in matrix form:

$$\frac{\partial p_a}{\partial z_b} = p_a(\delta_{ab} - p_b), \qquad Dp(z) = \mathrm{diag}(p) - pp^{\top}.$$

Indeed $(\mathrm{diag}(p) - pp^{\top})\mathbf{1} = p - p\,(\mathbf{1}^{\top}p) = 0$. This symmetric matrix is a covariance matrix in disguise: for any $v \in \mathbb{R}^V$,

$$v^{\top} Dp(z)\, v = \sum_a p_a v_a^2 - \Big(\sum_a p_a v_a\Big)^2 = \mathrm{Var}_{a\sim p}(v_a) \geq 0.$$

A physicist recognises the fluctuation–response relation: the response of the occupations to a change of the energies is the covariance of the fluctuations.

Shift invariance has consequences inside the model. Adding $c\mathbf{1}$ to the output bias b_{out} adds c to every logit of every column and changes no prediction. The loss therefore cannot see that direction of b_{out}, and its gradient is always orthogonal to $\mathbf{1}$; Chapter 4 shows why directly.

A puzzle to carry forward. Somewhere else in the model there is a whole vector of parameters whose only effect is to add the same number to every input of a softmax. By this section it has no effect on anything the model computes, and its gradient is exactly zero: one group of the specimen's numbers can never learn. Which group? Chapter 5 gives the answer in its exercises.

3.5 Log-sum-exp and the free energy

The logarithm of the partition function deserves its own name. At temperature $\tau = 1/\beta$ define

$$\mathrm{lse}_\beta(z) = \frac{1}{\beta}\log \sum_a e^{\beta z_a}, \qquad \mathscr{F}_\beta(E) = -\frac{1}{\beta}\log \sum_a e^{-\beta E_a},$$

so that the free energy and log-sum-exp are the same function up to signs: $\mathscr{F}_\beta(-z) = -\mathrm{lse}_\beta(z)$. We write $\mathrm{lse} = \mathrm{lse}_1$. Three facts make it the most useful function in this book.

Its gradient is the Gibbs distribution. Differentiating,

$$\frac{\partial\, \mathrm{lse}_\beta(z)}{\partial z_a} = p_a^{(\beta)}, \qquad \frac{\partial \mathscr{F}_\beta(E)}{\partial E_a} = p_a^{(\beta)}.$$

Mind the signs: with $E = -z$, the derivative of $\mathscr{F}_\beta(-z)$ with respect to z_a is $-p_a$. Calling log-sum-exp itself "the free energy" loses exactly that minus sign.

It is a smooth maximum. Factor out the largest exponential: with $m = \max_a z_a$,

$$m \;\leq\; \mathrm{lse}_\beta(z) \;\leq\; m + \frac{\log V}{\beta}.$$

As $\tau \to 0$, log-sum-exp tends to the maximum logit, and the free energy to the ground-state energy. In Figure 3.1 (p. 38) the free energy is the small triangle F on the axis. It sits below the lowest rung, never more than $\tau \log V$ below it, and it follows the ladder when the ladder slides, because $\operatorname{lse}(z + c\mathbf{1}) = \operatorname{lse}(z) + c$.

It is *a variational principle*. Substitute $\log p_a = -\beta E_a - \log \mathcal{Z}_\beta$ into the definitions of mean energy and entropy. For every distribution r,

$$\sum_a r_a E_a - \tau H(r) = \mathcal{F}_\beta(E) + \tau \operatorname{KL}(r \,\|\, p^{(\beta)}),$$

where $\operatorname{KL}(r \,\|\, p) = \sum_a r_a \log(r_a / p_a)$ is the Kullback–Leibler divergence. The divergence KL is never negative and vanishes only at $r = p$; we prove this in Chapter 4. So the Gibbs distribution is the unique minimiser of $U - \tau H$, and the minimum value is the free energy. In the logits' language, $\operatorname{lse}_\beta(z) = \max_r \{\langle r, z\rangle + H(r)/\beta\}$.

Computing it

The definition is a recipe for disaster on a computer. A Float64 number cannot exceed about 1.8×10^{308}, so e^x overflows to ∞ once x passes about 709.8, and for very negative x it underflows to 0. Logits of size 710 are rare, but a logit of 10 at temperature $1/100$ is the same thing, and then softmax computes ∞/∞, which is not a number. The cure is shift invariance itself. For any c,

$$\operatorname{lse}(z) = c + \log \sum_a e^{z_a - c}, \qquad \operatorname{softmax}(z) = \operatorname{softmax}(z - c\mathbf{1}),$$

and with $c = \max_a z_a$ every exponent is at most 0 and one of them is exactly 0. The sum then lies between 1 and V: it cannot overflow, and the logarithm never sees zero. The maximum has kinks where two logits tie, but the expression on the right is identically the smooth function on the left, so its derivative is still softmax.

The same care decides how the loss of Chapter 4 is computed: never as the logarithm of a probability, which may have underflowed to zero, but directly from the logits, as $-\log p_y = \operatorname{lse}(z) - z_y$.

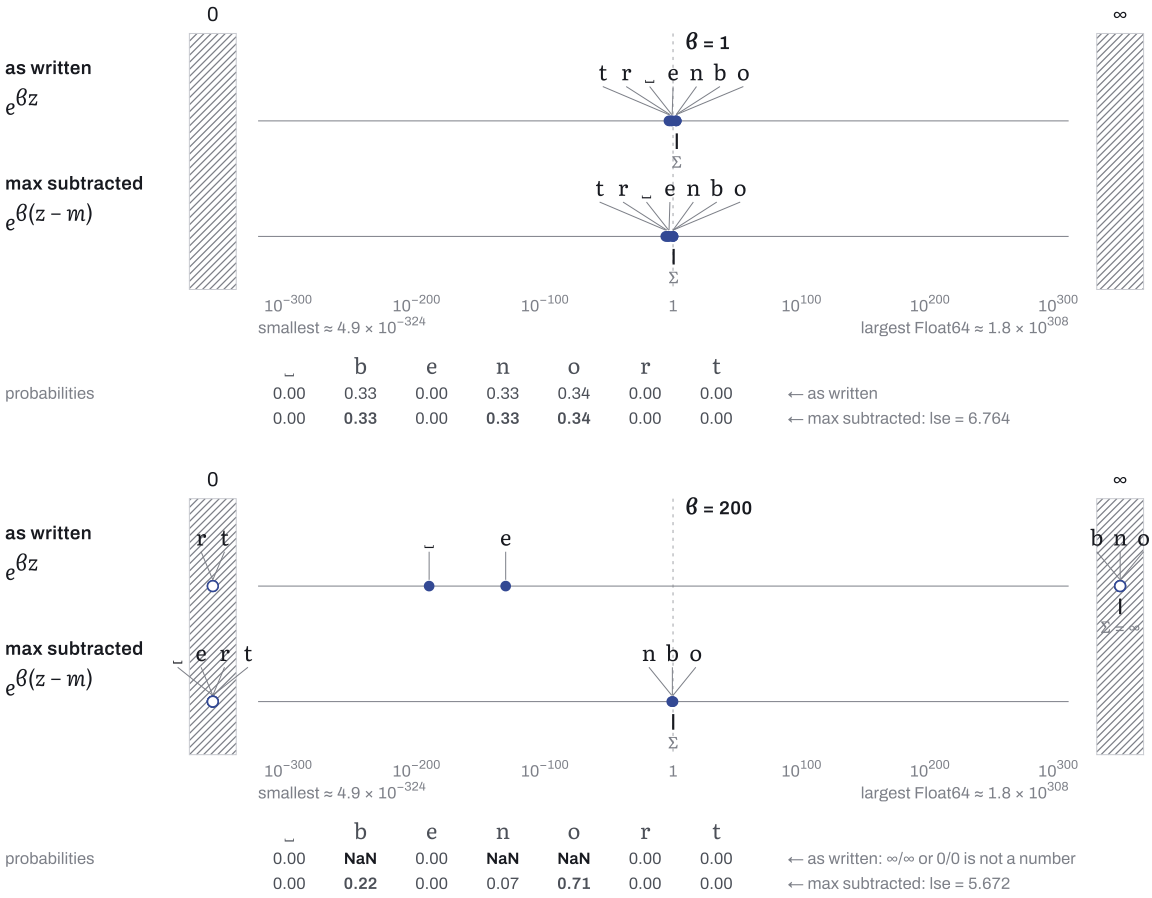


Figure 3.3 Why subtract the maximum. The seven exponentials of the logits of Figure 3.1 (p. 38), multiplied by β, on a logarithmic axis that spans the whole range of Float64. Top: computed as written, $e^{\beta z_a}$. Bottom: computed after subtracting the maximum, $e^{\beta(z_a - m)}$. Drag β to the right: the naive exponentials leave the representable range, their sum becomes ∞, and the probabilities become ∞/∞; the shifted ones never exceed 1.

3.6 In the code

Both functions are one line each:

src/softmax.jl logsumexp · line 14

```
14 logsumexp(z) = (m = maximum(z); m + log(sum(zₐ -> exp(zₐ - m), z)))
```

src/softmax.jl softmax · line 24

```
24 softmax(z; dims = :) = softmax!(float.(z); dims)
```

`softmax!` overwrites its argument, and the keyword `dims` normalises slices of an array: `dims = 1` for the columns of Z, one distribution per token, and `dims = 2` for the rows of an attention matrix. Note the subtraction of the maximum, along the same dimension.

Exercises

EXERCISE 3.1 Derive $\partial p_a / \partial z_b = p_a(\delta_{ab} - p_b)$. Show that the Jacobian is symmetric, positive semidefinite, and that its kernel is exactly $\mathbb{R}\mathbf{1}$ when every $p_a > 0$. What happens to it in the frozen limit $\tau \to 0$ with a unique ground state?

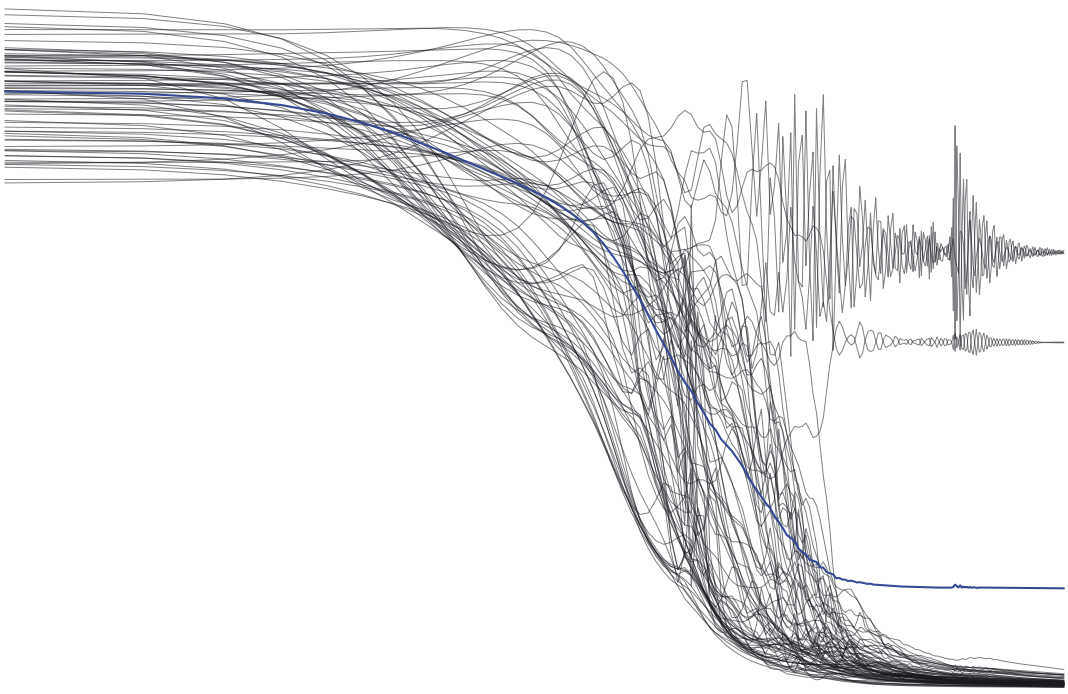
EXERCISE 3.2 Prove $\max_a z_a \le \mathrm{lse}_\beta(z) \le \max_a z_a + \log V/\beta$. When is each bound attained?

EXERCISE 3.3 Let $U(\beta) = \sum_a p_a^{(\beta)} E_a$ be the mean energy. Show that $dU/d\beta = -\mathrm{Var}_{p^{(\beta)}}(E) \le 0$: cooling never raises the mean energy. Show also that the entropy $H(p^{(\beta)})$ never increases as β grows.

EXERCISE 3.4 In Figure 3.1 (p. 38), drag one of the upper rungs down until it snaps level with the lowest one, then drag the end of the ruler down to cool the ladder. What happens to the tied pair, and to everyone else? Compare with the formula for the frozen limit.

EXERCISE 3.5 In Figure 3.1 (p. 38), drag the axis. Which of the numbers on the right change, and by how much? Explain why F must move by exactly the shift.

EXERCISE 3.6 In Figure 3.1 (p. 38), click through the positions of the window "o_be_or_". At which position is the trained model unsure of the next character, and between which characters? Find, in the corpus, the reason. (Chapter 4 makes this exact.)



Chapter 4. *The surprise of each of the specimen's 80 predictions, as it learns.* Each hairline is −log p(y) for one (context, next character) pair, over 200 steps of training (AdamW, η 0.02 → 0.002 (warmup 20, cosine), β_2 0.95, weight decay 0.01 on matrices, clip 1); time runs left to right on a logarithmic scale, surprise upward on a square-root scale from 0 to 2.67 nats. The value-ink line is their mean, the loss, which ends at 0.0589 nats: the plateaux are contexts the text itself leaves ambiguous.

CHAPTER FOUR

4 Measuring surprise

Training charges the model, for every prediction, its surprise $-\log q(y)$ at the character that actually came; the mean surprise is a cross-entropy, which exceeds the entropy of the text by exactly a KL divergence, and its gradient with respect to the logits is the prediction minus the observation.

MATHS	PHYSICS	ML
$-\log q_y$	*surprisal*	"per-token loss, negative log-likelihood"
$\mathscr{L}(\theta) = \frac{1}{N}\sum_n -\log q_\theta(y_n \mid c_n)$	*mean surprisal*	"loss, cross-entropy loss"
$H(p) = -\sum_a p_a \log p_a$	*Gibbs entropy*	"entropy"
$H(p,q) = -\sum_a p_a \log q_a$	*mean surprisal under p*	"cross-entropy"
$\mathrm{KL}(p \,\|\, q)$	*relative entropy*	"KL divergence"
$\log V$	*entropy of the uniform distribution*	"random baseline"
$\prod_n q_\theta(y_n \mid c_n)$	*likelihood*	"likelihood"
$\bar{Z} = (P - Y)/N$	*predicted minus observed occupation*	"gradient of the loss with respect to the logits"

4.1 The price of a prediction

AT EACH POSITION THE MODEL STATES A DISTRIBUTION $q \in \Delta^{V-1}$ over the next character (Chapter 3). Then the text reveals the character y that actually comes. How much should the model be charged?

With $\log_2$ instead of $\log$ the unit is the *bit*; one nat is $1/\log 2 \approx 1.44$ bits. Shannon (1948) built information theory on this quantity; the name "surprisal" is Tribus's (1961).

Three requirements pin the answer down. A prediction of certainty that comes true, $q_y = 1$, should cost nothing. The cost should grow as q_y falls. And the chain rule of probability (Chapter 1) writes the probability of a text as a *product* of next-character probabilities, so the cost of a text should be the *sum* of the costs of its characters. A continuous function $f : (0,1] \to \mathbb{R}$ with $f(st) = f(s) + f(t)$ is a multiple of the logarithm, $f(s) = -k \log s$ (Exercise 4.1, p. 55). We take $k = 1$ and the natural logarithm, and call

$$s(q, y) = -\log q_y$$

the *surprise* of the prediction q when y occurs. It is zero for an event that was certain, infinite for one that was declared impossible, $\log 2$ for a fair coin, and $\log V$ for a guess spread evenly over V characters. Its unit is the *nat*.

The model does not compute q_y and then its logarithm. Since $q = \operatorname{softmax}(z)$, Chapter 3 gives the surprise directly from the logits, with no exponential that could overflow and no probability that could underflow to zero:

$$-\log q_y = \operatorname{lse}(z) - z_y.$$

4.2 The loss

A training batch of B windows of length T makes $N = TB$ predictions; for the specimen that is $10 \times 8 = 80$. Write c_n for the context of prediction n, the characters of its window up to and including position t, and y_n for the character that follows. The *loss* is the mean surprise,

$$\mathscr{L}(\theta) = \frac{1}{N} \sum_{n=1}^{N} -\log q_\theta(y_n \mid c_n) = \frac{1}{N} \sum_{n=1}^{N} \Big[\operatorname{lse}(z_n) - Z_{y_n, n} \Big].$$

In matrix form, with $Y \in \{0,1\}^{V \times N}$ the matrix whose column n is the one-hot vector e_{y_n},

$$\mathscr{L} = \frac{1}{N} \Big[\sum_{n} \operatorname{lse}(Z_{:,n}) - \langle Y, Z \rangle_F \Big].$$

The number $\log V$ is the yardstick. A model that knows nothing and says so, $q_a = 1/V$, pays exactly $\log V$ for every prediction. The specimen is initialised with small random parameters, so its log- its start near zero and its predictions near uniform, and its first loss is close to $\log V$. That is not a bound, however: a confident wrong prediction costs more than $\log V$, and without limit.

Figure 4.1 shows all eighty charges at once. Each row is one training window, set under the corpus where it lies, and each bar stands under the character it predicts. The outline is what the untrained model pays; the solid bar is what the model pays after the chosen number of training steps. Run the training and watch the bars fall.

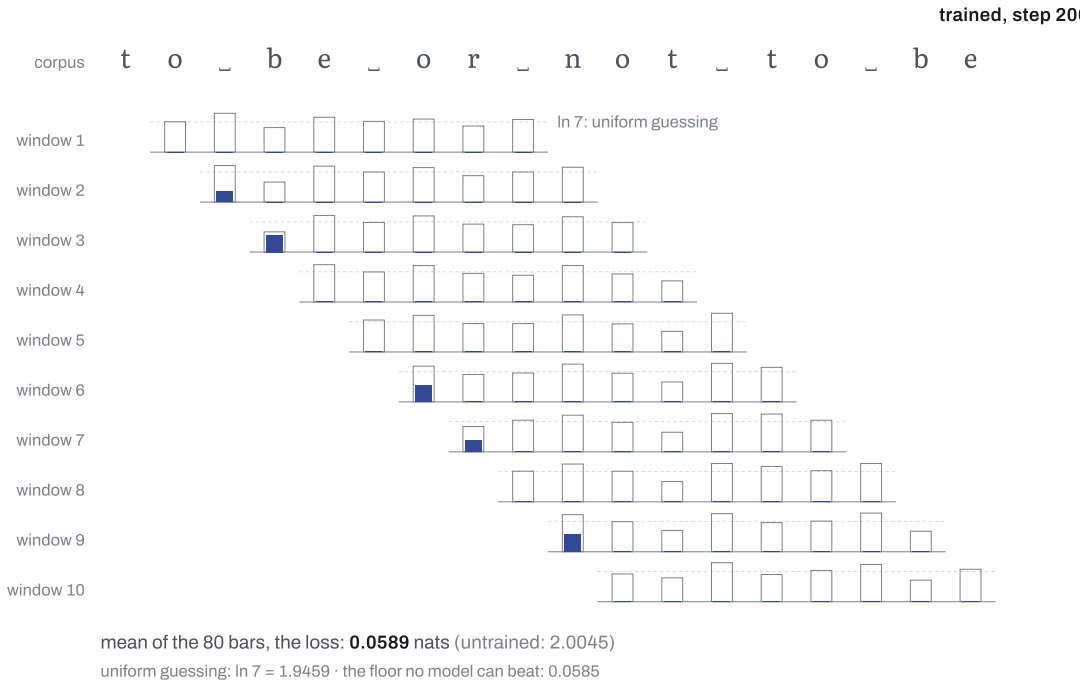
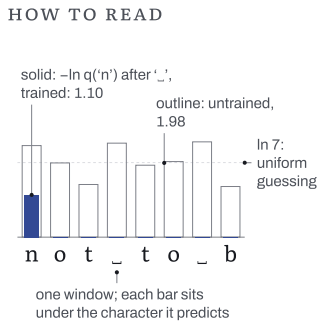


Figure 4.1 The surprise $-\log q(y)$ of each of the specimen's eighty predictions. Row b is training window b, placed under the corpus; each bar stands under the character it predicts. Outlines: the untrained model. Solid bars: the model after the step chosen on the line below, replayed exactly as it was trained. The dashed hairline is $\log V$, uniform guessing; the loss is the mean of all the bars.



The frontispiece draws the same eighty quantities as curves: each hairline is the surprise of one prediction through a training run, with time to the right on a logarithmic scale and surprise upward. The value-ink line is their mean, the loss. The two plateaux that refuse to fall are the five predictions of §4.5.

Almost every bar falls to nothing. Five do not. Each is the first bar of its row, and each settles at a level of its own: two at one height, three at another. These five are the subject of §4.5.

4.3 Maximum likelihood

The windows overlap, so most characters of the corpus are targets in several windows, each time with a different context. The product above is therefore not the probability of the corpus read once from left to right; it is a product of conditional probabilities, one per (context, target) pair of the training set, which is all that training needs.

The loss is a logarithm in disguise:

$$\mathscr{L}(\theta) = -\frac{1}{N}\log \prod_{n=1}^{N} q_\theta(y_n \mid c_n).$$

The product is the *likelihood*: the probability that the model assigns to the targets that were actually observed, given their contexts. The logarithm is strictly increasing and $-1/N$ reverses the order, so minimising the mean surprise and maximising the likelihood are the same problem. Training a language model is maximum-likelihood estimation, and the loss is the negative log-likelihood per prediction.

4.4 Cross-entropy and the KL divergence

In statistics a loss with this property, minimised in expectation by reporting the true distribution, is called a *proper scoring rule*. With three or more outcomes the logarithmic score is, up to affine changes, the only proper rule that depends on q only through the probability q_y of what actually happened.

Suppose the next character really is random, with distribution p, and the model predicts q. The expected surprise is the *cross-entropy*

$$H(p, q) = -\sum_a p_a \log q_a.$$

When $q = p$ it is the *entropy* $H(p) = -\sum_a p_a \log p_a$, the surprise that even a perfect predictor must expect. The difference is the *Kullback–Leibler divergence*:

$$H(p, q) = H(p) + \mathrm{KL}(p \,\|\, q), \qquad \mathrm{KL}(p \,\|\, q) = \sum_{a\,:\,p_a > 0} p_a \log \frac{p_a}{q_a}.$$

(Add and subtract $\sum_a p_a \log p_a$.) The divergence is never negative. For every $t > 0$, $-\log t \geq 1 - t$, with equality only at $t = 1$. So

$$\mathrm{KL}(p \,\|\, q) = \sum_{a\,:\,p_a > 0} p_a \Big(-\log \frac{q_a}{p_a} \Big) \geq \sum_{a\,:\,p_a > 0} p_a \Big(1 - \frac{q_a}{p_a} \Big) = 1 - \sum_{a\,:\,p_a > 0} q_a \geq$$

with equality exactly when $q_a = p_a$ wherever $p_a > 0$ and q puts no mass anywhere else, that is, when $q = p$. This is Gibbs' inequality, and it is the fact Chapter 3 borrowed to prove that softmax minimises the free energy. Here it says something just as useful: *the expected surprise is smallest when the model predicts the*

true distribution, and the smallest value is the entropy of the truth. The KL divergence measures, in nats, how much a prediction costs above that.

4.5 The floor

Now apply this to the training set itself. Two predictions with the same context receive the same input, so the model gives them the same distribution q_c. Group the N predictions by context: let n_c be the number of predictions with context c, and $\hat{p}_c$ the distribution of the characters that followed it. Regrouping the sum in $\mathscr{L}$ gives, exactly,

$$\mathscr{L}(\theta) = \sum_c \frac{n_c}{N} H(\hat{p}_c, q_c) = \underbrace{\sum_c \frac{n_c}{N} H(\hat{p}_c)}_{\text{the text}} + \underbrace{\sum_c \frac{n_c}{N} \mathrm{KL}(\hat{p}_c \,\|\, q_c)}_{\text{the model}}.$$

The first term does not depend on θ at all; it is a property of the corpus and of the way it was cut into windows. The second is never negative, and vanishes only if the model predicts, for every context, exactly what followed it. So the first term is a floor that no model, however large, can go below.

For most contexts $\hat{p}_c$ is a single character with probability one, and its entropy is zero. Which contexts are left open? A context is a window prefix, and every prefix of length two or more occurs in the specimen's training set either once or always with the same successor. The contexts of length one are different: there the model sees one character and nothing before it. Two windows begin with "o": in one the "o" of "to" is followed by a space, in the other the "o" of "or" is followed by "r". Three windows begin with a space, followed by "b", "o" and "n". For those contexts the best possible prediction is $\hat{p}_c$, uniform on two and on three characters, and the floor is

$$\mathscr{L}_{\min} = \frac{2 \log 2 + 3 \log 3}{80}.$$

Figure 4.2 (p. 52) computes this from the corpus and the windows, and splits the loss into floor and divergence at every step of training.

The ambiguity is an artefact of windowing, not of the text. Each window starts in the middle of the corpus with no memory of what came before; had its first prediction been allowed to see one



Figure 4.2 The loss regrouped by context. The two contexts the training set leaves open are shown in full: what followed them (outlined bars, one per character) and what the model predicts (solid bars); every other context is followed by one character only. On the right, the cross-entropy of each context split into the entropy of what followed (outlined) and the divergence of the model from it (solid). Bottom: the loss, which is the floor plus the mean divergence.

more character, it would have known whether its "o" was the "o" of "to" or of "or", and the floor would be zero (Exercise 4.3, p. 55). Real models have the same problem at the start of every window, which is one reason long contexts help.

Two more things in the figure are worth noticing. The open contexts end up predicting almost exactly the uniform $\hat{p}_c$, which is what maximum likelihood demands. And the decided contexts never quite reach divergence zero: that would need q_c to be exactly one-hot, which a softmax with finite logits never is (Chapter 3). The logit gaps would have to grow without bound. So the loss approaches the floor but never meets it.

4.6 The gradient

Training needs the derivative of $\mathscr{L}$ with respect to everything, and it starts with the logits. For one prediction,

$\ell_n = \operatorname{lse}(z_n) - Z_{y_n,n}$. Chapter 3 showed that $\partial \operatorname{lse}/\partial z_a = p_a$, and $\partial Z_{y,n}/\partial Z_{a,n} = \delta_{ay}$. Averaging over the N predictions,

$$\bar{Z}_{an} = \frac{\partial \mathscr{L}}{\partial Z_{an}} = \frac{p_{an} - \delta_{a,y_n}}{N}, \qquad \bar{Z} = \frac{P - Y}{N},$$

where $P = \operatorname{softmax}(Z)$ column by column. The bar is the notation of Chapter 7 for the adjoint, the sensitivity of the loss to a quantity; in this book adjoints are drawn in the second ink, hatched. In words: the gradient with respect to the logits is the *predicted* occupation minus the *observed* one. Gradient descent moves z_n against it. It raises the logit of the character that came, by $(1 - p_y)/N$ times the step size, and lowers every other logit in proportion to the probability the model gave it.

Each column of $\bar{Z}$ sums to zero, $\mathbf{1}^\top (p - e_y) = 1 - 1 = 0$. That is shift invariance again: the loss cannot see a common shift of the logits, so its gradient has no component along $\mathbf{1}$. In particular the gradient of the output bias, $\bar{b}_{\text{out}} = \bar{Z}\mathbf{1}_N$, is always orthogonal to $\mathbf{1}$, as promised in Chapter 3.

The gradient also explains the floor from the other side. The three predictions whose context is a single space have the same logits z and the same p, but different targets. Their gradients add up to

$$(p - e_b) + (p - e_o) + (p - e_n) = 3\,(p - \hat{p}), \qquad \hat{p} = \tfrac{1}{3}(e_b + e_o + e_n).$$

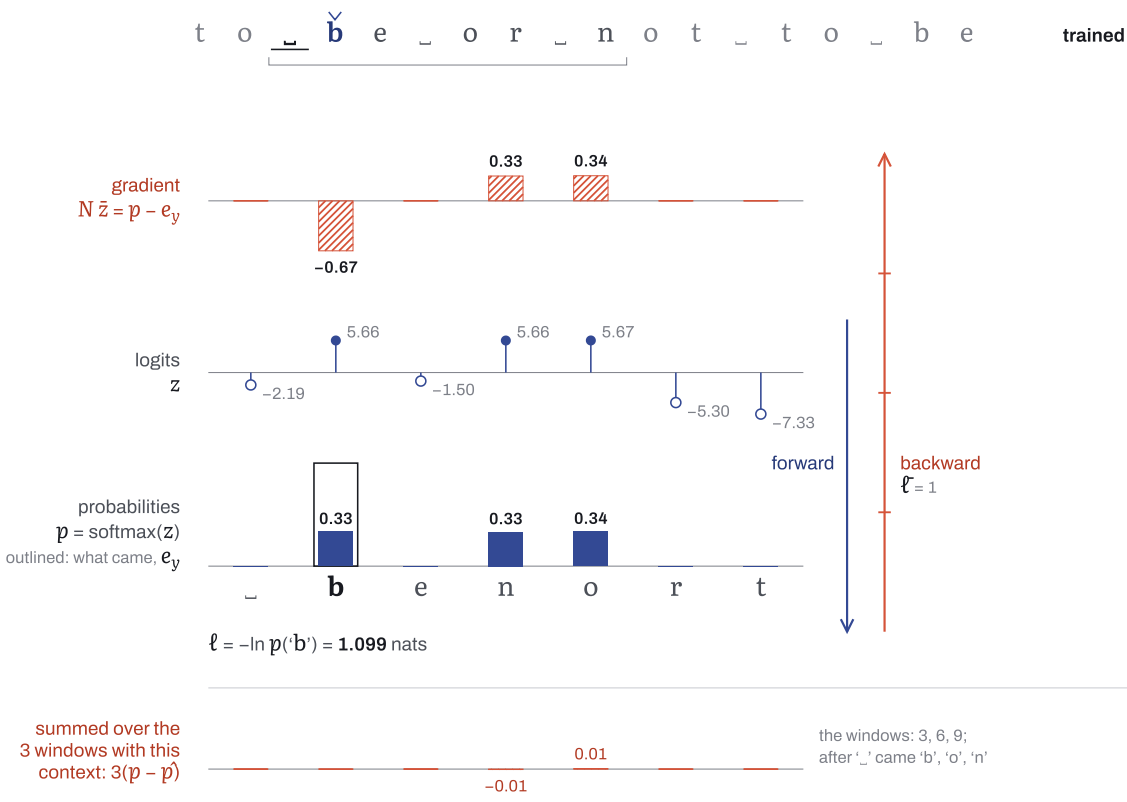
Each window on its own pulls its own character up. Together they pull towards $\hat{p}$, and they stop pulling exactly when $p = \hat{p}$. Click the first "␣" of a window in Figure 4.3 (p. 54) to see the three pulls and their sum.

4.7 In the code

The loss and its pullback are one function. It subtracts each column's maximum m_n, exponentiates, sums each column to s_n, and returns the mean surprise together with a closure `back` that returns $\bar{Z}$:

src/softmax.jl crossentropy · lines 43–60

```
function crossentropy(Z, y)
    N = size(Z, 2)
    N > 0 || throw(ArgumentError("cross-entropy needs at least one token"))
```



HOW TO READ

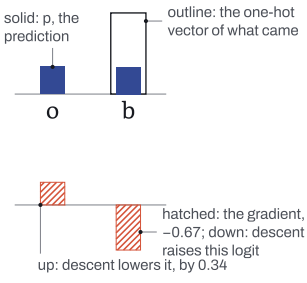


Figure 4.3 One prediction, forward and backward. Going down (value ink): the logits z, their softmax p, and the surprise ℓ of the character that came, whose one-hot vector e_y is outlined. Going up (covector ink, hatched): the gradient $N\bar{z} = p - e_y$, which arrives above the logits it belongs to. When other windows share the context, the bottom row adds their gradients. Choose any of the eighty predictions on the corpus line.

```
    length(y) == N || throw(DimensionMismatch("one target is required per logit column"))
    m = maximum(Z; dims = 1)
    s = map(eachcol(Z), vec(m)) do z, mₙ
        sum(zₐ -> exp(zₐ - mₙ), z)
    end
    ℓ = sum(log(s[n]) - (Z[y[n], n] - m[n]) for n in 1:N) / N
```

```
    function back(ℓ̄ = 1)
        Z̄ = exp.(Z .- m) ./ s'                  # softmax(z_n), for every n
        for n in 1:N
            Z̄[y[n], n] -= 1                      # - onehot(y_n)
        end
        Z̄ .*= eltype(Z)(ℓ̄) / N
    end
    ℓ, back
end
```

The surprise of prediction n is computed as $\log s_n - (Z_{y_n,n} - m_n)$ rather than as $\mathrm{lse}(z_n) - Z_{y_n,n}$. The two are equal in exact arithmetic. In floating point, a large common shift of the logits (Chapter 3) would make the second form subtract two nearly equal large numbers and lose the surprise in the rounding; the first subtracts the shift from both terms before they meet. `back` divides the shifted exponentials by their sums, which is P, subtracts one at each target, and scales by $\bar{\mathscr{L}}/N$. It is the first pullback the backward pass calls, and every other gradient in the model is computed from its output.

Exercises

EXERCISE 4.1

Let $f : (0,1] \to \mathbb{R}$ be continuous with $f(st) = f(s) + f(t)$. Show that $f(s) = -k \log s$ for some constant k. (Consider $g(x) = f(e^{-x})$ on $[0, \infty)$.)

EXERCISE 4.2

A coin lands heads. One forecaster said $q(\text{heads}) = 0.9$, another 0.5. What does each pay, in nats and in bits? What would each have paid had it landed tails? Show that a forecaster who always says 1 pays an infinite amount the first time it is wrong.

EXERCISE 4.3

Suppose the first prediction of every window could also see the character before the window. List the contexts of length one that become contexts of length two, and show that the specimen's floor would then be zero.

EXERCISE 4.4

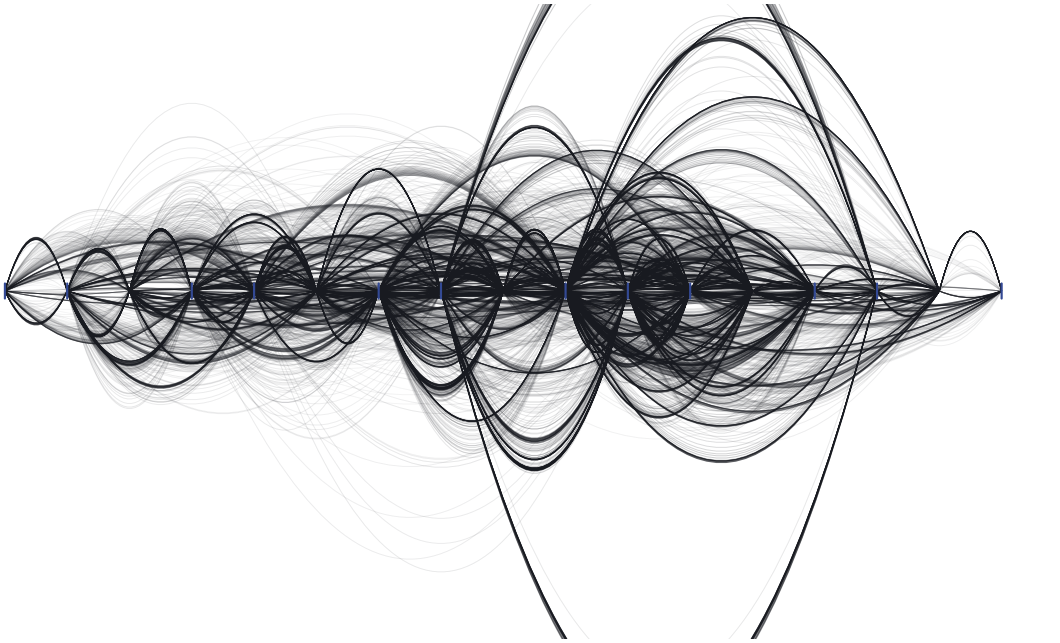
Show that plain gradient descent never changes the sum $\sum_a (b_{\text{out}})_a$ of the output biases. Does AdamW, whose step is not proportional to the gradient (Chapter 8), preserve it too?

EXERCISE 4.5

In Figure 4.2 (p. 52), drag the training back to step 0. How much of the loss is floor and how much divergence? Find the step at which the divergence first falls below the floor.

EXERCISE 4.6

In Figure 4.1 (p. 49), point at the five bars that do not fall. For each, write down the context the model sees and the characters that follow that context in the training set, and check that the bar settles at $\log 2$ or $\log 3$.



Chapter 5. *Every reading the specimen's two attention heads made while it learned.* The corpus "to be or not to be" lies along the baseline (value ticks mark letters, gaps mark spaces). Reader i reading source j < i with weight A[i,j] is an arc from j to i of height span × weight; head 1 above the line, head 2 below. 40 moments of training (AdamW, η 0.02 → 0.002 (warmup 20, cosine), β_2 0.95, weight decay 0.01 on matrices, clip 1), later moments inked more strongly. Head 1 learns short, local readings; head 2 learns to reach far back.

CHAPTER FIVE

5 Attention

Attention is the one map in the model through which positions talk to each other: column i of its output is an average of value vectors from positions $j \le i$, weighted by a Gibbs distribution over query–key scores.

MATHS	PHYSICS	ML
row-stochastic, lower-triangular $A \in \mathbb{R}^{T \times T}$	*causal transition kernel*	"attention pattern"
$o_i = \sum_j A_{ij} v_j$, a convex combination	*expectation value* $\langle v \rangle_i$	"attention output"
$S_{ij} = q_i \cdot k_j / \sqrt{d}$	*negative energy*	"attention score, logit"
$\sqrt{d}$	*temperature*	"scale factor"
$A_{ij} = 0$ for $j > i$	*causality*	"causal mask"
$\mathbb{R}^D = \mathbb{R}^d \oplus \cdots \oplus \mathbb{R}^d$	*direct sum of subsystems*	"multi-head attention"

5.1 Where columns meet

EVERY MAP WE HAVE MET SO FAR ACTS ON ONE COLUMN AT A time. The embedding sends the character at position t to the column $Ee_{x_t} + P_{:,t}$. LayerNorm, the MLP and the final linear map, which we meet in Chapter 6, also act column by column, with the same parameters for every column. Each of them has the form

$$X = (x_1, \ldots, x_T) \longmapsto \big(f(x_1), \ldots, f(x_T)\big).$$

A model assembled only from such maps predicts the next character from the current character and its position, and from nothing else. It is a bigram model with a clock.

Attention is the exception. It is the only map in the model whose output column i depends on input columns other than i. Tokens interact here and nowhere else. We assemble it in three steps: an average, a score, and a mask.

5.2 An average of earlier columns

Rows are readers and columns are sources, everywhere in this book. Since tokens are columns, $O = VA^{\mathsf{T}}$ is the natural order of the factors; the transpose is the price of writing tokens as columns.

Let $v_1, \ldots, v_T \in \mathbb{R}^d$ be vectors, one per position, written as the columns of $V \in \mathbb{R}^{d \times T}$. Let $A \in \mathbb{R}^{T \times T}$ be *row-stochastic*: $A_{ij} \geq 0$, and every row sums to one. Then set

$$o_i = \sum_{j=1}^{T} A_{ij}\, v_j, \qquad \text{in matrix form} \qquad O = VA^{\mathsf{T}} \in \mathbb{R}^{d \times T}.$$

Row i of A is a probability distribution on the positions, and o_i is the expectation of v under it. Geometrically, o_i lies in the convex hull of the v_j. The indices read as a sentence: *reader* i reads *source* j with weight A_{ij}.

Figure 5.1 shows this for the trained specimen. Each character of the window is a reader. Its two heads, which we explain in §5.6 (p. 63), read with two different distributions: one is drawn above the line, the other below.

5.3 Scores, and why a softmax

Where do the weights come from? Every position emits three vectors. Each is an affine image of the position's column $\hat{x}_i$ of the normalised stream:

$$q_i = W_q \hat{x}_i + b_q, \qquad k_i = W_k \hat{x}_i + b_k, \qquad v_i = W_v \hat{x}_i + b_v.$$

The *query* q_i says what reader i is looking for, the *key* k_j says what source j offers, and the *value* v_j is what source j hands over when it is read. A pair is scored by a scaled inner product, and the weights are the softmax of the scores over the sources:

$$S_{ij} = \frac{q_i \cdot k_j}{\sqrt{d}}, \qquad A_{ij} = \frac{e^{S_{ij}}}{\sum_{j' \leq i} e^{S_{ij'}}}.$$

In the language of Chapter 3, $-q_i \cdot k_j$ is an energy, $\sqrt{d}$ is a temperature, and row i of A is the Gibbs distribution

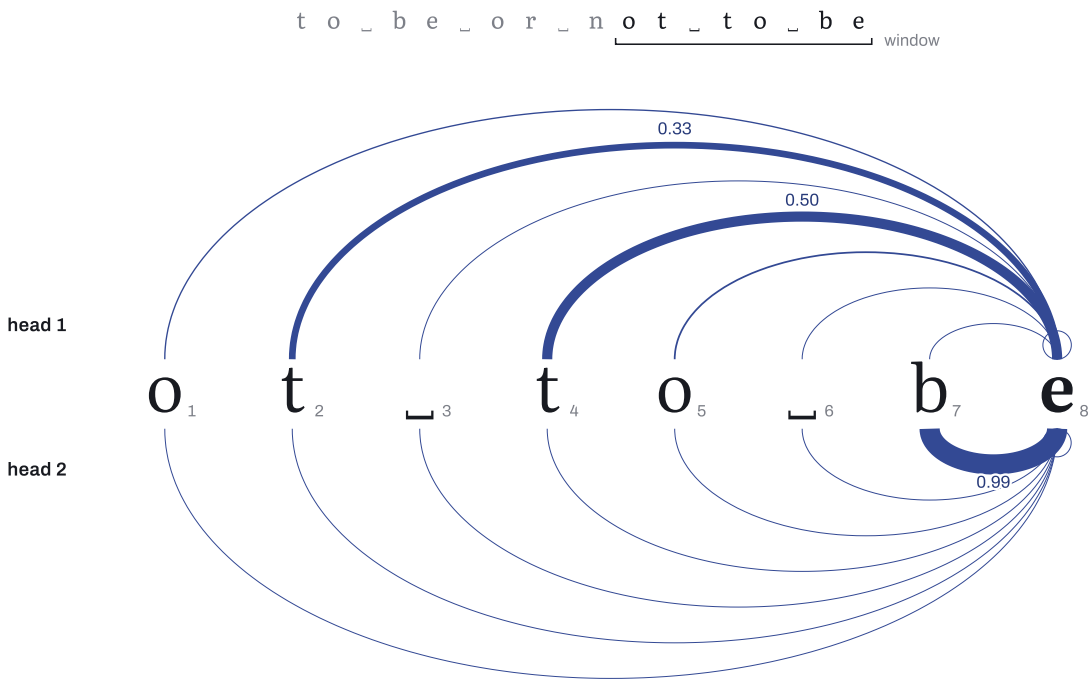
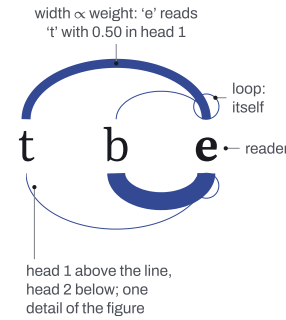


Figure 5.1 What each character reads. The arcs leave the selected reader and land on the earlier characters it reads; the width of an arc is the weight A_{ij}, and the arcs of one head sum to one. Head 1 is drawn above the line and head 2 below. A loop is a character reading itself. Drag the window along the line to change the text.

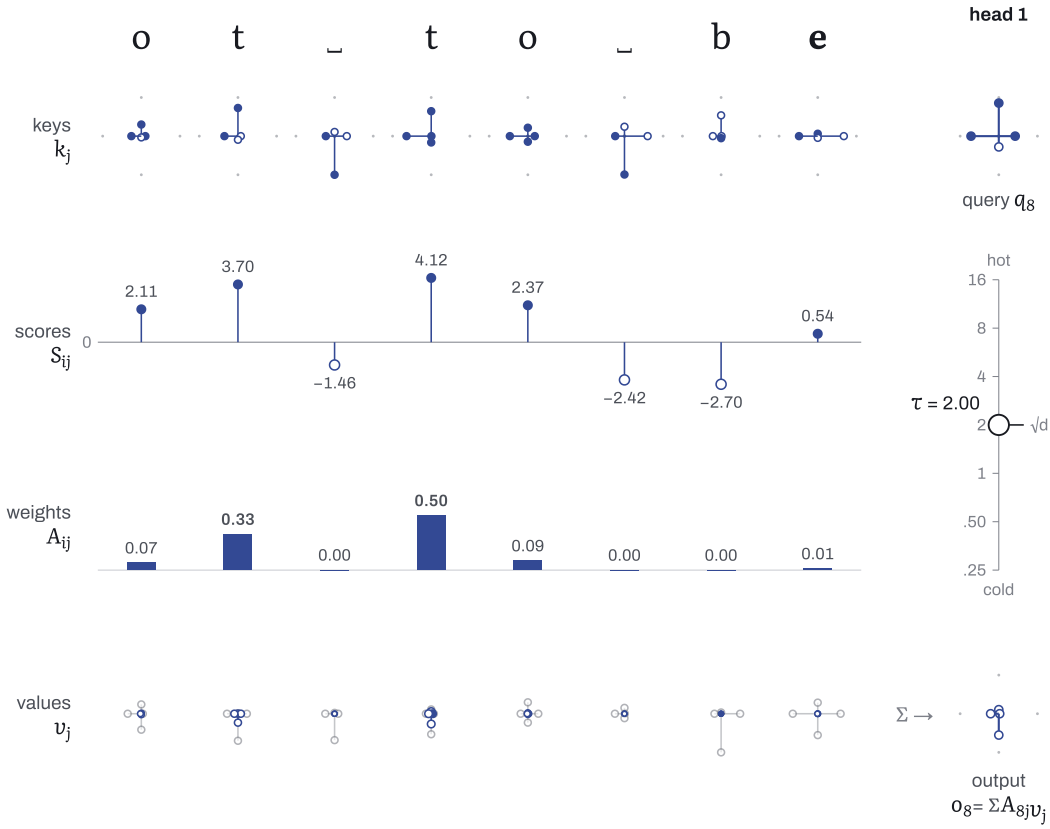
HOW TO READ



$$A_{ij} = \frac{1}{Z_i} e^{-E_{ij}/\tau}, \qquad E_{ij} = -q_i \cdot k_j, \quad \tau = \sqrt{d}.$$

A source is read in proportion to the exponential of how well its key matches the query.

The score is a bilinear form on the stream. Write $W_q^{(h)}, W_k^{(h)} \in \mathbb{R}^{d \times D}$ for the rows of W_q, W_k that belong to one head (§5.6, p. 63). Up to the bias terms, $q_i \cdot k_j = \hat{x}_i^{\top} W_q^{(h)\top} W_k^{(h)} \hat{x}_j$. The matrix $W_q^{(h)\top} W_k^{(h)}$ is a general $D \times D$ matrix of rank at most d, and



REMINDER: SPOKE GLYPHS

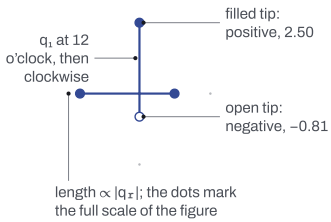


Figure 5.2 One reader and one head, from top to bottom: the keys k_j of the sources, the reader's query q_i, the scores $S_{ij} = q_i \cdot k_j / \sqrt{d}$, the weights A_{ij}, and the values, each scaled by its weight; their sum is o_i. Drag the temperature τ away from $\sqrt{d}$: cold rows read one source, hot rows read all of them equally.

nothing makes it symmetric: "what does i look for in j" and "what does j look for in i" are different questions.

Why $\sqrt{d}$

5.4 The variance argument concerns initialisation, when q and k really are close to independent. After training, the scale of the scores is whatever the loss wanted; the $\sqrt{d}$ just sets a sensible starting temperature.

Suppose the components of q and k are independent, with mean zero and variance one. Then $q \cdot k = \sum_{r=1}^{d} q_r k_r$ is a sum of d independent terms of variance one, so its variance is d. Unscaled

scores therefore grow like $\sqrt{d}$ as heads get wider. The softmax of numbers with spread $\sqrt{d}$ is a Gibbs distribution at inverse temperature of order $\sqrt{d}$: as d grows it freezes onto its largest entry, and its derivative, which is all that learning ever sees (Chapter 7), goes to zero. Dividing by $\sqrt{d}$ keeps the scores of order one whatever the head width. In the physical reading, $\sqrt{d}$ is the temperature at which the distribution is neither frozen nor uniform.

5.5 The causal mask

A language model predicts x_{t+1} from $x_{\le t}$ (Chapter 1). One forward pass makes all T predictions of a window at once, and to keep them honest, position i must not read any position $j > i$. We add the mask

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i, \end{cases} \qquad A = \operatorname{softmax}_{\text{rows}}(S + M).$$

Since $e^{-\infty} = 0$, the upper triangle of A vanishes, and A is lower-triangular. Every row still sums to one, now over $j \le i$. The first reader can read only itself: $A_{11} = 1$.

Lower-triangularity is causality written as a matrix. Column i of the output depends on columns $1, \dots, i$ of the input and on nothing later, so the Jacobian of the map $X \mapsto O$ is block lower-triangular. One pass therefore yields T conditional distributions, each conditioned on its own past. These are exactly the factors of the chain rule of probability, and the loss uses all of them.

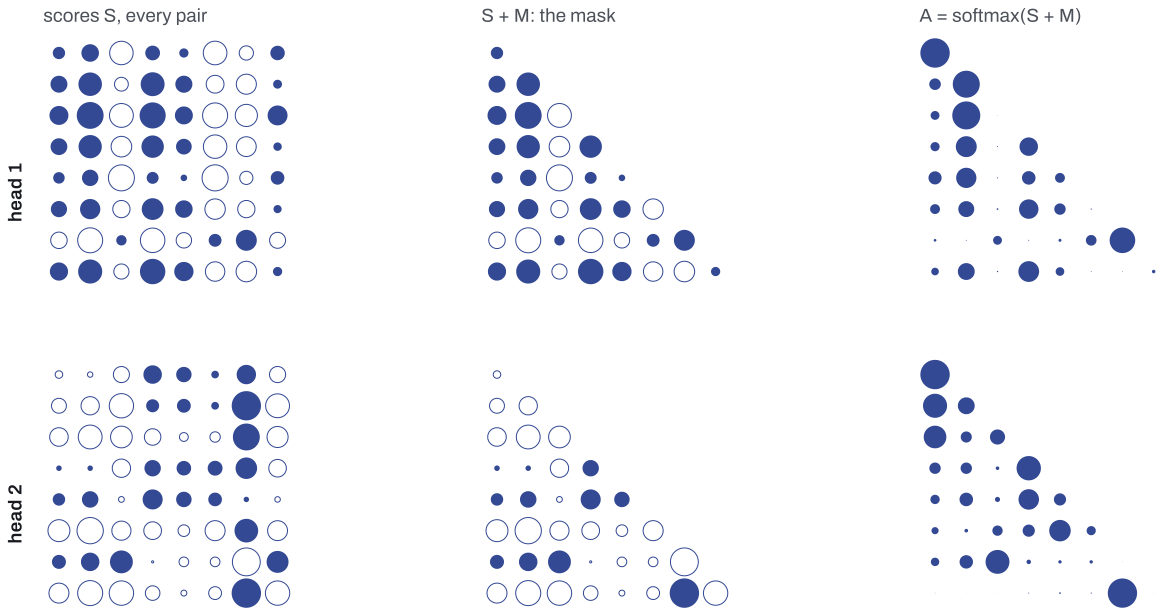
5.6 Heads: a direct sum

The specimen has $D = 8$ and $H = 2$ heads of width $d = D/H = 4$. Split the space as $\mathbb{R}^8 = \mathbb{R}^4 \oplus \mathbb{R}^4$: rows 1–4 of Q, K and V belong to head 1 and rows 5–8 to head 2. Each head computes its own scores and its own pattern $A^{(h)}$ from its own block, and the outputs are stacked:

$$O^{(h)} = V^{(h)} A^{(h)\mathsf{T}} \in \mathbb{R}^{d \times T}, \qquad O = \begin{pmatrix} O^{(1)} \\ O^{(2)} \end{pmatrix} \in \mathbb{R}^{D \times T}.$$

The output map W_o then mixes the blocks, and the result is added to the residual stream,

$$X \longleftarrow X + W_o O + b_o \mathbf{1}^{\mathsf{T}}.$$



HOW TO READ

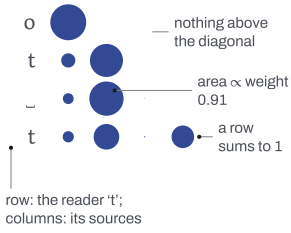


Figure 5.3 The attention matrices of the specimen's two heads for one window. Row i is reader i and column j is source j; the area of a dot is A_{ij}. Nothing is drawn above the diagonal. Run the forward computation: scores for every pair, then the mask, then the softmax of each row.

The scores S_{ij} are computed for every pair, the future included, and the trace stores them before the mask is applied. Half of them are then thrown away. For long contexts that is wasteful, and fast implementations never compute the upper triangle.

Two heads let a position read two different things at once, and each pays for it with a smaller d. The four matrices W_q, W_k, W_v, W_o and their biases hold $4(D^2 + D)$ numbers, which is 288 of the specimen's parameters.

5.7 In the code

In the code the chapter is two functions. The first computes the patterns A of every head and every window from Q and K:

src/attention.jl attention · lines 70–90

```
function attention(Q, K, H, T)
    D, N = size(Q)
    A = similar(Q, T, T, H, N ÷ T)
    α = 1 / √eltype(Q)(D ÷ H)                          # 1/√d, in the number type of Q
    M = eltype(Q)[j > i ? -Inf : 0 for i in 1:T, j in 1:T]     # the causal mask
    foreach_head(A, Q, K) do a, q, k
        mul!(a, q', k, α, false)                       # S = qᵀk/√d
        softmax!(a .+= M; dims = 2)                    # A = softmax_rows(S + M)
    end
    function back(Ā)
        S̄, Q̄, K̄ = similar(A), similar(Q), similar(K)
        foreach_head(A, Ā, S̄, Q, K, Q̄, K̄) do a, ā, s̄, q, k, q̄, k̄
            s̄ .= a .* ā                                 # S̄ = A ⊙ Ā …
            s̄ .-= a .* sum(s̄, dims = 2)                 # … - A ⊙ (row sums of A ⊙ Ā)
            mul!(q̄, k, s̄', α, false)                   # Q̄ = k S̄ᵀ/√d
            mul!(k̄, q, s̄, α, false)                    # K̄ = q S̄ /√d
        end
        Q̄, K̄
    end
    A, back
end
```

The second applies them, $O = VA^{\mathsf{T}}$, head by head:

src/attention.jl mix · lines 104–118

```
function mix(A, V)
    O = similar(V)
    foreach_head(A, V, O) do a, v, o
        mul!(o, v, a')                                 # O = V Aᵀ
    end
    function back(Ō)
        Ā, V̄ = similar(A), similar(V)
        foreach_head(A, V, Ō, Ā, V̄) do a, v, ō, ā, v̄
            mul!(ā, ō', v)                             # Ā = Ōᵀ V
            mul!(v̄, ō, a)                              # V̄ = Ō A
        end
        Ā, V̄
    end
    O, back
end
```

Each returns its value together with its pullback, the function `back` that turns the adjoint of the output into adjoints of the inputs. The bars are the covectors of Chapter 7, where these pullbacks are derived.

Exercises

EXERCISE 5.1

Show that if all the scores in row i are equal, then o_i is the plain average of $v_1, \ldots, v_i$. Which reader sees no difference between the masked and the unmasked softmax?

EXERCISE 5.2

Show that adding the same constant to every score in a row leaves that row of A unchanged. Deduce that the key bias b_k has no effect on A, and hence that its gradient is exactly zero. (The model carries d parameters per head that it cannot use.)

EXERCISE 5.3

Remove the positional code, so that $\hat{x}_j$ depends only on the character at position j. Show that o_i is then unchanged by any permutation of the sources $1, \ldots, i-1$. What does that say about a model without positions?

EXERCISE 5.4

Compute the Jacobian $\partial o_i / \partial v_j$ and $\partial o_i / \partial S_{ij}$ for one head. Write the second one in terms of A_{ij}, v_j and o_i, and interpret it: raising one score pulls the output towards that source's value, by how much?



Chapter 6. *Every column that entered one of the specimen's LayerNorms, drawn as its journey onto the sphere.* 9120 centred columns $\tilde{x}$ (80 per LayerNorm × 3 LayerNorms × 38 moments of training, AdamW, η 0.02 → 0.002 (warmup 20, cosine), β_2 0.95, weight decay 0.01 on matrices, clip 1). Each stroke runs radially (radius $\propto \log(1 + \text{length})$) from $|\tilde{x}|$ to $|\hat{x}| = |\tilde{x}|/\sigma$, which is just inside $\sqrt{D} = 2.828$; its angle is the direction of $\tilde{x}$ projected onto the plane of the two leading principal directions of all centred columns. Value ink: the trained model. The circle is not drawn; it is where the strokes end.

CHAPTER SIX

6 The block

A block makes two corrections to the stream of columns, $X \leftarrow X + f(X)$, each of the form of one step of Euler's method: in the first the tokens read from each other, in the second each token computes alone. Each correction reads a copy of the stream that LayerNorm has centred and scaled to just inside a sphere.

MATHS	PHYSICS	ML
$X \in \mathbb{R}^{D \times N}$, updated by $X \leftarrow X + f(X)$	*the state, evolved in depth*	"residual stream"
$x_{k+1} = x_k + f_k(x_k)$	*an explicit Euler step*	"residual connection, skip connection"
$\Pi = I - \mathbf{1}\mathbf{1}^\top / D$	*projection out of the uniform mode*	"mean subtraction, centring"
$\hat{x} = \Pi x / \sigma$, $\lVert\hat{x}\rVert \lesssim \sqrt{D}$	*projection to (just inside) a sphere*	"layer normalisation, LayerNorm"
$y = \gamma \odot \hat{x} + \beta$	*a diagonal affine map*	"learned gain and bias, "scale and shift""
$y = \sum_k r_k \, (W_2)_{:,k} + b_2$, $r = \mathrm{relu}(W_1 \hat{f} + b_1)$	*a sum of switched sources*	"MLP, feed-forward layer"
$\theta \in \mathbb{R}^p$	*a point in configuration space*	"the weights, the model"

6.1 The stream

AFTER THE EMBEDDING OF CHAPTER 2 THE MODEL HOLDS A matrix $X_0 \in \mathbb{R}^{D \times N}$, one column per token. Everything that

In the code the two residual sums are the lines `X₁ = X₀ + Y₁` and `X₂ = X₁ + Y₂` of `block` (§6.5). Their pullbacks, `X̄₁ .+= X̄₂` and `X̄₀ .+= X̄₁`, are the road for the gradient described below.

follows keeps this shape until the very last map. A block is a map $\mathbb{R}^{D\times N} \to \mathbb{R}^{D\times N}$, and it is built in a particular way: it never replaces X, it adds to it. With LN_1, LN_2 the two normalisations of §6.2, the block is

$$X_{\mathrm{mid}} = X + \mathrm{Attn}\left(\mathrm{LN}_1(X)\right), \qquad X' = X_{\mathrm{mid}} + \mathrm{MLP}\left(\mathrm{LN}_2(X_{\mathrm{mid}})\right),$$

where Attn is the attention of Chapter 5 together with its output map $W_o \cdot + b_o$, and MLP is the two-layer map of §6.3 (p. 74). Each line reads the current state, computes a *branch*, and adds the branch back. The matrix being corrected is called the *residual stream*. It is the model's one working memory: the embedding writes the first state into it, every branch reads a normalised copy of it and writes an increment into it, and at the end the readout reads the last state.

Each line has the shape of a familiar formula. For an ordinary differential equation $\dot{x} = f(s, x)$ on a grid $s_{k+1} = s_k + h$, the explicit Euler method is

$$x_{k+1} = x_k + h f(s_k, x_k).$$

A residual update $X_{k+1} = X_k + f_k(X_k)$ is exactly an update of this form, with step $h = 1$ and a vector field f_k that is different at every step, since every branch has parameters of its own. The specimen has $L = 1$ block and therefore two steps; a model with L blocks takes $2L$. Depth plays the role of time, and the stream is the state being evolved rather than a state discarded and rebuilt at every layer.

Residual connections come from image recognition: He, Zhang, Ren and Sun, *Deep residual learning* (2016). The name records the idea that a branch need only learn the *residual* $f(x) = \text{target} - x$, a correction to the identity, rather than the whole map. Reading the stack of updates as an Euler scheme goes back to E (2017) and to Haber and Ruthotto (2017); Chen, Rubanova, Bettencourt and Duvenaud (2018) took the limit seriously and made the depth continuous ("neural ODEs"). The transformer of this book does not.

The analogy is a statement about the form of the update, and no more. Nothing in the architecture makes the steps small, or makes f_k depend smoothly on k, so the model is not a discretisation of some underlying flow and nothing converges to one as layers are added. What the form does give is a default. A branch that outputs zero leaves the stream untouched, so a block can only *refine* the state it is handed; a freshly initialised model whose branches are small is close to the identity, and the embedding survives it intact. The other gift arrives in Chapter 7: the derivative of $x \mapsto x + f(x)$ is $I + Df$, and the I is a road along which the gradient travels back without passing through any branch.

Figure 6.1 draws the whole model in this picture. Each token is a vertical stem along which its column flows downwards. The two branches of the block leave the stem, compute, and merge back at a $+$. All the maps act on one column at a time, except

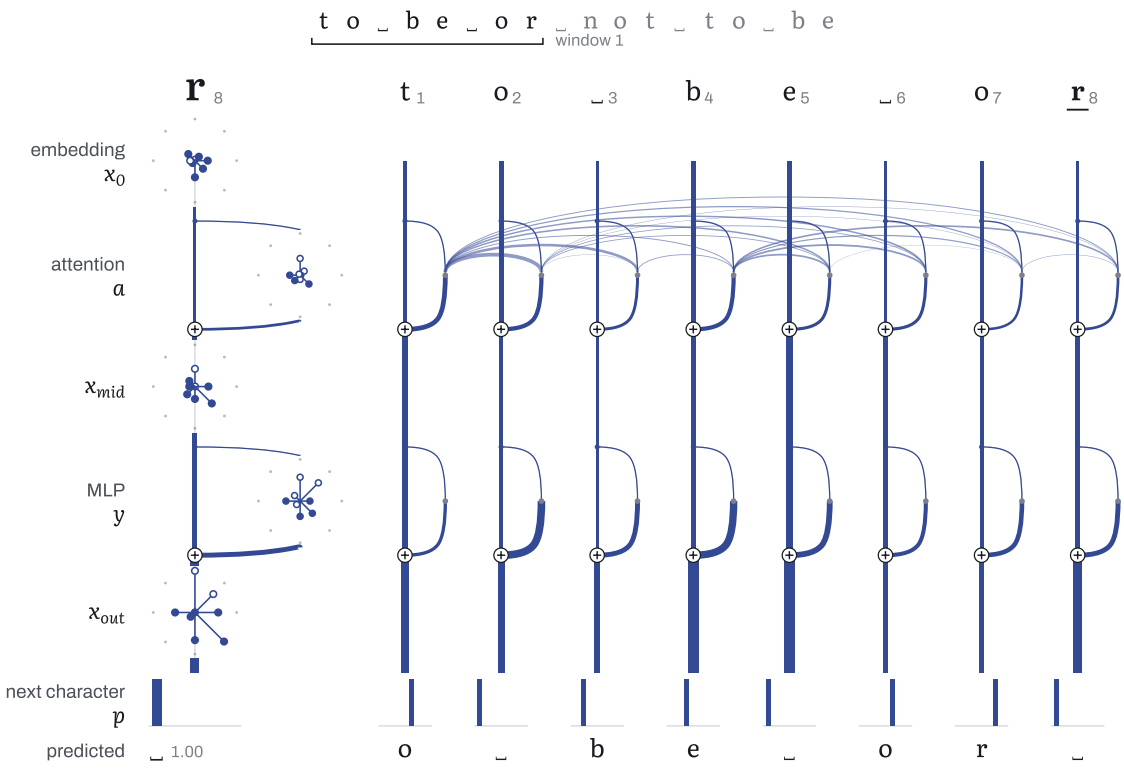


Figure 6.1 The whole model for one window, one stem per token. The stream runs down each stem; the attention branch and the MLP branch leave it, compute, and merge back at a +. The width of the stem is the size of the column, $\|x\|/\sqrt{D}$, and the width of a branch is the size of the update it adds. Only the attention branch reaches across to other stems: an arc is the part of a later token's update that an earlier token hands it. On the left, the selected column is drawn with its vectors. Run the forward pass, or click a stem.

attention, which reaches across to the earlier stems: it is still the only place where tokens meet.

6.2 LayerNorm

The branches do not read the stream itself. Each reads a copy that has been normalised, column by column, by a map called *layer normalisation*, or LayerNorm. For one column $x \in \mathbb{R}^D$ define

$$\mu = \frac{\mathbf{1}^\top x}{D}, \qquad u = x - \mu\mathbf{1} = \Pi x, \qquad \sigma = \sqrt{\frac{\|u\|^2}{D} + \varepsilon}, \qquad \hat{x} = \frac{u}{\sigma}, \qquad y = \gamma \odot \hat{x} + \beta,$$

with $\Pi = I - \mathbf{1}\mathbf{1}^\top/D$, a fixed $\varepsilon = 10^{-5}$, and two learned vectors $\gamma, \beta \in \mathbb{R}^D$. In index form, $\hat{x}_i = (x_i - \mu)/\sigma$ and $y_i = \gamma_i \hat{x}_i + \beta_i$. The mean μ and the variance $\sigma^2 - \varepsilon$ are taken over the D entries of one column, never across tokens, and the variance divides by D, not by $D - 1$: it is the mean square of the entries of u, not an estimate of anything. Each column is normalised on its own, with the same γ and β for all of them.

Read geometrically, LayerNorm does three things.

It centres. Π is symmetric and $\Pi^2 = \Pi$, its kernel is the line spanned by $\mathbf{1}$ and its image is the hyperplane $\mathbf{1}^\perp$ of vectors whose entries sum to zero. So $u = \Pi x$ is the orthogonal projection of x onto $\mathbf{1}^\perp$: the component of x along $\mathbf{1}$, the *uniform mode* in which every entry moves together, is removed and nothing else is touched.

It scales. If ε were zero, $\hat{x} = \sqrt{D}\, u/\|u\|$ would lie on the sphere of radius $\sqrt{D}$ in $\mathbf{1}^\perp$; equivalently, the root mean square of its entries would be exactly 1. With $\varepsilon > 0$,

$$\|\hat{x}\|^2 = \frac{\|u\|^2}{\sigma^2} = D\Big(1 - \frac{\varepsilon}{\sigma^2}\Big) < D,$$

so the column lands *just inside* that sphere, by an amount that depends on the column: the shorter u was, the deeper it lands. The point of ε is the column with $u = 0$, whose entries are all equal; there $\hat{x} = 0$ instead of $0/0$.

It applies a diagonal affine map. $y = \gamma \odot \hat{x} + \beta$ stretches axis i by γ_i and shifts it by β_i. These $2D$ numbers are learned, and they take the column off the sphere again: what the branch receives lies on (just inside) an ellipsoid, the image of the sphere under $\mathrm{diag}(\gamma)$, shifted by β. The sphere is an intermediate stage, and the gain and offset give back, in a controlled way, a scale and a centre of the model's own choosing.

There is an exact sphere behind ε, one dimension up. Append the number $\sqrt{D\varepsilon}$ to u, scale the vector $(u, \sqrt{D\varepsilon}) \in \mathbb{R}^{D+1}$ to length $\sqrt{D}$, and drop the last entry: what remains is precisely $\hat{x}$. The missing coordinate holds the length by which $\hat{x}$ falls short.

Centring and scaling do not change the direction of u. Up to the tiny effect of ε, what LayerNorm keeps of a column is the unit vector $u/\|u\|$ and nothing else; it forgets two numbers, the component along $\mathbf{1}$ and the length of what is left. Figure 6.2 draws exactly those two numbers. Each column x becomes the point

$$(a, r) = \Big(\frac{\mathbf{1}^\top x}{\sqrt{D}},\ \|\Pi x\|\Big), \qquad a^2 + r^2 = \|x\|^2,$$

in a half-plane whose horizontal axis is the direction $\mathbf{1}/\sqrt{D}$ and whose vertical axis is distance from it. The circle of radius $\sqrt{D}$ is the sphere $\|x\| = \sqrt{D}$. Centring moves every point horizontally onto the vertical axis; scaling moves it along that axis to just below the top of the circle; and since this picture records only the two forgotten numbers, every column of the window arrives at the same place in it.

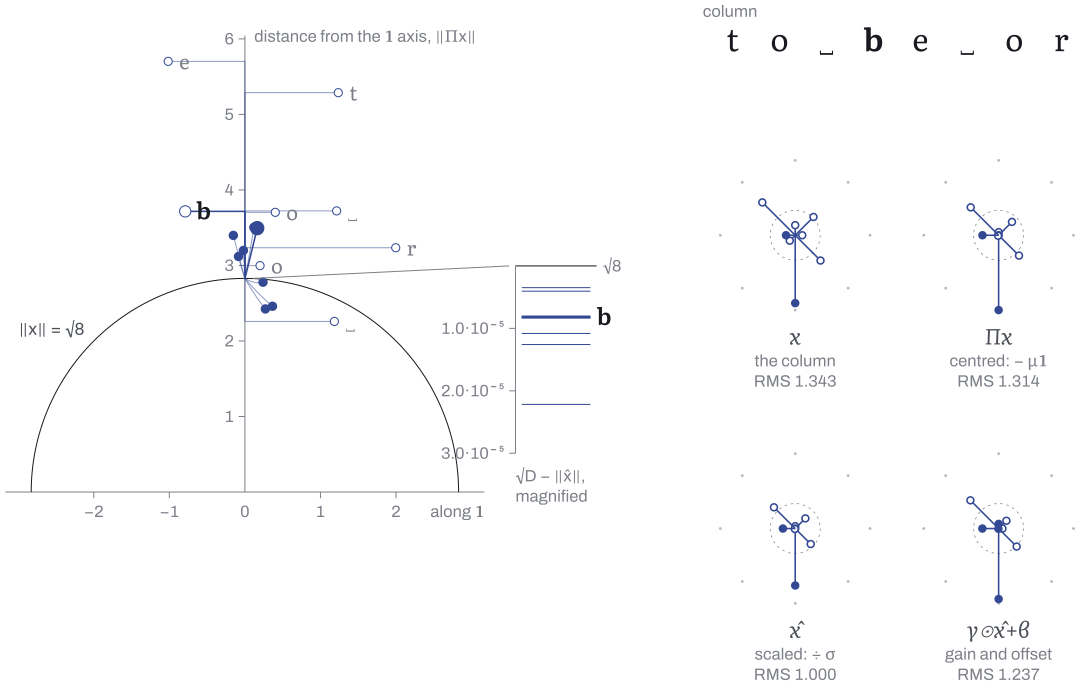
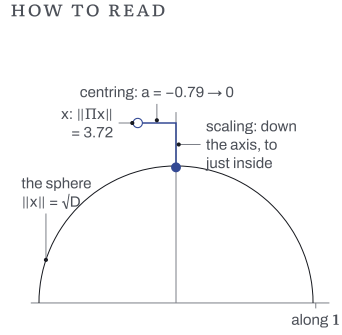


Figure 6.2 LayerNorm, geometrically, for the second normalisation of the block, the one in front of the MLP. On the left every column of the window is a point $(\mathbf{1}^\top x/\sqrt{D},\ \|\Pi x\|)$; centring moves it across onto the vertical axis, scaling moves it along that axis to just inside the circle $\|x\| = \sqrt{D}$, and the gain and offset move it off again. The inset magnifies how far inside each column lands. On the right, one column as a spoke glyph at each stage, with the dotted circle at root mean square 1. Run the stages, or click a column.



Why normalise at all? The stream accumulates: every branch adds to it, and nothing in the forward pass ever shrinks it. A

branch that read the raw stream would see inputs whose size depends on how much has been written before, and would have to work at every scale. A normalised copy has a fixed size, so each branch sees its input on the same footing, whatever its depth. The price is that a branch cannot see the mean or the length of a column; whatever the model wants a branch to know must be written in the direction. In Chapter 7 the same geometry reappears in the derivative: the pullback of LayerNorm projects the incoming adjoint onto $\mathbf{1}^{\perp}$ and onto the tangent space of the sphere, so no gradient ever asks to change what LayerNorm forgets.

Ba, Kiros and Hinton introduced layer normalisation in 2016. The original transformer (Vaswani et al., 2017) normalised *after* each residual sum, $\mathrm{LN}(x + f(x))$, which normalises the stream itself. GPT-2 (Radford et al., 2019) moved the normalisation to the input of each branch, as here, and added one more before the readout; this "pre-norm" arrangement leaves the stream un-normalised, and is the one most models now use.

6.3 The MLP

The second branch is the same map for every column, a composition of an affine map, an entrywise nonlinearity, and another affine map. For the column $\hat{f} = \mathrm{LN}_2(x_{\mathrm{mid}}) \in \mathbb{R}^D$,

$$u = W_1 \hat{f} + b_1 \in \mathbb{R}^F, \qquad r = \mathrm{relu}(u), \qquad y = W_2 r + b_2 \in \mathbb{R}^D,$$

with $W_1 \in \mathbb{R}^{F \times D}$, $W_2 \in \mathbb{R}^{D \times F}$, $b_1 \in \mathbb{R}^F$, $b_2 \in \mathbb{R}^D$ and the *rectifier* $\mathrm{relu}(u)_k = \max(u_k, 0)$. The width F of the middle is $4D$ by convention, 32 in the specimen. On all the columns at once this reads $Y = W_2 \mathrm{relu}(W_1 \hat{F} + b_1 \mathbf{1}^{\top}) + b_2 \mathbf{1}^{\top}$, and the stream becomes $X' = X_{\mathrm{mid}} + Y$. ML calls the branch a *multilayer perceptron* or *feed-forward layer*, and its F middle coordinates *hidden units*.

The map is clearest read through the columns of W_2. Write $w_k \in \mathbb{R}^D$ for row k of W_1; then

$$y = \sum_{k=1}^{F} \max\left(w_k \cdot \hat{f} + b_{1k}, 0 \right) (W_2)_{:,k} + b_2.$$

Unit k owns a fixed vector, the column $(W_2)_{:,k} \in \mathbb{R}^D$, and a detector, the half-space $w_k \cdot \hat{f} + b_{1k} > 0$. When the input lies in the half-space the unit *fires* and adds its vector, scaled by how far inside the input lies; otherwise it adds nothing at all. The MLP is a sum of F switched sources. On each region of $\mathbb{R}^D$ where the set of firing units does not change, it is an affine map, so the whole branch is continuous and piecewise affine, with flat pieces glued along the F hyperplanes $w_k \cdot \hat{f} + b_{1k} = 0$. The rectifier is the only nonlinearity of this kind in the model; the others are the softmax and the normalisation.

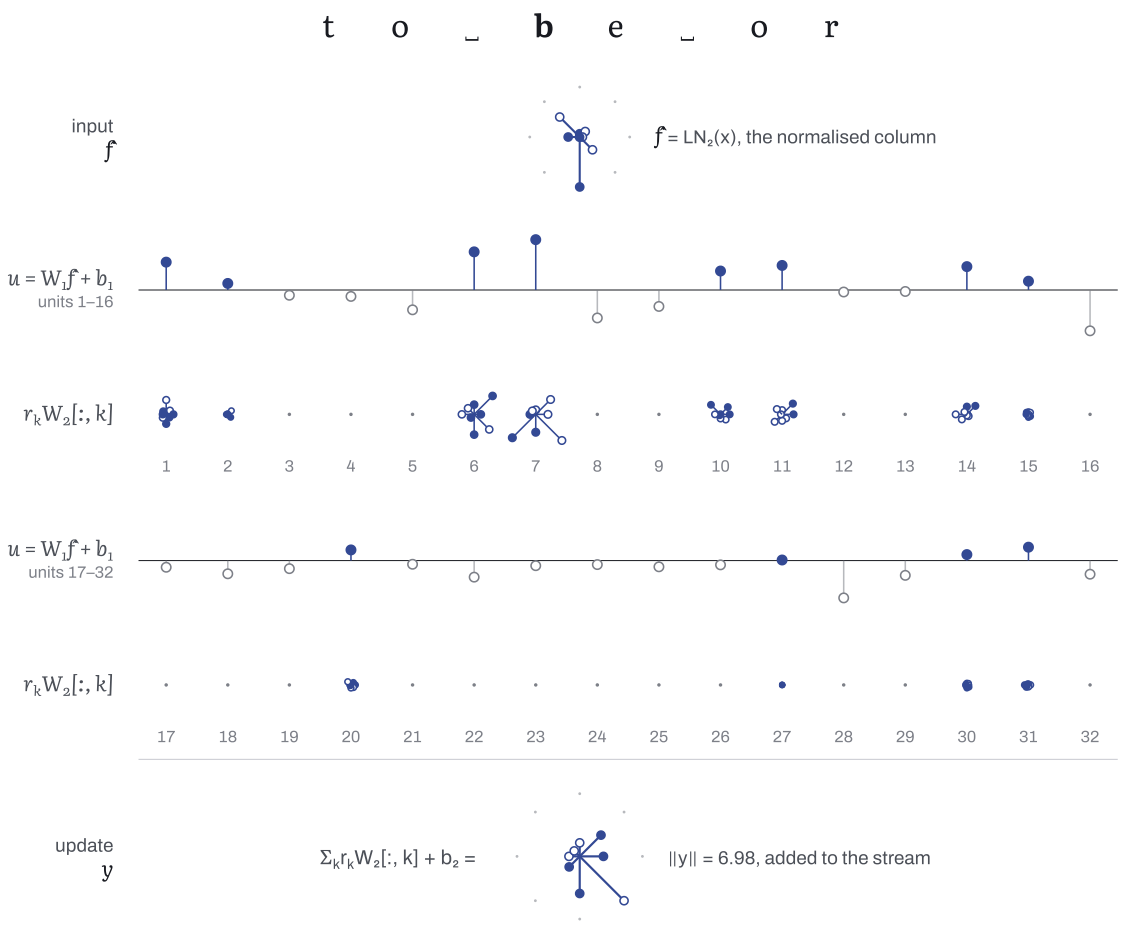
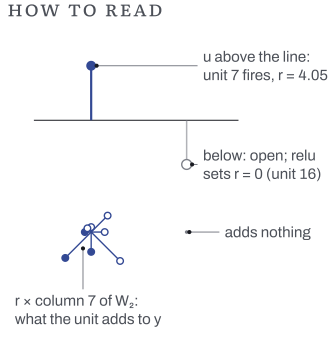


Figure 6.3 The MLP of one column. The normalised column $\hat{f}$ is mapped to F numbers u_k, drawn about a zero line; the rectifier keeps the positive ones and sets the others to zero. Below each firing unit is what it adds, r_k times its column of W_2, on one scale with their sum, the update y at the foot. Click a column, or point at a unit.



Unlike attention, the MLP never looks at another column: a token computes here alone, with the same W_1, W_2 for every position. Between them, W_1 and W_2 hold more than half of the

specimen's parameters. Attention decides *where* to read; the MLP is where most of the model's fixed, learned content sits, a table of F vectors indexed by F half-spaces.

6.4 One composition

We can now write the whole model down. With C the one-hot matrix of the input characters and B the block,

$$\ell(\theta) = \mathrm{CE}\Big(\mathrm{softmax}\big(W_{\text{out}}\ \mathrm{LN}_f\big(B(EC + P) \big) + b_{\text{out}}\mathbf{1}^{\mathsf{T}} \big),\ y \Big).$$

Read from the inside out: embed, run the block, normalise once more, map each column to V logits with the *readout* $Z = W_{\text{out}}X_f + b_{\text{out}}\mathbf{1}^{\mathsf{T}}$, turn each column of logits into the Gibbs distribution of Chapter 3, and charge the cross-entropy of Chapter 4 against the true next characters y. A model with L blocks has $B_L \circ \cdots \circ B_1$ in place of B. Every arrow in Figure 6.4 is one of these maps.

All the parameters together are one vector $\theta \in \mathbb{R}^p$. Counting along the arrows,

$$\begin{aligned} p &= \underbrace{DV}_{E} + L\big(\underbrace{4D^2 + 4D}_{\text{attention}} + \underbrace{2DF + F + D}_{\text{MLP}} + \underbrace{4D}_{\mathrm{LN}_1,\ \mathrm{LN}_2} \big) + \underbrace{2D}_{\mathrm{LN}_f} + \underbrace{DV + V}_{\text{readout}} \\ &= 2DV + V + 2D + L\,(4D^2 + 2DF + 9D + F), \end{aligned}$$

which for the specimen is $p = 1007$. Neither the context length T nor the number of heads H appears: positions are a fixed code and heads only partition rows that are already counted. In the code the named parameters $E, \gamma_1, \ldots, b_{\text{out}}$ are *views* onto one vector, in a fixed order, so that an optimiser in Chapter 8 sees nothing but a point in $\mathbb{R}^p$ and a vector of the same length to move it by.

Running this composition pushes values forward through every map, top to bottom. Training will need the opposite: the derivative of the one number at the bottom with respect to all p numbers at the side. That is the subject of Chapter 7.

6.5 In the code

The block is written from four functions. The first two are the affine map, applied to every column with the same weights, and the rectifier:

The rectifier is an ideal diode: a small change of u_k passes through when the unit fires and is blocked when it does not. At $u_k = 0$ exactly it has no derivative; the code uses the slope 0 there, a convention rather than a theorem (Chapter 7). The perceptron is Rosenblatt's (1958); the transformer paper calls this branch the "position-wise feed-forward network".

The order is binding: E; then for each block $\gamma_1\ \beta_1\ W_q\ b_q\ W_k\ b_k\ W_v\ b_v\ W_o\ b_o\ \gamma_2\ \beta_2\ W_1\ b_1\ W_2\ b_2$; then $\gamma_f\ \beta_f\ W_{\text{out}}\ b_{\text{out}}$; each matrix column by column. `carve(θ, c)` in `src/parameters.jl` cuts θ into these views, and `flatten` glues the gradients back together in the same order.



Figure 6.4 The specimen as one composition of maps, from characters to a single number. Each arrow is a map, with the parameters it uses drawn as squares of area proportional to how many numbers they hold; each node is a space, labelled with the shape of its array for one window. Only attention acts across columns.

src/block.jl affine · lines 16–20

```
function affine(X, W, b)
    Y = muladd(W, X, b)
    back(Ȳ) = (W' * Ȳ, Ȳ * X', vec(sum(Ȳ, dims = 2)))
    Y, back
end
```

src/block.jl relu · lines 29–33

```
function relu(U)
    R = max.(U, 0)
    back(R̄) = R̄ .* (R .> 0)
    R, back
end
```

LayerNorm computes μ, σ and $\hat{x}$ for every column at once; its pullback is derived in Chapter 7.

src/block.jl layernorm · lines 52–64

```
function layernorm(X, γ, β)
    D = size(X, 1)
    ε = eltype(X)(1e-5)
    X̃ = X .- sum(X, dims = 1) ./ D                          # centre
    σ = sqrt.(sum(abs2, X̃, dims = 1) ./ D .+ ε)
    X̂ = X̃ ./ σ                                              # inside the sphere
    function back(Ȳ)
        G = γ .* Ȳ
        X̄ = (G .- sum(G, dims = 1) ./ D .- X̂ .* sum(X̂ .* G, dims = 1) ./ D) ./ σ
        X̄, vec(sum(Ȳ .* X̂, dims = 2)), vec(sum(Ȳ, dims = 2))
    end
    γ .* X̂ .+ β, back
end
```

The block itself is the two residual sums, and its pullback reads the same lines backwards:

src/block.jl block · lines 79–112

```
function block(X₀, p, H, T; trace = nothing)
    Â,  back_Â  = layernorm(X₀, p.γ1, p.β1)
    Q,  back_Q  = affine(Â, p.Wq, p.bq)
    K,  back_K  = affine(Â, p.Wk, p.bk)
    V,  back_V  = affine(Â, p.Wv, p.bv)
    A,  back_A  = attention(Q, K, H, T)
    O,  back_O  = mix(A, V)
    Y₁, back_Y₁ = affine(O, p.Wo, p.bo)
    X₁ = X₀ + Y₁
    F̂,  back_F̂  = layernorm(X₁, p.γ2, p.β2)
    U,  back_U  = affine(F̂, p.W1, p.b1)
    R,  back_R  = relu(U)
```

```
    Y₂, back_Y₂ = affine(R, p.W2, p.b2)
    X₂ = X₁ + Y₂
    record!(trace, (; ln1 = Â, Q, K, V, A, O, Xmid = X₁, ln2 = F̂, U, R, Y = Y₂, Xout = X₂))

    function back(X̄₂)
        R̄, W̄2, b̄2   = back_Y₂(X̄₂)
        Ū            = back_R(R̄)
        F̄̂, W̄1, b̄1   = back_U(Ū)
        X̄₁, ȳ2, β̄2  = back_F̂(F̄̂)
        X̄₁ .+= X̄₂                                          # the residual path
        Ō, W̄o, b̄o   = back_Y₁(X̄₁)
        Ā, V̄         = back_O(Ō)
        Q̄, K̄         = back_A(Ā)
        Âq, W̄q, b̄q  = back_Q(Q̄)
        Âk, W̄k, b̄k  = back_K(K̄)
        Âv, W̄v, b̄v  = back_V(V̄)
        X̄₀, ȳ1, β̄1  = back_Â(Âq + Âk + Âv)
        X̄₀ .+= X̄₁                                          # the residual path
        X̄₀, (ȳ1, β̄1, W̄q, b̄q, W̄k, b̄k, W̄v, b̄v, W̄o, b̄o, ȳ2, β̄2, W̄1, b̄1, W̄2, b̄2)
    end
    X₂, back
end
```

The whole model is the embedding, L blocks, a final LayerNorm and the readout:

src/model.jl gpt · lines 16–42

```
function gpt(θ, x, c::Config; trace = nothing)
    T = size(x, 1)
    T > 0 || throw(ArgumentError("sequences must contain at least one token"))
    T ≤ c.T || throw(ArgumentError("sequences of length $T exceed the context length $(c.T)"))
    p = carve(θ, c)
    X, back_embed = embed(x, p.E)
    record!(trace, (; X0 = X))
    back_blocks = Vector{Any}(undef, c.L)
    for ℓ in 1:c.L
        X, back_blocks[ℓ] = block(X, p.blocks[ℓ], c.H, T; trace)
    end
    Xf, back_Xf = layernorm(X, p.γf, p.βf)
    Z,  back_Z  = affine(Xf, p.Wout, p.bout)
    record!(trace, (; Xf, Z))

    function back(Z̄)
        X̄f, W̄out, b̄out = back_Z(Z̄)
        X̄, γ̄f, β̄f = back_Xf(X̄f)
        p̄blocks = Vector{Any}(undef, c.L)
        for ℓ in c.L:-1:1
            X̄, p̄blocks[ℓ] = back_blocks[ℓ](X̄)
        end
        Ē = back_embed(X̄)
        flatten(Ē, p̄blocks, γ̄f, β̄f, W̄out, b̄out)
```

```
    end
    Z, back
end
```

Every function returns its value together with its pullback, `back`. The forward pass is the first half of each function; Chapter 7 is about the second.

Exercises

EXERCISE 6.1

Show that $\Pi = I - \mathbf{1}\mathbf{1}^{\top}/D$ is symmetric and idempotent, with kernel $\operatorname{span}(\mathbf{1})$ and image $\mathbf{1}^{\perp}$. Then show $\|\hat{x}\|^2 = D(1 - \varepsilon/\sigma^2)$, and deduce that, for $\|u\|^2 \gg D\varepsilon$, the column lands a distance $\sqrt{D} - \|\hat{x}\| \approx \varepsilon D^{3/2}/(2\|u\|^2)$ inside the sphere.

EXERCISE 6.2

Show that for $\varepsilon = 0$, $\mathrm{LN}(cx + b\mathbf{1}) = \mathrm{LN}(x)$ for every $c > 0$ and $b \in \mathbb{R}$. What does LayerNorm do to $-x$? With $\varepsilon > 0$, which of these invariances survive exactly?

EXERCISE 6.3

Let $S \subseteq \{1, \dots, F\}$ be the set of units that fire at $\hat{f}$, and P_S the diagonal matrix with ones at the positions in S. Show that near $\hat{f}$ (away from the hyperplanes) the MLP is the affine map $\hat{f} \mapsto W_2 P_S W_1 \hat{f} + W_2 P_S b_1 + b_2$. What is its rank at most? Show that the whole branch is continuous.

EXERCISE 6.4

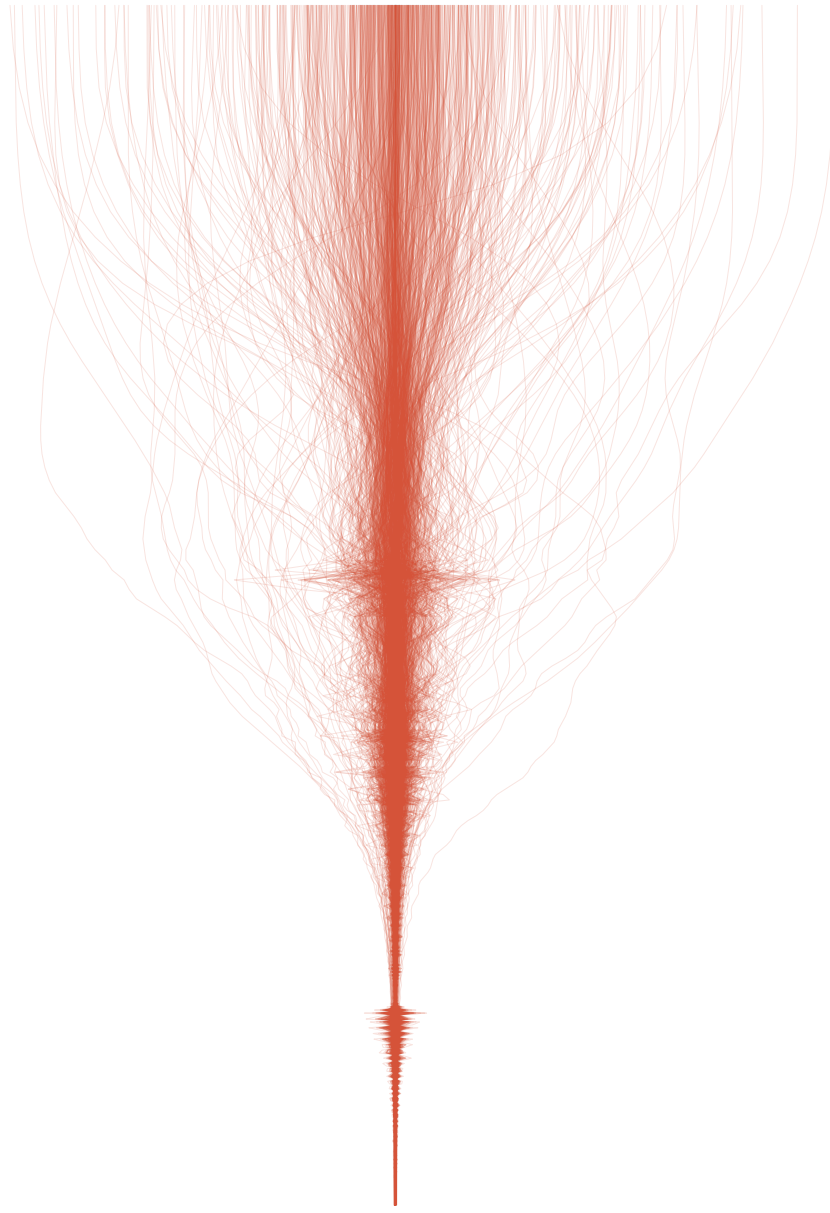
Try it in Figure 6.2 (p. 73). Which column of the window lands deepest inside the sphere? Check your answer against the first exercise, using $\|\Pi x\|$ from the readout.

EXERCISE 6.5

Try it in Figure 6.3 (p. 75). Click through the eight columns of the window. Is there a unit that fires for every one of them, and one that fires for none? What does a unit that never fires contribute to the window, and what will the gradient of its weights be (Chapter 7)?

EXERCISE 6.6

Derive the parameter count $p = 2DV + V + 2D + L(4D^2 + 2DF + 9D + F)$ from Figure 6.4 (p. 77), and evaluate it for the Shakespeare model of Chapter 9, with $V = 65$, $D = 128$, $F = 512$ and $L = 4$. Which single kind of array dominates as D grows?



Chapter 7. *Every component of the gradient $\nabla\vartheta \in \mathbb{R}^{1007}$, followed through the same training run as the cover.* Time runs down the plate on a logarithmic scale (200 steps, AdamW, η 0.02 → 0.002 (warmup 20, cosine), β_2 0.95, weight decay 0.01 on matrices, clip 1). Each path is one component of $\partial\ell/\partial\theta_i$ (before clipping), averaged as Adam averages it (first moment, β_1 = 0.9, bias-corrected), placed horizontally at $\operatorname{asinh}(\partial\ell/\partial\theta_i \,/\, g_0)$ with g_0 = 3.03e-2 (the initial root-mean-square component): linear near zero, logarithmic beyond. As the loss reaches its floor every component falls back to zero.

CHAPTER SEVEN

7

Pulling back

The derivative of the loss with respect to every parameter costs about as much as one forward pass: start at the loss with the number one, and pull it back through the transpose of each map's derivative, in reverse order, adding wherever a value was used twice.

MATHS	PHYSICS	ML
$Df(x)$, pushing dx forward to dy	*linear response*	"Jacobian, forward-mode derivative"
$\bar{x} = Df(x)^{\mathsf{T}}\bar{y}$	*the adjoint map*	"backpropagation through f, vector–Jacobian product"
$\bar{x} = \partial\ell/\partial x$, a covector	*a generalised force, conjugate to x*	"the gradient, "grad""
$(f_k \circ \cdots \circ f_1)^* = f_1^* \circ \cdots \circ f_k^*$	*the adjoint method*	"backward pass, reverse-mode differentiation"
$\Delta^*(\bar{u}, \bar{v}) = \bar{u} + \bar{v}$	*responses superpose*	"gradient accumulation"
$\big(\,\ell(\theta + he_i) - \ell(\theta - he_i)\,\big)/2h$	*a symmetric perturbation experiment*	"numerical gradient, gradient check"

7.1 One number, many parameters

FIX THE TRAINING WINDOWS. THEN THE MODEL AND ITS loss are one function of the parameters,

$$\ell : \mathbb{R}^p \longrightarrow \mathbb{R}, \qquad \theta \longmapsto \text{the mean surprise of the targets,}$$

with $p = 1007$ for the specimen. Learning, in Chapter 8, moves θ downhill, and for that it needs the gradient $\nabla\ell(\theta) \in \mathbb{R}^p$: for every parameter, how fast the loss changes when that parameter alone is nudged.

The obvious way to get it is to nudge. For each coordinate i,

$$\frac{\partial \ell}{\partial \theta_i} \approx \frac{\ell(\theta + he_i) - \ell(\theta - he_i)}{2h},$$

which costs two evaluations of ℓ, that is two forward passes, per parameter: $2p = 2014$ for the specimen and about 1.6 million for the Shakespeare model of Chapter 9, every time the parameters move. It is also only approximate, and we shall see in §7.5 (p. 92) that its accuracy cannot be pushed very far.

This chapter derives the other way. One forward pass computes the loss and records its intermediate values. One backward pass then computes all p derivatives exactly (up to rounding), at a cost of a small constant number of forward passes, whatever p is. The backward pass is not a new idea; it is the chain rule, read from the end. What makes it cheap is the order in which the chain rule's factors are multiplied, and what makes it simple is that each factor is a transpose.

It helps to be careful about what kind of object the gradient is. The derivative of ℓ at θ is a linear map $d\ell(\theta) : \mathbb{R}^p \to \mathbb{R}$, which eats a displacement h of the parameters and returns the first-order change of the loss. A linear map to the numbers is a *covector*. On $\mathbb{R}^p$ with its usual inner product every covector is $h \mapsto \langle g, h\rangle$ for exactly one vector g, and that vector is called the gradient. So $\nabla\ell$ is a covector wearing the clothes of a vector. It answers the question "how much does the loss change if I move this way?", not "where is the parameter?". This is the distinction the book's two inks draw: values, which are pushed forward, in value ink; and covectors, which are pulled back from the loss, in covector ink.

7.2 The pullback

Let $f : \mathbb{R}^n \to \mathbb{R}^m$ be differentiable, $y = f(x)$. Its derivative at x is the linear map $Df(x) : \mathbb{R}^n \to \mathbb{R}^m$ with

$$f(x + dx) = f(x) + Df(x)\,dx + o(\|dx\|).$$

It pushes a small change of the input forward to the change of the output, $dy = Df(x)\,dx$. This is the forward direction, the direction in which the model computes.

Now suppose that y is used, further on, to compute a loss, and that we know how the loss responds to y: a covector $\bar{y} \in \mathbb{R}^m$ with $d\ell = \langle \bar{y},\, dy \rangle$. Then

$$d\ell = \langle \bar{y},\, Df(x)\, dx \rangle = \langle Df(x)^\top \bar{y},\, dx \rangle,$$

and the response of the loss to x is therefore

$$\boxed{\bar{x} = Df(x)^\top \bar{y}}, \qquad \bar{x}_j = \sum_{i=1}^{m} \bar{y}_i \frac{\partial y_i}{\partial x_j}.$$

The map $\bar{y} \mapsto \bar{x}$ is the *pullback* of f at x. It runs against the computation, from the output space back to the input space, and it is linear. The bar is the book's notation for it: $\bar{x} = \partial \ell / \partial x$ has the same shape as x, and is read "x-bar", or the adjoint of x. In matrix form the pullback is the transpose of the Jacobian; in practice the Jacobian is never formed, and the pullback is a function that computes $Df(x)^\top \bar{y}$ directly, as cheaply as f itself.

Compose. The model is a chain of maps, $\ell = f_k \circ \cdots \circ f_1$, the last of which returns a single number. With J_i the derivative of f_i at its own input, the chain rule says $D\ell = J_k J_{k-1} \cdots J_1$, and transposing reverses the order:

$$\nabla \ell = (J_k \cdots J_1)^\top \cdot 1 = J_1^\top \Big(J_2^\top \big(\cdots (J_k^\top \cdot 1) \cdots \big) \Big).$$

Read the brackets from the inside out. Start at the loss with $\bar{\ell} = 1$; apply the pullback of the last map, then of the one before it, and so on back to the parameters. Every intermediate result is a covector of the size of the value it belongs to, and no step multiplies two matrices. The backward pass is this bracket, evaluated. The same product can be bracketed the other way, $J_k(\cdots(J_2(J_1 v)))$: push one direction v forward through the maps. That is *forward-mode* differentiation; it yields one directional derivative $D\ell\, v$ per pass, so the whole gradient takes p passes, one per coordinate, no better than finite differences in count. Reverse mode yields the whole gradient in one pass because the chain ends in one number. For a map with many outputs and few inputs the comparison would go the other way.

How much does it cost? Each map of the model is a matrix product, an elementwise function, a sum, a softmax or a lookup, and the pullback of each costs at most a small constant times the map itself; we derive them all below. Adding up, a forward and a backward pass together cost a constant number of forward passes, independently of p. In the web edition, Figure 7.4 (p. 93) measures the constant in your browser. The price is memory: the

Reverse-mode differentiation was described by Seppo Linnainmaa in his 1970 master's thesis, as a way of computing the rounding error of a long calculation. Paul Werbos proposed it for neural networks in 1974; Rumelhart, Hinton and Williams made it famous as *backpropagation* in 1986. Physicists know it as the adjoint method: the costate of Pontryagin's optimal control, and the adjoint equations of weather forecasting, which pull the sensitivity of one forecast number back through a whole simulation.

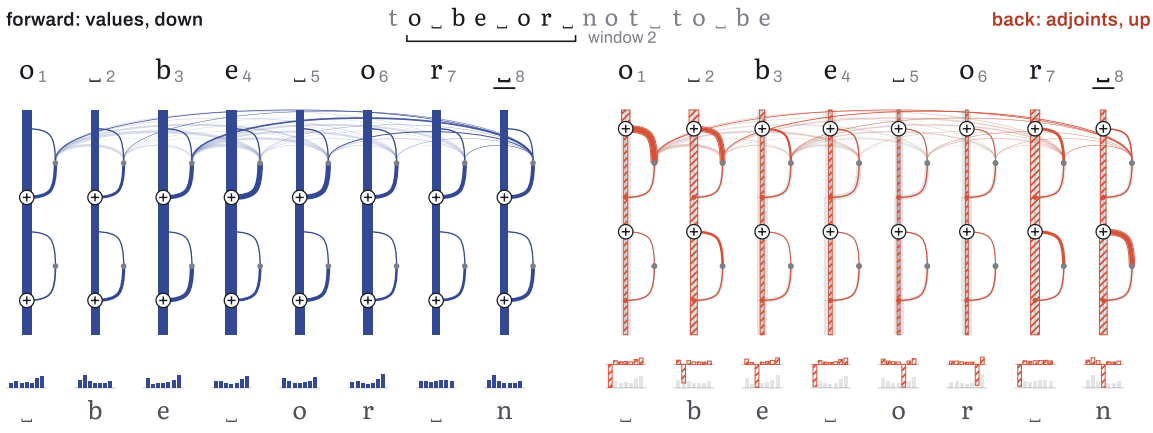
pullback of a map needs the values that went into it, so the forward pass must keep its intermediates until the backward pass has used them.

In the code, every operation returns its value *together with* its pullback, a function `back` that has kept what it needs. The loss itself is two such operations, and its pullback is the composition of theirs, in reverse order:

src/model.jl loss · lines 50–57

```
function loss(θ, x, y, c::Config)
    size(x) == size(y) || throw(DimensionMismatch("inputs and targets must have the same shape"))
    isempty(x) && throw(ArgumentError("the mean loss needs a nonempty batch"))
    Z, back_Z = gpt(θ, x, c)
    ℓ, back_ℓ = crossentropy(Z, y)
    back(ℓ̄ = 1) = back_Z(back_ℓ(ℓ̄))
    ℓ, back
end
```

Figure 7.1 shows the whole of it on the specimen's stems: the values flow down, then one covector per token flows back up the same paths.



HOW TO READ

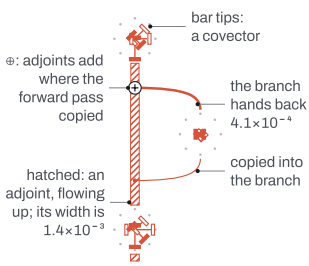


Figure 7.1 Forward, then back. The forward pass flows down the stems in value ink; then the loss sends one covector up each stem, through the same maps in reverse order, in covector ink. The size of each adjoint is drawn where it lands. At every + the adjoint goes both ways, and at every fork the adjoints coming back add. Scrub the clock, or click a stem. Then switch to the trained model: only the predictions whose context is ambiguous still send anything back.

7.3 Fan-out: adjoints add

A chain is the simplest program. The model is not quite a chain: some values are used more than once. The rule for that case looks like the one rule of the backward pass that is not a transpose. It is one too.

Using a value twice is applying the duplication map $\Delta(x) = (x, x)$. Its derivative duplicates perturbations, $D\Delta\, h = (h, h)$, and

$$\langle(\bar{u}, \bar{v}),\ (h, h)\rangle = \langle \bar{u} + \bar{v},\ h\rangle, \qquad \text{so} \qquad \Delta^*(\bar{u}, \bar{v}) = \bar{u} + \bar{v}.$$

The adjoint of a value used several times is the sum of the adjoints of its uses. A responsive quantity has several routes to the loss, and their responses superpose. Three places in the model use this.

The residual stream. A residual update $y = x + f(x)$ uses x twice, once on the identity path and once in the branch. Its pullback is

$$\bar{x} = \bar{y} + Df(x)^\top\, \bar{y}.$$

The first term is the adjoint travelling straight back along the stream; the second is the same adjoint pulled through the branch. The backward pass of a residual network has a residual stream of its own, and it is the transpose of the forward one: the linearised update $dX_{k+1} = (I + Df_k)\, dX_k$ of the Euler steps of §6.1 (p. 69) has the adjoint $\bar{X}_k = \bar{X}_{k+1} + Df_k^\top \bar{X}_{k+1}$. The identity gives every branch a direct line to the loss. It does not, by itself, keep the product of many such factors from growing or shrinking.

One input, three maps. In attention the normalised column $\hat{x}$ feeds the query, key and value maps. Its adjoint is the sum of the three pullbacks,

$$\bar{\hat{x}} = W_q^\top \bar{q} + W_k^\top \bar{k} + W_v^\top \bar{v}.$$

Parameters used at every token. A bias is added to every column, $Y = WX + b\mathbf{1}^\top$, so its adjoint is a sum over the columns, $\bar{b} = \bar{Y}\mathbf{1}$: a sum, not a mean (the single factor $1/N$ of the mean lives in the loss). The embedding is the extreme case. Column a of E is read once for every occurrence of the character a, so

$$\bar{E}_{:,a} = \sum_{n\,:\,x_n = a} \bar{X}_{0,\,:,\,n},$$

the adjoints of all the positions where a occurs, added into one column. This is called a scatter-add. Overwriting instead of

In the code the two residual adjoints are the lines `X̄₁ .+= X̄₂` and `X̄₀ .+= X̄₁` in the pullback of `block`, and the three-way sum is `back_Â(Âq + Âk + Âv)` (§6.5). The residual lines are marked with a comment.

adding is a classic bug: it throws away all but one observation of each character.

7.4 The pullbacks, one by one

Every map in the model is one of six kinds. Here is the pullback of each, derived by the recipe of §7.2 (p. 84): write the differential dy, pair it with $\bar{y}$, and move everything onto dx. Matrices carry the Frobenius inner product $\langle U, V\rangle = \operatorname{tr}(U^\top V) = \sum_{ij} U_{ij}V_{ij}$, and the cyclic property of the trace does the moving.

Affine maps

$Y = WX + b\mathbf{1}^\top$ with $W \in \mathbb{R}^{r\times s}$, $X \in \mathbb{R}^{s\times N}$, $b \in \mathbb{R}^r$: the readout, the query, key, value and output maps, and both layers of the MLP. The differential is $dY = dW\,X + W\,dX + db\,\mathbf{1}^\top$, and

$$\langle \bar{Y},\, dY\rangle = \langle \bar{Y}X^\top,\, dW\rangle + \langle W^\top\bar{Y},\, dX\rangle + \langle \bar{Y}\mathbf{1},\, db\rangle.$$

Read off the three adjoints, in matrix form and in index form:

$$\bar{W} = \bar{Y}X^\top, \quad \bar{W}_{ij} = \sum_n \bar{Y}_{in}X_{jn}; \qquad \bar{X} = W^\top\bar{Y}, \quad \bar{X}_{jn} = \sum_i W_{ij}\bar{Y}_{in}; \qquad \bar{b} = \bar{Y}\mathbf{1}, \quad \bar{b}_i = \sum_n \bar{Y}_{in}.$$

The input adjoint is the output adjoint pulled back through the transpose. The weight adjoint is a sum over the columns of outer products, $\bar{W} = \sum_n \bar{y}_n\, x_n^\top$: for every token, the response at the output times the signal at the input. Each token contributes a matrix of rank one, and the gradient of W is their sum. Figure 7.2 shows it for the readout, where $\bar{z}$ comes straight from the loss.

The rectifier

$r = \operatorname{relu}(u)$, entry by entry. Where $u_i > 0$ the map is the identity and where $u_i < 0$ it is constant, so $Dr(u) = \operatorname{diag}(\mathbf{1}_{u>0})$, a diagonal of ones and zeros, and

$$\bar{u} = \bar{r} \odot \mathbf{1}_{u>0}.$$

The pullback is a mask: a unit that was off in the forward pass passes no adjoint back. At $u_i = 0$ there is no derivative, the left and right slopes being 0 and 1; the code returns 0 there. That is a convention, not a theorem, and finite differences must stay away from the kink to check it.

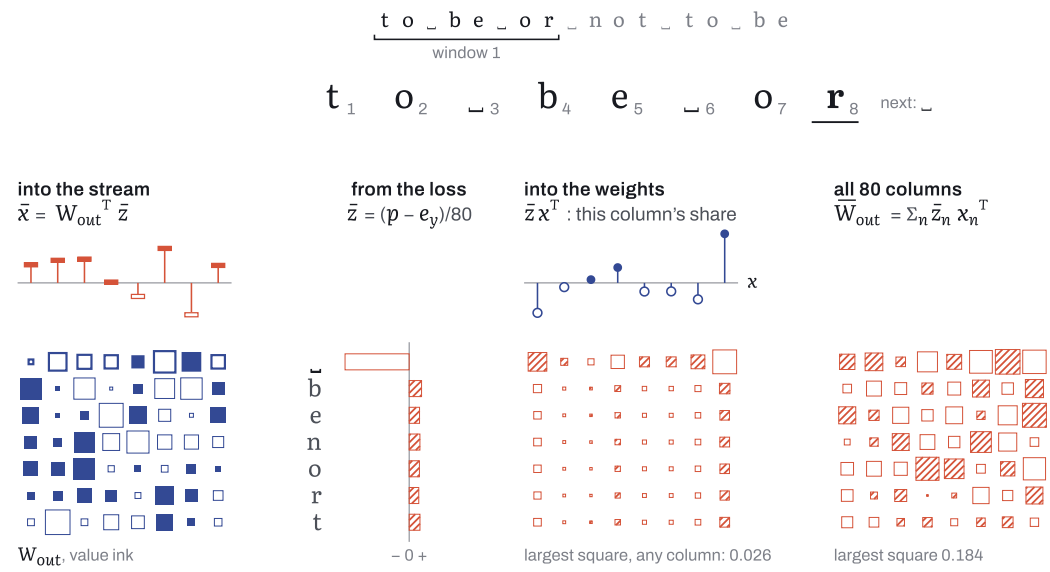
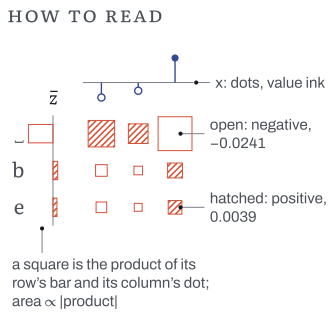


Figure 7.2 The pullback of the readout $z = W_{\text{out}}x + b_{\text{out}}$ for one column, before training. The adjoint $\bar{z} = (p - e_y)/N$ arrives from the loss; $\bar{x} = W_{\text{out}}^{\top}\bar{z}$ carries it back into the stream, and $\bar{z}\,x^{\top}$ is this column's rank-one share of the gradient of the weights. The full gradient $\overline{W}_{\text{out}}$ is the sum of the shares of all the columns. Click a column, move the window, point at a square.



Softmax, and the seed of everything

For $p = \mathrm{softmax}(z)$, the quotient rule gives $\partial p_i / \partial z_j = p_i \delta_{ij} - p_i p_j$, so

$$Dp(z) = \mathrm{diag}(p) - pp^{\top}, \qquad \bar{z} = p \odot \left(\bar{p} - \langle p, \bar{p} \rangle \mathbf{1} \right).$$

This Jacobian is symmetric, and for any v, $v^{\top} Dp(z)\, v = \sum_i p_i v_i^2 - \left(\sum_i p_i v_i \right)^2 = \mathrm{Var}_{i \sim p}(v_i) \geq 0$: it is the covariance matrix of the Gibbs distribution of Chapter 3. It annihilates $\mathbf{1}$, which is shift invariance once more: adding the same number to every logit moves nothing, so the pullback never contains such a component, and every $\bar{z}$ it produces has entries summing to zero.

The model never pulls back through the softmax of its output, because the loss is written directly in terms of the logits (Chapter 4):

$$\ell = \frac{1}{N} \sum_{n=1}^{N} \left[\mathrm{lse}(z_n) - Z_{y_n, n} \right], \qquad \boxed{\bar{Z}_{:,n} = \frac{p_n - e_{y_n}}{N}},$$

since the derivative of lse is the softmax and that of $Z_{y,n}$ picks out the target. This is the seed of the backward pass: the prediction minus the truth, one column per token, with the loss's single factor $1/N$.

src/softmax.jl crossentropy · lines 43–60

```
function crossentropy(Z, y)
    N = size(Z, 2)
    N > 0 || throw(ArgumentError("cross-entropy needs at least one token"))
    length(y) == N || throw(DimensionMismatch("one target is required per logit column"))
    m = maximum(Z; dims = 1)
    s = map(eachcol(Z), vec(m)) do z, mₙ
        sum(zₐ -> exp(zₐ - mₙ), z)
    end
    ℓ = sum(log(s[n]) - (Z[y[n], n] - m[n]) for n in 1:N) / N
    function back(ℓ̄ = 1)
        Z̄ = exp.(Z .- m) ./ s'                    # softmax(zₙ), for every n
        for n in 1:N
            Z̄[y[n], n] -= 1                       # - onehot(yₙ)
        end
        Z̄ .*= eltype(Z)(ℓ̄) / N
    end
    ℓ, back
end
```

LayerNorm

For one column (§6.2, p. 71), $u = \Pi x$, $\sigma = \sqrt{\|u\|^2/D + \varepsilon}$, $\hat{x} = u/\sigma$, $y = \gamma \odot \hat{x} + \beta$. The affine part is easy: $\bar{\gamma} = \sum_n \bar{y}_n \odot \hat{x}_n$, $\bar{\beta} = \sum_n \bar{y}_n$, and the adjoint of $\hat{x}$ is $g = \gamma \odot \bar{y}$. For the rest, $du = \Pi\, dx$ and $d\sigma = u^\top dx/(D\sigma)$, which gives

$$d\hat{x} = \frac{1}{\sigma}\Big(\Pi - \frac{\hat{x}\hat{x}^\top}{D}\Big)\, dx.$$

This Jacobian is symmetric, so it is its own transpose, and

$$\boxed{\bar{x} = \frac{1}{\sigma}\Big(\Pi - \frac{\hat{x}\hat{x}^\top}{D}\Big)\, g = \frac{1}{\sigma}\Big(g - \operatorname{mean}(g)\,\mathbf{1} - \hat{x}\,\operatorname{mean}(g \odot \hat{x})\Big)}$$

for every $\varepsilon \geq 0$. The geometry of the forward map reappears in its pullback. Π removes the component of g along $\mathbf{1}$, which LayerNorm cannot see, since centring forgets it. The second term removes the component along $\hat{x}$, the radial direction, which normalisation forgets: rescaling a column does not change where it lands on the sphere. What survives is the part of g tangent to the sphere and orthogonal to $\mathbf{1}$, divided by σ. With $\varepsilon > 0$ the radial response is not exactly zero but small, of order ε/σ^2, because the column lands only just inside the sphere; the formula is exact either way.

Attention

Per head and window (Chapter 5): $S = Q^\top K/\sqrt{d}$, $A = \operatorname{softmax}_{\text{rows}}(S + M)$, $O = VA^\top$. Pull back in reverse order. The product $O = VA^\top$ is bilinear, and the affine rule, applied to each factor, gives

$$\bar{V} = \bar{O}A, \qquad \bar{A} = \bar{O}^\top V, \qquad \bar{A}_{ij} = \bar{o}_i \cdot v_j.$$

Notice that $\bar{A}$ is a full $T \times T$ matrix. The product $VA^\top$ does not know that A is lower-triangular, and $\bar{A}_{ij}$ for $j > i$ is how the loss *would* respond if reader i could read the future source j. Next comes the softmax of each row, restricted to the allowed sources $j \leq i$; by the softmax rule,

$$\bar{S}_{ij} = \begin{cases} A_{ij}\,\big(\,\bar{A}_{ij} - \sum_{k \leq i} A_{ik}\bar{A}_{ik}\,\big), & j \leq i, \\ 0, & j > i. \end{cases}$$

The factor A_{ij}, which is zero above the diagonal, discards the future: no gradient comes from it. The mask is not differentiated;

it restricts the function to the allowed coordinates. Finally $S = Q^\top K/\sqrt{d}$ is bilinear in Q and K:

$$\bar{Q} = \frac{K\bar{S}^\top}{\sqrt{d}}, \quad \bar{q}_i = \frac{1}{\sqrt{d}} \sum_j \bar{S}_{ij}\, k_j; \qquad \bar{K} = \frac{Q\bar{S}}{\sqrt{d}}, \quad \bar{k}_j = \frac{1}{\sqrt{d}} \sum_i \bar{S}_{ij}\, q_i.$$

Three checks of orientation come for free: the first row of $\bar{S}$ is zero, since $A_{11} = 1$ cannot change; everything above the diagonal is zero; and every row of $\bar{S}$ sums to zero. The last one is worth a section of its own (§7.6, p. 94). The code is the `back` of `attention` and of `mix` in §5.7 (p. 65).

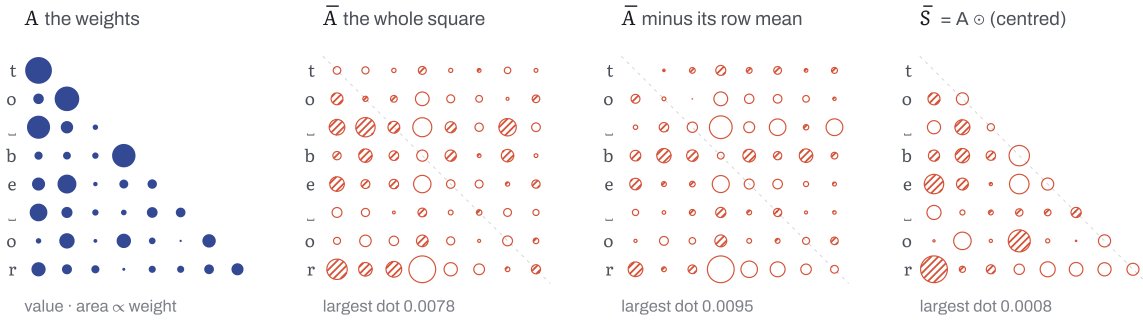


Figure 7.3 No gradient comes from the future. The weights A of one head before training, the adjoint $\bar{A}$, which the product $O = VA^\top$ defines on the whole square, and the adjoint $\bar{S}$ of the scores. Run the pullback: subtract from each row of $\bar{A}$ its mean under A, then multiply by A, and everything above the diagonal is lost. Every row of $\bar{S}$ sums to zero. Point at a row.

That completes the list: embedding, affine maps, the rectifier, LayerNorm, attention, the loss, joined by sums at every fan-out. The backward pass of the whole model is these pullbacks, applied in the reverse of the order of §6.4 (p. 76).

7.5 The gradient check

Every formula above was derived by hand, and a derivation by hand can be wrong: a missing transpose, a sum where a mean belongs, a forgotten use of a shared value. The program will run anyway and train worse. So we compare with an experiment, the central difference of the opening section.

How good is the experiment? Taylor's theorem gives

$$\frac{\ell(\theta + he_i) - \ell(\theta - he_i)}{2h} = \frac{\partial \ell}{\partial \theta_i} + \frac{h^2}{6} \frac{\partial^3 \ell}{\partial \theta_i^3} + O(h^4) :$$

the symmetric difference cancels the even terms, and the error falls like h^2. But the two losses are computed in floating point, each with a relative error of order the unit roundoff $u = 2^{-53} \approx 10^{-16}$. Their difference, which is small, inherits an absolute error of order $u\,\ell$, and dividing by $2h$ turns it into an error of order $u\ell/h$. The total,

$$O(h^2) + O(u\ell/h),$$

is smallest when the two terms balance, at h of order $u^{1/3}$, about 10^{-5} for quantities of order one. A smaller step is not a more accurate experiment: it subtracts two numbers that agree in more of their digits, and keeps only the digits that are noise.

Figure 7.4 runs the experiment on the specimen before training. On the left, one parameter from each of four arrays, against the step: the error follows the h^2 line down, turns where it meets the rounding line, and climbs back up it. On the right, every one of the parameters at once.

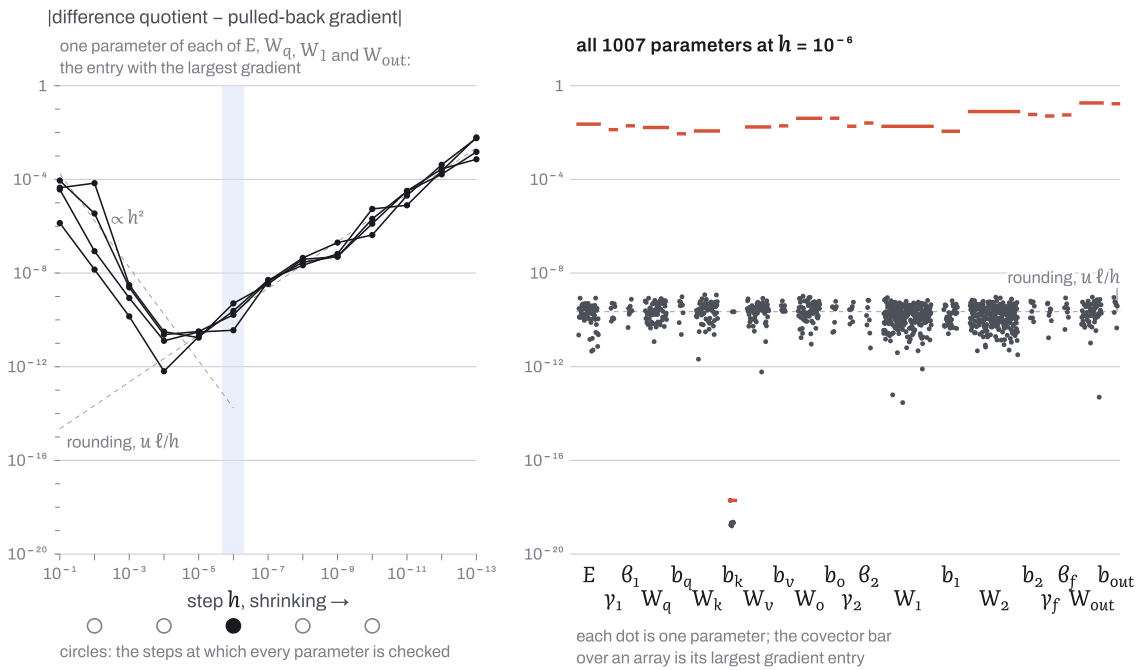


Figure 7.4 Checking the derivation against an experiment, before training. Left: the error of a central difference against the pulled-back gradient for four parameters, as the step h shrinks: it falls like h^2 until rounding takes over, and then it rises like $u\ell/h$. Right: the error for every parameter at one step, grouped by array, with the size of each array's largest gradient entry marked by a bar in covector ink. Click a circle on the left to check every parameter at another step.

The check also comes in a sharper form, which tests one pullback at a time. For any map f and any covector $\bar{y}$, the gradient of the scalar function $x \mapsto \langle \bar{y}, f(x) \rangle$ is $Df(x)^\top \bar{y}$, which is exactly what the pullback claims to return. So draw a random $\bar{y}$, run `back`, and compare with central differences of that scalar function. This is the *adjoint test*. The test suite applies it to every operation of the model, and the gradient check to the model as a whole.

What does a passing check prove? That on these inputs, at these points, the formulas and the code agree to the precision the experiment allows: wrong transposes, missing sums and misplaced factors show up as errors of order one, many decades above the cloud. It does not prove the formulas in general, which is what the derivations are for; and at a kink of the rectifier it proves nothing, because there is no derivative there to compare with.

7.6 The parameter that never learns

An exercise of Chapter 5 noticed that the key bias b_k shifts every score in a row by the same amount. Now we can say exactly what that does to learning. Write the keys as $K = \widetilde{K} + b_k \mathbf{1}^\top$. Then

$$S = \frac{Q^\top \widetilde{K}}{\sqrt{d}} + \frac{(Q^\top b_k)\,\mathbf{1}^\top}{\sqrt{d}},$$

and the second term is constant along every row. The softmax of a row does not change when a constant is added to it, so A, the outputs and the loss do not depend on b_k at all. The backward pass agrees. By the affine rule, $\bar{b}_k = \bar{K}\mathbf{1} = Q\bar{S}\mathbf{1}/\sqrt{d}$, and $\bar{S}\mathbf{1} = 0$ because every row of $\bar{S}$ sums to zero: $\sum_j A_{ij}\,(\,\bar{A}_{ij} - c_i\,) = c_i - c_i = 0$ with $c_i = \sum_k A_{ik}\bar{A}_{ik}$. So

$$\bar{b}_k = 0,$$

identically, for every input and every value of the other parameters. Of the 1007 numbers of the specimen, these eight can never learn. They are kept on purpose: they belong to the standard architecture, which this book follows, and they make a good puzzle. Many implementations drop them.

In floating point, "identically zero" becomes "zero up to rounding". The computed rows of $\bar{S}$ sum to numbers like 10^{-19} rather than to zero (point at a row of Figure 7.3 (p. 92)), and the computed $\bar{b}_k$ is of that size, at the very bottom of the scale of Figure

Rounding-level gradients still move parameters, if the optimiser divides by the size of the gradient. Adam does (Chapter 8): at the floor of the loss its small constant ϵ is all that stands between a gradient of order 10^{-17} and a step of order the learning rate. In the specimen's training run the key bias drifts away from zero by about 10^{-11}, and the eight paths of b_k in the cover's trace of every parameter are the ones that never visibly move.

7.4 (p. 93), far below every other array's gradient. For the difference quotient the loss does not change at all, except in its last bit, so the check of b_k measures nothing but rounding.

The output bias has a weaker version of the same symmetry. Adding $c\mathbf{1}$ to b_{out} shifts every logit of a column equally and leaves every probability alone, so $\mathbf{1}^\top \bar{b}_{\text{out}} = \sum_n \mathbf{1}^\top \bar{Z}_{:,n} = 0$: the gradient never changes the mean of b_{out}, only the differences between its entries. Unlike b_k, it still learns those.

7.7 In the code

The gradient check is a dozen lines; it differences one coordinate at a time:

src/gradcheck.jl gradcheck · lines 14–26

```
function gradcheck(f, x, x̄; h = 1e-6)
    x = copy(x)
    g = zeros(size(x))
    for i in eachindex(x)
        xᵢ = x[i]
        x[i] = xᵢ + h;  f₊ = f(x)
        x[i] = xᵢ - h;  f₋ = f(x)
        x[i] = xᵢ
        g[i] = (f₊ - f₋) / 2h
    end
    scale = max(norm(g), norm(x̄))
    iszero(scale) ? 0.0 : norm(g - x̄) / scale
end
```

The adjoint test builds the scalar function $x \mapsto \langle \bar{y}, f(x) \rangle$ for a random $\bar{y}$ and hands it to `gradcheck`:

src/gradcheck.jl checkpullback · lines 36–45

```
function checkpullback(op, args...; wrt)
    y, back = op(args...)
    ȳ = y isa Number ? randn() : randn(size(y))
    x̄s = back(ȳ)
    x̄s isa Tuple || (x̄s = (x̄s,))
    map(enumerate(wrt)) do (k, i)
        f(xᵢ) = dot(ȳ, first(op(ntuple(j -> j == i ? xᵢ : args[j], length(args))...)))
        gradcheck(f, args[i], x̄s[k])
    end
end
```

Exercises

EXERCISE 7.1 Derive the pullbacks of the elementwise product $y = a \odot b$ and of the transpose $Y = X^\top$. Then check the attention formulas $\bar{V} = \bar{O}A$ and $\bar{A} = \bar{O}^\top V$ by writing $O = VA^\top$ in index form.

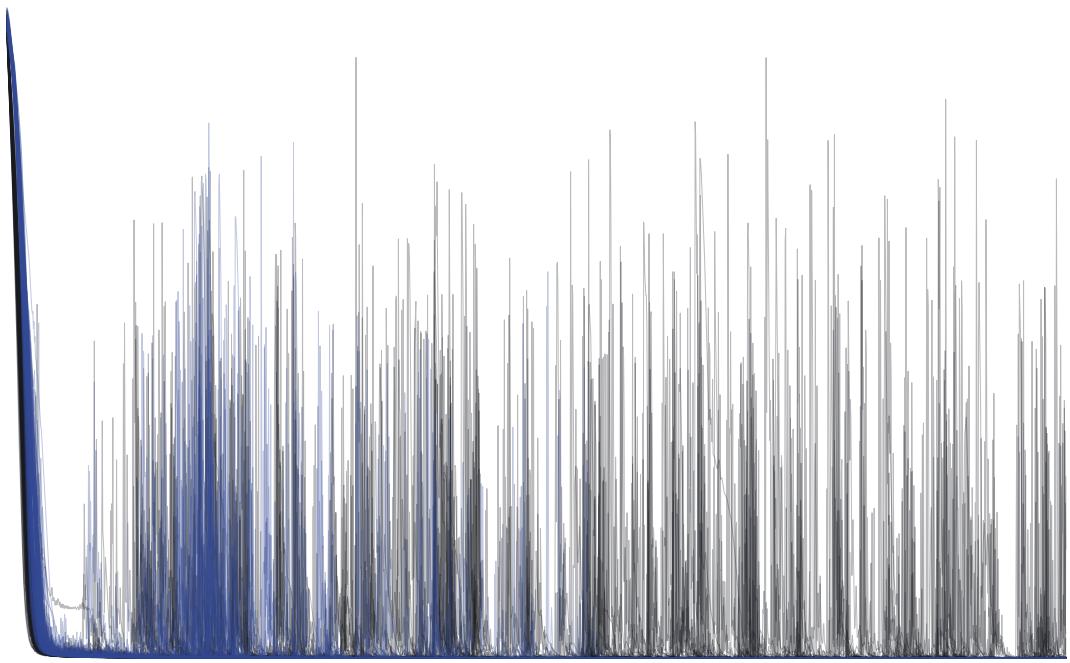
EXERCISE 7.2 A chain of k maps between spaces of dimension $n_0, n_1, \ldots, n_k$ has Jacobians $J_i \in \mathbb{R}^{n_i \times n_{i-1}}$. Count the multiplications needed to form the full Jacobian $J_k \cdots J_1$ bracketed from the right, and bracketed from the left. Which is cheaper when $n_k = 1$, and which when $n_0 = 1$? What are n_0 and n_k for the specimen's loss?

EXERCISE 7.3 For the LayerNorm pullback, show that $\mathbf{1}^\top \bar{x} = 0$, and that $\hat{x}^\top \bar{x} = 0$ when $\varepsilon = 0$. Compute $\hat{x}^\top \bar{x}$ for $\varepsilon > 0$. Which perturbations of the input column cannot change the output, and why can their adjoints not appear?

EXERCISE 7.4 Show that the softmax Jacobian $\mathrm{diag}(p) - pp^\top$ is positive semidefinite and that, for finite logits, its null space is spanned by $\mathbf{1}$. Deduce that the entries of $\bar{b}_{\text{out}}$ always sum to zero. Then show that the value bias b_v reaches the output of attention only as the constant $W_o b_v$ added to every column (the rows of A sum to one), so that it duplicates part of b_o. Is its gradient zero?

EXERCISE 7.5 In Figure 7.4 (p. 93), click the step $h = 10^{-2}$, then $h = 10^{-10}$. Which arrays have the largest errors in each case, and why? At every step most dots of b_k sit at the bottom of the scale, and a few sit exactly on the rounding line. Explain both: why is the difference quotient of b_k usually exactly zero, and why, when it is not, is it of size $u\ell/h$?

EXERCISE 7.6 In Figure 7.3 (p. 92), point at the first row, then at the last. Why is the first row of $\bar{S}$ empty? Which row of $\bar{A}$ loses the most to the mask, and why does losing it cost the model nothing?



Chapter 8. *The specimen trained from 120 seeds: plain Adam keeps leaving the floor; a cosine schedule lets it stay.* Loss (logarithmic, 0.05 to 2.2 nats) against step (linear, 0 to 2000). Ink: Adam (β_2 0.95, no weight decay, no clipping) at constant $\eta = 0.02$. Value ink: the same with 20 warmup steps and cosine decay to 0.002. Seeds 1–120. After step 300 the constant-rate runs show 977 sudden jumps of more than 50% between consecutive steps.

CHAPTER EIGHT

Learning

8

Learning is a walk downhill in $\mathbb{R}^p$: gradient descent is Euler's method for the gradient flow $\dot{\theta} = -\nabla\ell(\theta)$, and everything else in a modern training loop (minibatches, Adam, schedules, clipping, initial variances) is a decision about how long, and how well aimed, each step may be.

MATHS	PHYSICS	ML
$\dot{\theta} = -\nabla\ell(\theta)$	*overdamped motion in a potential*	"gradient flow"
$\theta_{k+1} = \theta_k - \eta\nabla\ell(\theta_k)$	*an explicit Euler step*	"gradient descent, an update, a training step"
η	*the time step*	"learning rate, step size"
$\lambda_{\max}$ of the Hessian	*the stiffest mode, the curvature scale*	"sharpness"
$\hat{g}$ with $\mathbb{E}\hat{g} = \nabla\ell$	*a noisy measurement of the force*	"minibatch gradient, stochastic gradient"
$\theta \leftarrow \theta - \eta\, G\hat{m}$, G diagonal	*a coordinate-dependent mobility*	"Adam, preconditioning"
$\theta \leftarrow (1-\eta\lambda)\theta - \cdots$	*friction towards the origin*	"weight decay, AdamW"
projection onto $\{\|g\| \le c\}$	*a bound on the force*	"gradient clipping"

8.1 Watching it learn

FIX THE TRAINING SET: THE TEN WINDOWS OF THE CORPUS, eighty predictions in all. The loss of Chapter 4 is then one function on the space of parameters,

$$\ell : \mathbb{R}^p \to \mathbb{R}, \qquad \theta \longmapsto \frac{1}{N} \sum_{n=1}^{N} -\log p_\theta(y_n \mid \text{context}_n),$$

with $p = 1007$ and $N = 80$, and Chapter 7 computes its gradient $\nabla\ell(\theta) \in \mathbb{R}^p$ at the cost of about one forward pass. Training means finding a θ where ℓ is small. Two numbers frame the answer. A model that has learned nothing and spreads its probability evenly over the V characters pays $\ln V$ nats per prediction: that is *uniform guessing*. And no model can pay less than the *entropy floor* of §4.5 (p. 51), because five of the eighty predictions are made from a context of a single character that the windows continue in different ways. Two windows begin with "o" and three with a space, so the floor is $(2\ln 2 + 3\ln 3)/80$ nats, and not zero.

Figure 8.1 runs the whole of this chapter in front of you. Every step is a forward pass on the ten windows, a backward pass, and one update of all the parameters. It starts at θ_0, the specimen's initial parameters, and with the settings it opens with it repeats the training run whose result the rest of the book has been looking at, number for number.

The cover of this book is the same story told with every parameter: each of its 1007 hairlines is one coordinate θ_i followed through a training run, with time running down the page on a logarithmic scale. Its run starts from a different θ_0 (it was drawn by the JavaScript engine's own initialisation), so its numbers are not the ones of this chapter; its shape is.

Three things happen, in this order. For the first few steps almost nothing does: the loss sits on the line of uniform guessing and the model writes noise. Then the loss falls steeply, and the samples pass through a phase of spaces and frequent letters, then of the right words in the wrong places, before they settle on "not to be". Finally the curve flattens onto the floor and stays there. The step axis is logarithmic, so the late, slow part of the run takes most of the width of the picture but little of its time.

8.2 Gradient flow and its Euler steps

The simplest way to make a function smaller is to move against its gradient. As a differential equation in a fictitious time s, this is the *gradient flow*

$$\frac{d\theta}{ds} = -\nabla\ell(\theta).$$

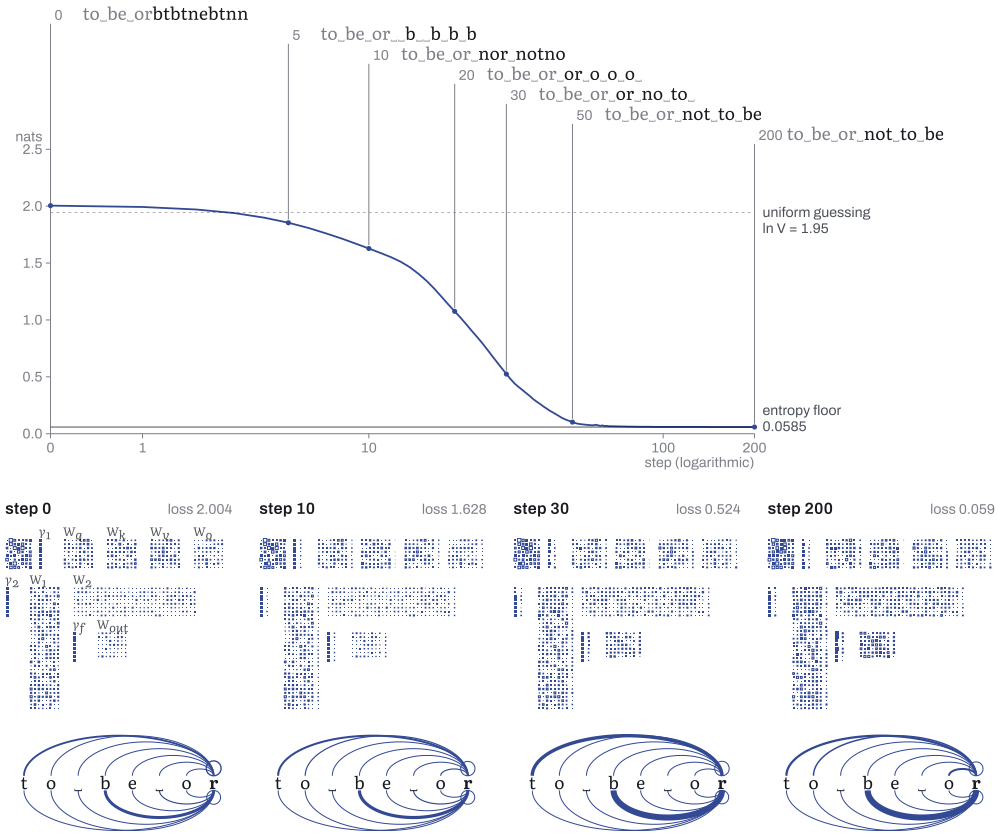
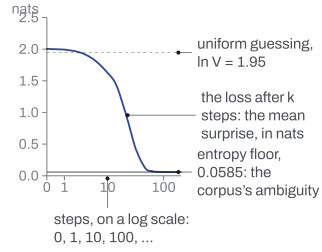


Figure 8.1 The specimen learning, live. Press ▶: every step is a forward pass, a backward pass and an update of all the parameters. The curve is the loss, with the line of uniform guessing at $\ln V$ and the entropy floor of the corpus; above it, what the model writes after "to be or" at a few steps along the way. Under the curve is the learning rate of every step. On the right, every parameter as one square, and what the last character of the window reads. Drag the learning rate on its axis, change the optimiser or the schedule, or start again from another seed.

HOW TO READ



The faint curve is the run recorded in `artifacts/specimen.json` by the Julia code. The browser's run is a separate computation, by the JavaScript engine, from the same θ_0 with the same settings; the two curves agree to the last few digits of a double.

Along a solution the loss can only decrease, since by the chain rule

$$\frac{d}{ds}\,\ell(\theta(s)) = \langle \nabla\ell,\, \dot{\theta}\rangle = -\|\nabla\ell(\theta(s))\|^2 \le 0,$$

with equality only where the gradient vanishes. A physicist reads this as overdamped motion: a particle in the potential ℓ, in a fluid so viscous that its velocity is proportional to the force on it, slides downhill and dissipates the potential energy it loses.

A computer cannot follow the flow; it can take steps. Replacing the velocity during one step of length η by its value at the start of the step is Euler's method, and Euler's method for the gradient flow is *gradient descent*:

$$\theta_{k+1} = \theta_k - \eta\,\nabla\ell(\theta_k).$$

The step size η is called the *learning rate*. It is the time step of the integrator, and like every time step it has to be small enough. How small?

In the literature Λ is usually called L and the loss "L-smooth". In this book L already counts the blocks.

The answer depends on how fast the gradient itself changes. Suppose that on the segment between θ and $\theta + s$ the gradient is Λ-Lipschitz, $\|\nabla\ell(\theta') - \nabla\ell(\theta'')\| \le \Lambda\|\theta' - \theta''\|$. Then

$$\ell(\theta + s) \le \ell(\theta) + \langle \nabla\ell(\theta),\, s\rangle + \frac{\Lambda}{2}\|s\|^2,$$

by the fundamental theorem of calculus applied to $t \mapsto \ell(\theta + ts)$. Put $s = -\eta g$ with $g = \nabla\ell(\theta)$:

$$\ell(\theta - \eta g) \;\le\; \ell(\theta) - \eta\Big(1 - \frac{\Lambda\eta}{2}\Big)\,\|g\|^2.$$

This is the *descent lemma*. The first-order term promises a decrease proportional to η; the curvature can take back an amount proportional to η^2. If $\eta < 2/\Lambda$ the promise wins and the step lowers the loss, and $\eta = 1/\Lambda$ guarantees a decrease of at least $\|g\|^2/(2\Lambda)$. No convexity is needed.

The number $2/\Lambda$ is no artefact of the proof. On a quadratic $\ell(\theta) = \tfrac{1}{2}\theta^{\top}H\theta$, gradient descent is the linear map $\theta \mapsto (I - \eta H)\theta$, which contracts the eigendirection of eigenvalue λ by the factor $1 - \eta\lambda$. It shrinks every direction if and only if $|1 - \eta\lambda| < 1$ for all of them, that is, if $\eta < 2/\lambda_{\max}$. Beyond that the stiffest mode overshoots further at every step, the familiar instability of an explicit integrator applied to a stiff equation. The largest eigenvalue of the Hessian, which plays the role of Λ near a point, is called the *sharpness* of the loss there.

For the specimen there is no single Λ. The rectifier of the MLP has kinks, where the gradient jumps, and the curvature changes from place to place. The lemma is a local statement, and Figure 8.2 shows how local. At three learning rates it runs plain gradient descent on the specimen and measures the sharpness at the current parameters as it goes.

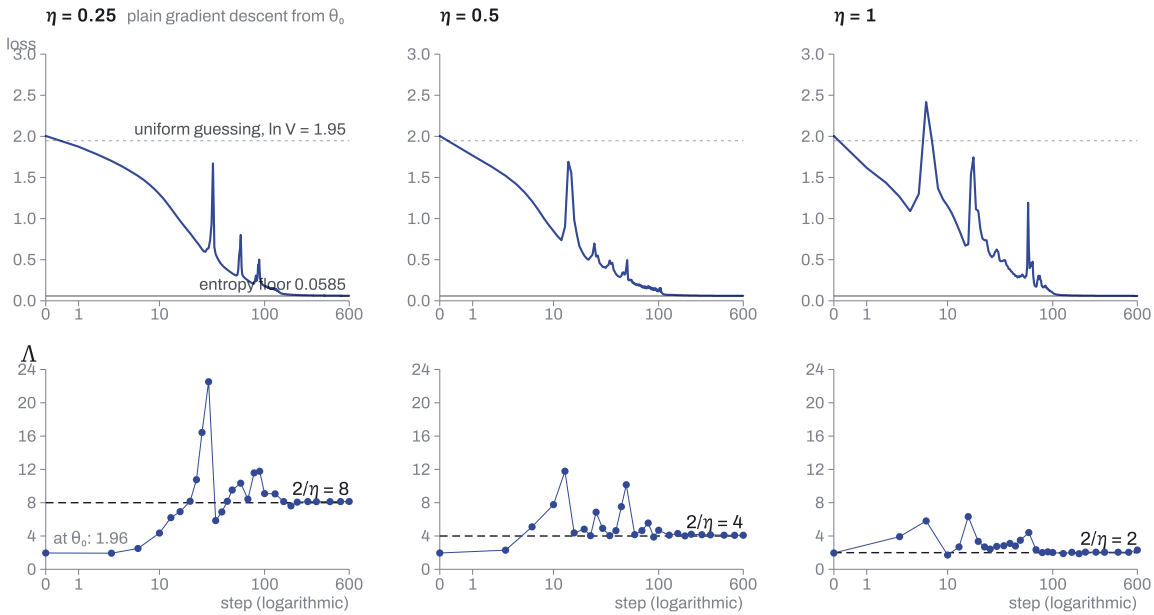


Figure 8.2 How large a step can gradient descent take? Plain gradient descent on the specimen at three learning rates. Below each loss curve, the sharpness Λ, the largest eigenvalue of the Hessian of the loss at the current parameters, and the line $2/\eta$: the descent lemma guarantees a decrease only while the sharpness stays below it. It starts below, climbs, and then settles on the line.

At the start the sharpness is well below $2/\eta$, and the loss falls as the lemma promises. But training does not leave the curvature alone: the descent drives θ into steeper and steeper regions, the sharpness grows until it reaches $2/\eta$, and there it stops. Gradient descent then walks along the edge of the region where its steps are stable, overshooting in the stiffest direction just enough to keep from going further. The loss keeps falling, though no longer at every step. The lemma explains where the edge is, and it does not explain why the run should stay on it: a sufficient condition for descent is not a law of motion.

This is the *edge of stability*, found by Cohen, Kaur, Li, Kolter and Talwalkar (2021) in networks far larger than the specimen. The sharpness is measured here as in Chapter 7: a Hessian–vector product is a central difference of two pulled-back gradients, and power iteration on it finds the largest eigenvalue without ever forming the Hessian.

8.3 Minibatches

The specimen computes its gradient on all ten windows at every step. A real training set does not allow that: the Shakespeare model of Chapter 9 has a million windows to choose from, and each step looks at a few dozen. Write the loss as a mean over M examples, $\ell = \frac{1}{M}\sum_{i=1}^{M} \ell_i$, choose b of them at random, $I_1, \ldots, I_b$, and use

$$\hat{g} = \frac{1}{b}\sum_{j=1}^{b} \nabla \ell_{I_j}$$

in place of $\nabla\ell$. If each I_j is uniform on $\{1, \ldots, M\}$ then $\mathbb{E}\nabla\ell_{I_j} = \frac{1}{M}\sum_i \nabla\ell_i = \nabla\ell$, by linearity alone, and so

$$\mathbb{E}\,\hat{g} = \nabla\ell.$$

The *minibatch gradient* $\hat{g}$ is an unbiased estimate of the gradient. If the I_j are drawn independently (with replacement), its covariance is Σ/b, where Σ is the covariance of a single example's gradient: the noise shrinks like $1/\sqrt{b}$. Drawing without replacement, as in practice, keeps the mean and multiplies the covariance by $(M-b)/(M-1)$, which is zero when the batch is the whole set. Gradient descent with $\hat{g}$ is *stochastic gradient descent*, Euler's method with a force measured afresh, and noisily, at every step.

Unbiasedness survives only linear operations. Clipping, and Adam's division by the size of the gradient, are applied to $\hat{g}$ and not to its mean, and they make the step a biased estimate of anything one might want. Nothing in practice seems to suffer much from this; nothing in theory says it shouldn't.

For the specimen nothing need be sampled: there are exactly $\binom{10}{b}$ minibatches of size b, and Figure 8.3 computes the gradient of every one. Before training the clouds are centred on a large full gradient, and they tighten as b grows. After training the full gradient is almost zero, as it should be at the bottom of the loss, but the clouds are not. The windows that begin with "o" or with a space still pull hard: at the floor the model gives probability one half to each continuation of "o", so the window whose next character is a space pushes towards the space, the window whose next character is "r" pushes towards "r", and only their sum vanishes. Minibatch noise does not die out at a minimum of the loss. It dies out only where every single example is satisfied, and the floor exists precisely because that cannot happen.

8.4 Adam

The parameters of a network are not alike. In the specimen the embedding, the attention maps, the MLP and the readout receive

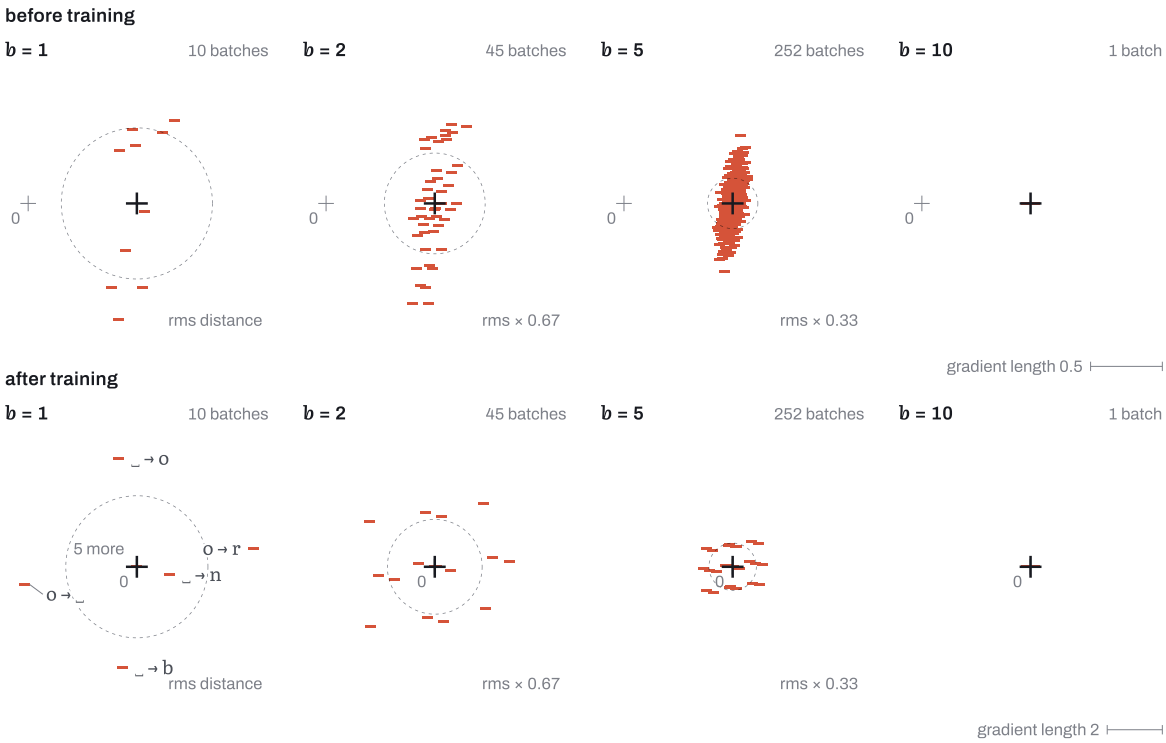
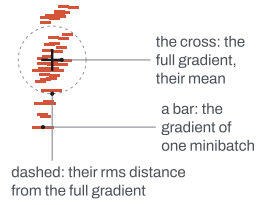


Figure 8.3 Every minibatch gradient of the specimen, projected onto a plane. Each dot is the gradient of the loss on b of the ten windows, for every possible choice of the b; the cross is the full gradient, which is exactly the mean of each cloud. Before training, and after it.

HOW TO READ



gradients of very different sizes, and so do the coordinates within one array (Figure 8.4, p. 107). A single learning rate is then too large for some coordinates and too small for others: gradient descent moves each θ_i in proportion to $\partial\ell/\partial\theta_i$, and a coordinate with a tiny gradient barely moves at all.

The remedy is a *preconditioner*: a positive definite G, and the step $\theta \leftarrow \theta - \eta G g$. Such a step is still steepest descent, measured in a different metric: $-Gg$ minimises $\langle g,\ s\rangle + \frac{1}{2}s^{\mathsf{T}}G^{-1}s$, the first-order change of the loss plus a penalty on the length of s in the norm of G^{-1}. Newton's method takes G to be the inverse Hessian. That costs p^2 numbers to store, and finding it costs more; the

practical choice is diagonal, one scale per coordinate, estimated from the gradients as they arrive.

Adam keeps two running averages per coordinate, of the gradient and of its square, with $m_0 = v_0 = 0$:

$$m_k = \beta_1 m_{k-1} + (1-\beta_1) g_k, \qquad v_k = \beta_2 v_{k-1} + (1-\beta_2)\, g_k \odot g_k.$$

Unrolled, $m_k = (1-\beta_1)\sum_{j \le k} \beta_1^{k-j} g_j$ is an average with geometric weights, whose weights add up to $1-\beta_1^k$ rather than to one because the sum started from zero. Dividing by that sum gives the bias-corrected averages $\hat{m}_k = m_k/(1-\beta_1^k)$ and $\hat{v}_k = v_k/(1-\beta_2^k)$, and the update is

$$\theta_{k+1} = \theta_k - \eta_k \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}, \qquad \text{that is} \qquad \theta_{k+1} = \theta_k - \eta_k\, G_k \hat{m}_k, \quad G_k = \operatorname{diag}\Big(\frac{}{\sqrt{\hat{v}}}$$

with square root and division taken coordinate by coordinate. So Adam is a diagonal preconditioner applied to a smoothed gradient, the *momentum* $\hat{m}$, and not to the gradient itself. Note that v estimates the second moment of the gradient, not its variance: nothing is subtracted.

What does the division do? If the gradient of a coordinate were the same at every step, g, then $\hat{m}_k = g$ and $\hat{v}_k = g^2$ exactly, for every k, thanks to the bias correction, and the step would be

$$-\eta \frac{g}{|g| + \epsilon} \approx -\eta \operatorname{sign}(g).$$

In this limit Adam is *sign descent*: every coordinate moves by the same amount η, in the direction that lowers the loss, whatever the size of its gradient. The magnitude of the gradient is thrown away and only its direction kept. The very first step is always of this kind, since $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$. Later the averages remember older gradients, and a coordinate whose gradient keeps changing sign gets $|\hat{m}| \ll \sqrt{\hat{v}}$ and moves less. Figure 8.4 compares, for every parameter, the step that gradient descent would take with the one Adam takes.

A consequence worth remembering: Adam's step does not change if the loss is multiplied by a constant, while the step of gradient descent scales with it. For gradient descent η has the units of θ^2 per unit of loss; for Adam it has the units of θ, and it is roughly the distance each coordinate moves per step. That is why the two need such different learning rates, which you can confirm in Figure 8.1 (p. 101) by switching between them.

Kingma and Ba, *Adam: a method for stochastic optimization* (2015). The name is short for adaptive moment estimation. The specimen uses $\beta_1 = 0.9$, $\beta_2 = 0.95$ and $\epsilon = 10^{-8}$; the ϵ is Adam's own and has nothing to do with the ε of LayerNorm.

The division is also where rounding enters. The key bias b_k has a gradient that is exactly zero in exact arithmetic (§7.6), and in floating point a few units of rounding error. Gradient descent multiplies that by η and nothing happens. Adam divides it by its own square root plus ϵ; for a gradient far below ϵ this is a division by about ϵ, which turns rounding into steady motion. The readout of Figure 8.1 shows how far b_k has strayed from zero, a number that should not exist.

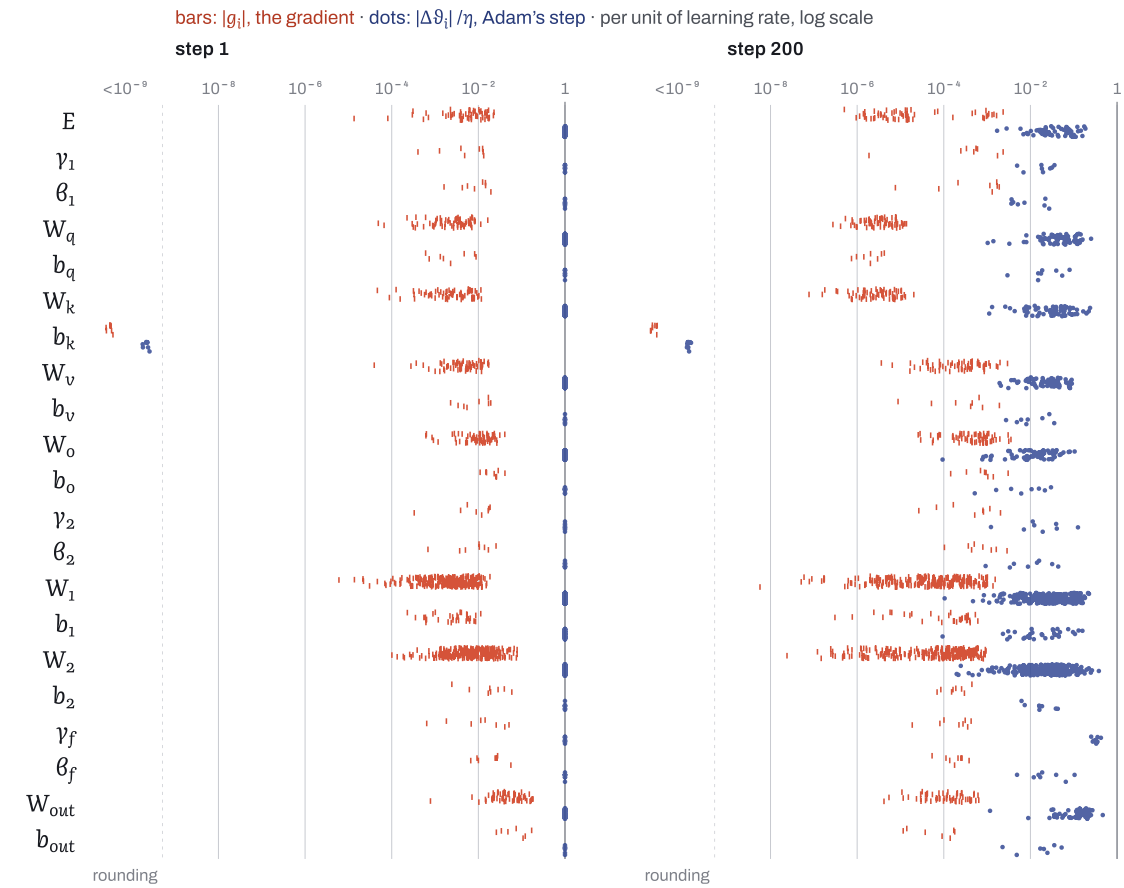
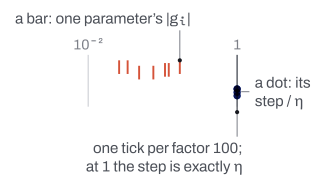


Figure 8.4 What Adam changes. For each parameter, grouped by array: the size of its gradient (bars), which is how far gradient descent would move it per unit of learning rate, and how far AdamW actually moves it, in the same unit (dots). At the first update every dot sits at one, except those of the key bias, whose gradient is zero up to rounding. Scrub through the training run.

HOW TO READ



Weight decay. It is common to pull the parameters gently towards zero, so that they grow no larger than the data demand. One way is to add $\frac{\lambda}{2}||\theta||^2$ to the loss, which adds $\lambda\theta$ to the gradient; for plain gradient descent the step becomes $\theta \leftarrow (1-\eta\lambda)\theta - \eta g$, a multiplicative shrinking of every coordinate. With Adam the two ideas come apart. Added to the gradient, $\lambda\theta$ passes through the preconditioner and is rescaled coordinate by coordinate, so coordinates with large gradients hardly

decay at all. *AdamW* keeps the decay out of the moments and applies it directly:

$$\theta_{k+1} = (1 - \eta_k \lambda)\, \theta_k - \eta_k\, G_k \hat{m}_k.$$

The specimen decays only the entries of its matrices. The gains γ, the offsets β and the biases set scales and zero points, and there is no reason for them to be small.

src/train.jl update! · lines 37, 71–79

```
update!(::Descent, θ, θ̄, η) = (θ .-= eltype(θ)(η) .* θ̄)
⋮
function update!(o::AdamW, θ, θ̄, η)
    (; m, v, β1, β2, ε, λ) = o
    η = eltype(θ)(η)
    t = o.t += 1
    c1, c2 = 1 - β1^t, 1 - β2^t                    # bias corrections
    @. m = β1 * m + (1 - β1) * θ̄
    @. v = β2 * v + (1 - β2) * θ̄^2
    @. θ -= η * ((m / c1) / (√(v / c2) + ε) + λ * θ)
end
```

Loshchilov and Hutter, *Decoupled weight decay regularization* (2019). Physically, the decay is a friction towards the origin, $\dot{\theta} = -\lambda\theta$, integrated by one Euler step and added to the step of the loss; exact integration over a time η would multiply by $e^{-\eta\lambda}$ instead.

src/train.jl decay · lines 87–93

```
function decay(c::Config, λ₀)
    λ = zeros(typeof(float(λ₀)), nparams(c))
    for x in leaves(carve(λ, c))
        ndims(x) == 2 && fill!(x, λ₀)
    end
    λ
end
```

8.5 Schedules and clipping

Nothing forces the time step to be constant, and in practice it is not. The specimen's run uses a *warmup* followed by a *cosine*: η_k rises linearly from zero to η over the first W updates, then falls along half a period of a cosine to $\eta_{\min}$ at the last update K,

$$\eta_k = \begin{cases} \eta\, k/W, & k \le W, \\ \eta_{\min} + \tfrac{1}{2}(\eta - \eta_{\min})\left(1 + \cos \pi \dfrac{k - W}{K - W}\right), & W < k \le K. \end{cases}$$

Each half has its reason. At the start Adam's averages rest on a handful of gradients, and the first steps, which are sign steps of full length η in every coordinate, are taken on the least information; the warmup keeps them short. At the end a large step can only rattle around the minimum: near the bottom of a bowl,

gradient descent with step η jumps across the bowl by an amount that shrinks with η, and stochastic gradients add noise of a size that does too. Large steps early and small ones late is the usual compromise between getting somewhere and settling there.

src/train.jl warmup_cosine · lines 103–112

```
function warmup_cosine(t; η, warmup, steps, ηmin = η / 10)
    0 ≤ warmup < steps || throw(ArgumentError("the schedule needs 0 ≤ warmup < steps"))
    0 ≤ t ≤ steps || throw(ArgumentError("the schedule step must lie in 0:steps"))
    η = float(η)
    ηmin = oftype(η, ηmin)
    t == 0 && return warmup == 0 ? η : zero(η)
    t ≤ warmup && return η * t / warmup
    u = oftype(η, t - warmup) / (steps - warmup)
    ηmin + (η - ηmin) * (1 + cos(π * u)) / 2
end
```

The second safeguard acts on the gradient. In a long run an unlucky minibatch, or a region of high curvature, occasionally produces a gradient far longer than usual, and one such step can undo a great deal of progress. *Clipping* caps its length. Given a radius c,

$$\operatorname{clip}_c(g) = \begin{cases} g, & \|g\| \le c, \\ c\,g/\|g\|, & \|g\| > c. \end{cases}$$

This is the Euclidean projection of g onto the ball of radius c: of all vectors of length at most c, $\operatorname{clip}_c(g)$ is the closest to g. For $\|g\| > c$ and any v in the ball, $\|g - v\| \ge \|g\| - \|v\| \ge \|g\| - c$, and $cg/\|g\|$ attains the bound. So clipping shortens a gradient and never turns it. The norm is that of the whole of θ's gradient at once, all arrays together, so the relative sizes of the parameters' gradients are preserved; and it is applied before Adam's averages see the gradient.

src/train.jl clip! · lines 23–29

```
function clip!(θ̄, r)
    r ≥ 0 || throw(ArgumentError("the clipping radius must be nonnegative"))
    r = eltype(θ̄)(r)
    n = norm(θ̄)
    n > r && (θ̄ .*= r / n)
    θ̄
end
```

Figure 8.5 (p. 110) shows both through the specimen's run: the learning rate as it was scheduled, and the length of each gradient

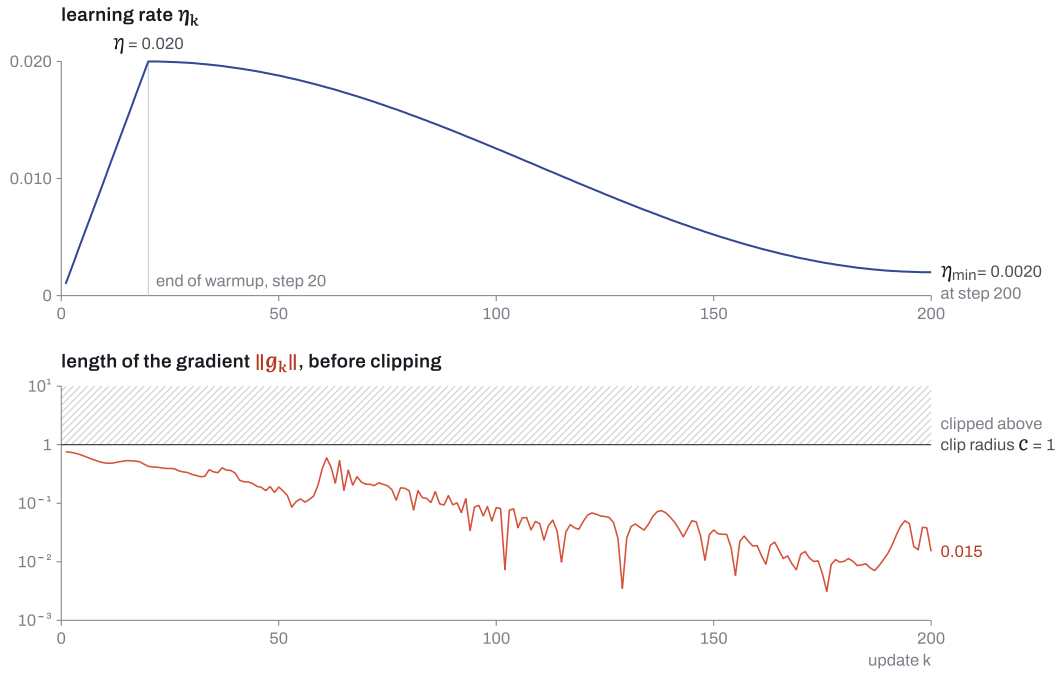


Figure 8.5 The learning rate and the length of the gradient through the specimen's training run. The rate rises linearly during the warmup and then falls along half a cosine. The gradient would be clipped whenever it is longer than the radius of the ball, which is hatched above the line; in this run it never is.

against the radius of the ball.

The clip never acts. The specimen's gradients start shorter than the radius and, with a few bumps, get shorter still, and the clip is in the recipe the way a fuse is in a circuit: for the run in which something goes wrong, which this one is not. The figure also shows the gradient's length falling by orders of magnitude as the loss approaches its floor, while, under Adam, the length of the step does not have to follow it. The next section is about what happens when it doesn't.

8.6 The slingshot

What does Adam do once the loss has reached its floor? Sign descent gives a hint. Near the bottom the gradient is small, but Adam divides it by its own recent size, and as long as the remaining gradient points consistently in some direction, the step stays

of order η per coordinate. The gradient flow slows to a halt at a minimum; Adam at a constant learning rate does not. It keeps moving, at nearly constant speed, through a region where the loss is almost flat, until it reaches a place where the curvature is too high for a step of that length. There the loss jumps, the gradient becomes large, $\hat{v}$ grows with it and shortens the steps, and the parameters fall back towards the floor, from a new place.

Figure 8.6 runs plain Adam from θ_0 for four thousand steps, with no weight decay and no clipping, twice: once at a constant learning rate, and once with the warmup and cosine of §8.5 (p. 108).

The name and the first careful description are from Thilak, Littwin, Zhai, Saremi, Paiss and Susskind, *The slingshot mechanism* (2022), who found these cycles late in training of transformers on small algorithmic tasks, and connected them to the sudden generalisation known as grokking. The specimen is not large enough to say anything about grokking.

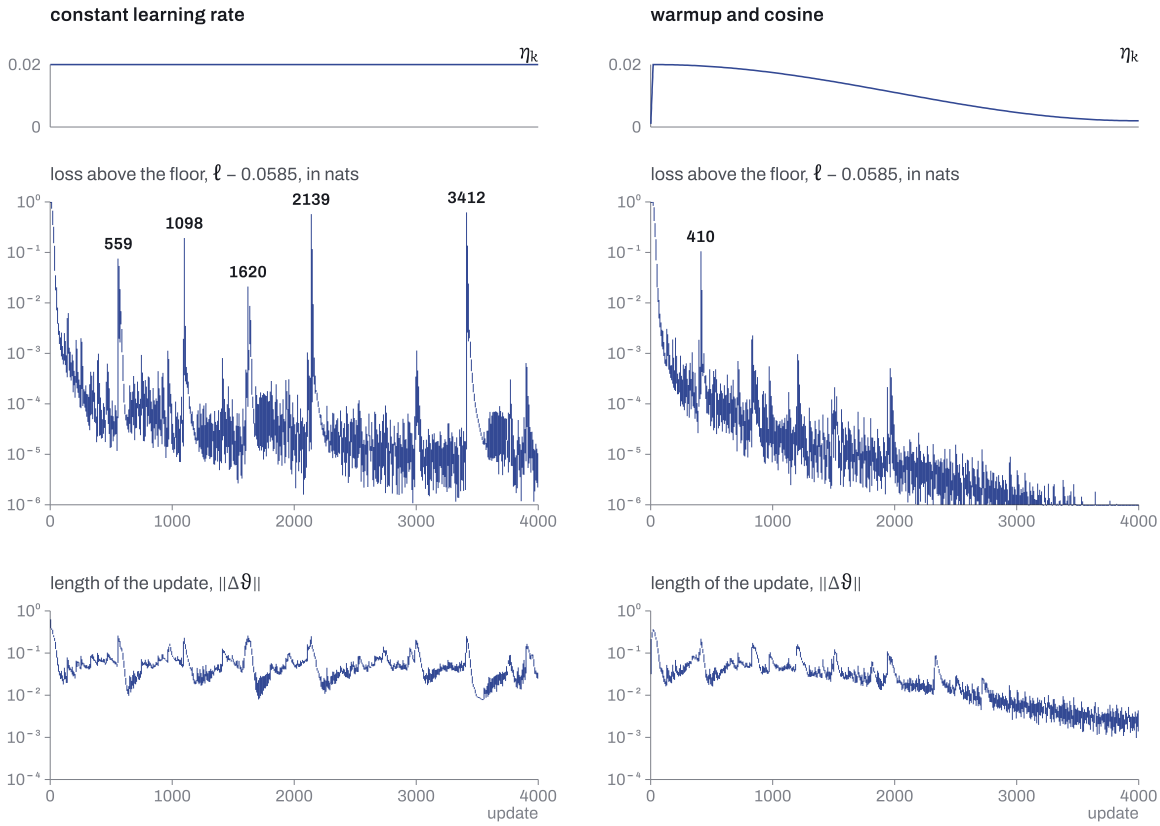


Figure 8.6 Adam at the floor. The excess of the loss over the entropy floor, on a logarithmic scale, and the length of each update, for four thousand steps of Adam: at a constant learning rate, and with the learning rate falling along a cosine. Each column of hairlines spans the smallest and the largest value over a few consecutive steps, so that no spike is lost. The labelled spikes are the steps at which the loss leaves the floor.

The frontispiece of this chapter records the same experiment from many starting points at once; its caption gives the settings.

At a constant rate the loss reaches the floor and then, again and again, leaves it: a spike of several orders of magnitude on this scale, followed by a slower return. The length of the update hardly reacts to the floor at all. Under the cosine the steps shrink with η, and the late spikes are gone; the one early spike comes while the rate is still near its peak. A falling schedule is not only a matter of settling more precisely: it is what stops the optimiser from throwing the parameters out of the minimum it has found.

8.7 Where to start

Every run starts somewhere, and gradient descent can only improve on its starting point locally. The initial parameters are random, and what matters is their scale. Consider one linear map $y = Wx$ with fan-in n, its entries drawn independently with mean zero and variance s^2, independent of x. Then $\mathbb{E}[y_i \mid x] = 0$ and, since the cross terms vanish,

$$\operatorname{Var}(y_i \mid x) = s^2 \|x\|^2 = n\, s^2 \cdot \frac{\|x\|^2}{n}.$$

The mean square of the output is ns^2 times that of the input. With $s^2 = 1/n$ the map preserves the size of what passes through it, in the sense of the root mean square of the entries; with a larger s^2 every layer amplifies, with a smaller one every layer damps.

The specimen applies this argument map by map. Every entry is drawn independently from a normal distribution, and only the standard deviations need choosing:

- **The maps that read a normalised column**, $W_q, W_k, W_v \in \mathbb{R}^{D \times D}$ and $W_1 \in \mathbb{R}^{F \times D}$, have fan-in D and get variance $1/D$, standard deviation $1/\sqrt{D}$. Their input comes out of a LayerNorm, which at the start has $\gamma = \mathbf{1}$ and $\beta = 0$, so its entries have mean square $\|\hat{x}\|^2/D$, just below one (§6.2, p. 71). The queries, keys, values and the MLP's preactivations therefore start with entries of mean square close to one, and the scores $q \cdot k/\sqrt{d}$ with variance of order one, the regime of §5.4 (p. 62).
- **The maps that write into the stream**, W_o with fan-in D and W_2 with fan-in F, get the fan-in variance divided by $2L$: $1/(2LD)$ and $1/(2LF)$. The stream receives $2L$ increments, one from each branch. If they are uncorrelated with the stream and with each other, their variances add like the steps of a

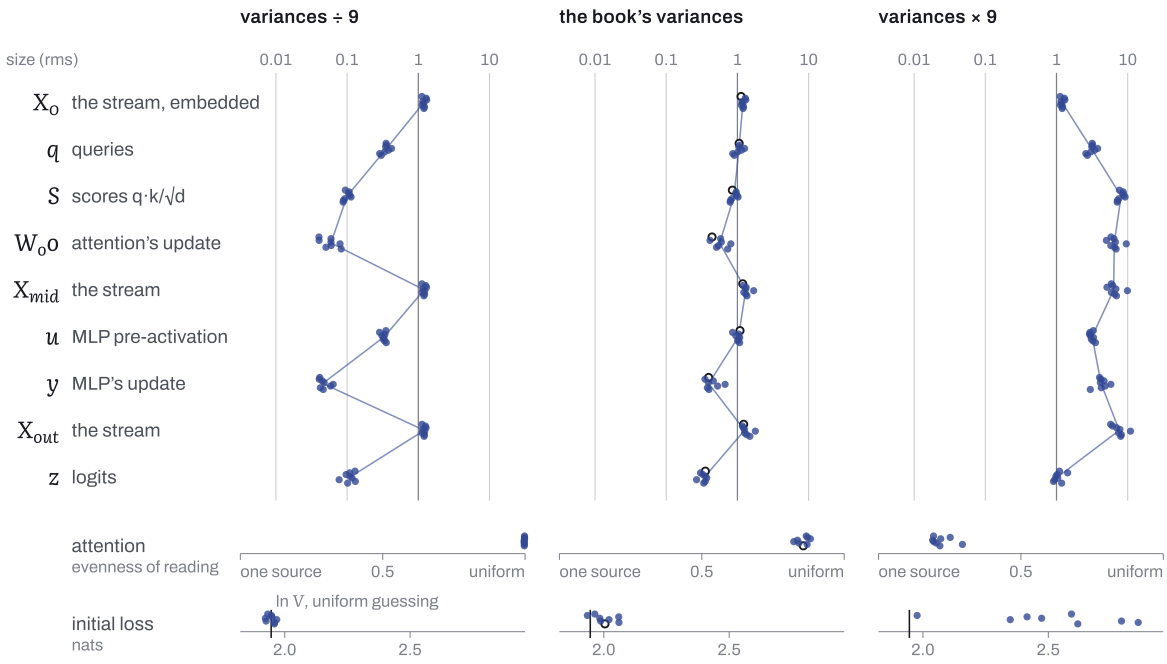
random walk: $X_{2L} = X_0 + \alpha \sum_k r_k$ with $\operatorname{Var} r_k = c$ has $\operatorname{Var} X_{2L} = \operatorname{Var} X_0 + 2L\alpha^2 c$, and $\alpha^2 = 1/(2L)$ keeps the total of the increments at c however deep the model. A final projection enters its branch linearly, so the factor is put into its variance. The rectifier halves the mean square of a symmetric input, so W_2's increment starts quieter still; the variance $2/F$ would compensate for that, and the specimen does not bother.

- **The embedding** E has variance one. The character's part of $x_0 = Ee_a + P_{:,t}$ then has mean square one per entry, against one half for the positional code, whose pairs contribute $\sin^2 + \cos^2 = 1$ for every two entries: neither drowns the other.
- **The readout** $W_{\text{out}} \in \mathbb{R}^{V \times D}$ has standard deviation $1/D$, variance $1/D^2$, far below the fan-in value $1/D$. It reads the final LayerNorm's output, whose squared length is close to D, so each logit starts with variance close to $D \cdot D^{-2} = 1/D$. Small, independent logits make a nearly uniform first guess: to second order in the logits the expected loss is $\ln V + (V-1)/(2VD)$ (Exercise 8.4, p. 116), just above uniform guessing. An untrained model should guess, not be confidently wrong, and the loss at step 0 of Figure 8.1 (p. 101) sits just above the line of uniform guessing for that reason.
- **Gains, offsets and biases** start at $\gamma = 1$, $\beta = 0$, $b = 0$: every LayerNorm begins as the bare projection to the sphere, and every affine map as a linear one.

src/parameters.jl initial · lines 108–117

```
function initial(name::Symbol, c::Config)
    (; D, F, L) = c
    name == :E                   ? (0.0, 1.0) :
    name in (:Wq, :Wk, :Wv, :W1) ? (0.0, 1 / √D) :
    name == :Wo                  ? (0.0, 1 / √(2L * D)) :
    name == :W2                  ? (0.0, 1 / √(2L * F)) :
    name == :Wout                ? (0.0, 1 / D) :
    name in (:γ1, :γ2, :γf)      ? (1.0, 0.0) :
                                   (0.0, 0.0)              # β1 β2 βf and the biases
end
```

Figure 8.7 (p. 114) checks the argument by breaking it. It draws the specimen's initial parameters with the book's variances, and with every weight variance except the embedding's multiplied by nine or divided by nine, and measures every intermediate of the first forward pass.



HOW TO READ

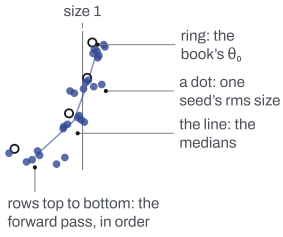


Figure 8.7 Why these initial variances. The size of every intermediate of the forward pass at initialisation, for the book's variances and for every weight variance multiplied by nine or divided by nine, one dot per seed. Below: how evenly the attention reads, and the initial loss against uniform guessing.

These are statements about the start of training, under the assumption that weights and inputs are independent. Nothing keeps them true afterwards, and the trained specimen has no reason to keep its activations of order one. The variance $1/n$ is usually credited to LeCun and to Glorot and Bengio (2010); the factor 2 that a ReLU calls for, because it zeroes half of a symmetric input, to He and colleagues (2015).

With the book's variances every station of the forward pass is of order one. Nine times larger, the scores spread out, most attention rows freeze onto a single source (Chapter 5), the branches overwhelm the stream they are meant to correct, and the first predictions are confident and wrong, with a loss above $\ln V$. Nine times smaller, the attention reads everything equally and the branches add almost nothing: the model starts as the embedding alone, a bigram model with a clock. With one block either error is a factor of three in the size of a few intermediates; in a deep model the factors compound from layer to layer.

8.8 In the code

One training step is three lines: the loss and its gradient from Chapter 7, the gradient clipped, and the parameters updated by the optimiser.

src/train.jl step! · lines 148–152

```
function step!(θ, opt, x, y, c::Config; η, clip = 1)
    ℓ, θ̄ = ∇loss(θ, x, y, c)
    update!(opt, θ, clip!(θ̄, clip), η)
    ℓ
end
```

Everything else in this chapter is a choice of `opt` and of `η`. The book's run (Figure 8.1, p. 101 with its first settings) is two hundred such steps with AdamW, decay on the matrices, the warmup and cosine of §8.5 (p. 108), and a clip that never acts.

Exercises

EXERCISE 8.1

Gradient descent on the quadratic $\ell(\theta) = \frac{1}{2}\theta^{\top} H \theta$, with H symmetric positive definite and eigenvalues $0 < \lambda_1 \le \cdots \le \lambda_p$. Show that $\theta_k = (I - \eta H)^k \theta_0$, and that $\theta_k \to 0$ from every start if and only if $0 < \eta < 2/\lambda_p$. Which learning rate makes the slowest direction converge fastest while keeping the fastest one stable, and what is then the contraction factor per step, in terms of $\kappa = \lambda_p/\lambda_1$?

EXERCISE 8.2

Show that the update of Adam (with $\epsilon = 0$) is unchanged when the loss is multiplied by a constant $c > 0$, and that the update of gradient descent is multiplied by c. If θ is measured in metres and ℓ in nats, what are the units of η for each?

EXERCISE 8.3

Bias correction. With $m_0 = 0$ and $g_k = g$ for every k, show that $m_k = (1 - \beta_1^k) g$. For a gradient that alternates, $g_k = (-1)^k g$, compute $\hat{m}_k$ for large k, and explain why Adam takes small steps along a direction in which the gradient keeps changing sign.

EXERCISE 8.4

At initialisation, condition on the final normalised column x and let $z = W_{\text{out}}x$ with the entries of W_{out} independent, of mean zero and variance $1/D^2$. Show that the V logits are independent with mean zero and variance $\|x\|^2/D^2 \approx 1/D$. Expand $\log \sum_a e^{z_a}$ to second order to show that the expected loss of one prediction is approximately $\ln V + \frac{V-1}{2V} \cdot \frac{1}{D}$. Evaluate it for the specimen and compare it with the loss at step 0 of Figure 8.1 (p. 101).

EXERCISE 8.5

Prove that $\text{clip}_c(g)$ is the unique point of the closed ball $\{\|v\| \le c\}$ nearest to g. Is clipping each array of θ separately to radius c also a projection onto a convex set? Onto which, and does it preserve the direction of the whole gradient?

EXERCISE 8.6

In Figure 8.1 (p. 101), switch to gradient descent and train. Then drag η upwards until the run learns within two hundred steps, and further until it fails. Compare the two learning rates with the sharpness in Figure 8.2 (p. 103).

EXERCISE 8.7

In Figure 8.1 (p. 101), choose a constant learning rate and keep pressing ▶ until the run has five thousand steps. Does the loss stay on the floor? Compare with Figure 8.6 (p. 111), which has no weight decay and no clipping, and say which of the two you would expect to make a slingshot rarer.

CHAPTER NINE

9 Speaking

To write, a language model runs on its own output: it draws the next character from its Gibbs distribution, appends it, and repeats; temperature trades fidelity for variety, and a model trained on Shakespeare shows exactly what next-character statistics can buy, and what they cannot.

MATHS	PHYSICS	ML
$a = \min\{a : \ p_1 + \cdots + p_a > u\}$, $u \sim U[0,1)$	*drawing a state from the Gibbs distribution*	"sampling"
$\arg\max_a z_a$	*ground state*, $\tau \to 0$	"greedy decoding"
$\mathrm{softmax}(z/\tau)$, $\tau \neq 1$	*Gibbs distribution at another temperature*	"temperature sampling"
$x_{1:t} \mapsto x_{t+1}$, repeated	*a Markov chain on contexts*	"autoregressive generation"
the last T characters	*a finite memory*	"context window"
$x_{1:k}$, fixed	*initial condition*	"prompt"
$e^{\mathscr{L}}$	*effective number of states*	"perplexity"

9.1 One character at a time

THE CHAIN RULE OF PROBABILITY (CHAPTER 1) FACTORS THE probability of a text into next-character probabilities,

$$P(x_1, \ldots, x_m) = P(x_1) \prod_{t=1}^{m-1} P(x_{t+1} \mid x_1, \ldots, x_t).$$

Read from left to right, it is also a recipe for drawing a text. Fix the first characters, the *prompt*. Then draw x_{t+1} from $q(\,\cdot \mid x_{1:t})$, append it, and repeat. Any family of normalised conditional distributions defines a normalised distribution on texts in this way, and drawing one factor at a time draws a text from it. The model supplies the conditionals: run it on the text so far, take the logits z at the last position, and turn them into $p = \operatorname{softmax}(z/\tau)$.

The model sees at most T characters. When the text grows longer, only the last T are fed in, numbered $1, \ldots, T$ again. That is a restriction on the model, a finite memory, not a consequence of the chain rule. The process is then a Markov chain whose state is the last T characters.

One draw needs one uniform random number $u \in [0,1)$. Stack the probabilities $p_1, \ldots, p_V$ end to end along $[0,1]$, in vocabulary order. The draw is the character whose segment contains u, that is, the first a with $p_1 + \cdots + p_a > u$. Since u lands in a segment with probability equal to its length, character a is drawn with probability p_a. This is inversion of the cumulative distribution function, and it is all the code does:

src/sample.jl draw · lines 14–20, 21

```
14 function draw(rng, p)
15     !isempty(p) && all(x -> isfinite(x) && x ≥ 0, p) ||
16         throw(ArgumentError("sampling needs a nonempty vector of finite nonnegative weights"))
17     F = cumsum(p)
18     isfinite(F[end]) && F[end] > 0 || throw(ArgumentError("sampling weights must have a finite positive sum"))
19     something(findfirst(>(rand(rng) * F[end]), F), length(p))
20 end
   ⋮
21 draw(p) = draw(Random.default_rng(), p)
```

src/sample.jl generate · lines 29–41

```
29 function generate(θ, c::Config, prompt, n; τ = 1, rng = Random.default_rng())
30     n ≥ 0 && isfinite(τ) && τ ≥ 0 || throw(ArgumentError("generation needs n ≥ 0 and finite τ ≥ 0"))
31     n == 0 || !isempty(prompt) || throw(ArgumentError("generation needs a nonempty prompt"))
32     τ = eltype(θ)(τ)
33     x = collect(prompt)
34     for _ in 1:n
35         context = x[max(end - c.T + 1, 1):end]
36         Z, _ = gpt(θ, reshape(context, :, 1), c)
37         z = Z[:, end]
```

```
        push!(x, τ == 0 ? argmax(z) : draw(rng, softmax(z ./ τ)))
    end
    x
end
```

9.2 Temperature in practice

The model was trained at $\tau = 1$, so $p = \text{softmax}(z)$ is the distribution that training fitted to the data, and $\tau = 1$ is the only temperature at which samples are drawn from the model itself. Every other temperature is a deliberate distortion, in one of two directions (Chapter 3).

As $\tau \to 0$ the Gibbs distribution freezes onto its ground state, and sampling becomes *greedy decoding*: always take the likeliest character. Greedy text is the model's most confident guess at each step, but not the most probable text: the likeliest first character may lead into contexts where every continuation is unlikely (Exercise 9.2, p. 126). Greedy text also tends to fall into loops, because a deterministic map from contexts to characters must eventually revisit a context, and from then on it repeats.

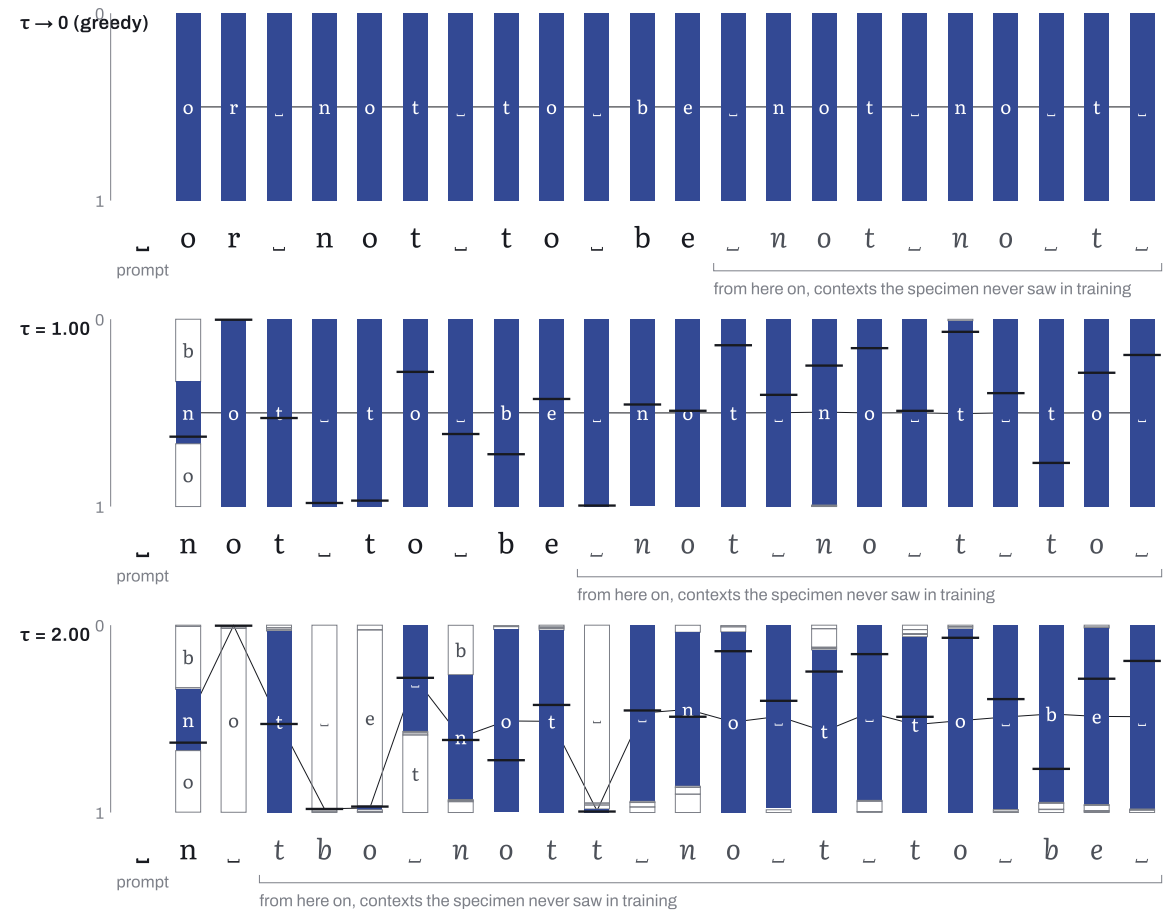
As τ grows, the distribution melts towards uniform and the text becomes more varied and less like the data. In between there is a trade: a little below 1, samples stay closer to the typical patterns of the training text and make fewer errors; above 1 they wander.

The code takes $\tau = 0$ to mean `argmax(z)`, which returns the *first* largest logit. The true limit of Chapter 3 shares the probability equally among tied ground states. With Float64 logits exact ties almost never happen, so the shortcut costs nothing in practice, but it is a convention, not the limit.

9.3 The specimen speaks

Figure 9.1 (p. 120) lets the specimen write from a one-character prompt, a space. Each column is one draw: the interval [0,1] cut into the seven probabilities, the random number u as a tick, and the character whose segment it hits. The thin line through the chosen segments is the path the text took; the outlined segments are the paths not taken.

At $\tau = 1$ the first column is cut into three nearly equal parts: after a space at the start of a window, the specimen cannot tell "␣be", "␣or" and "␣not" apart, which is the threefold ground state of Chapter 3 and the floor of Chapter 4 seen from the writer's side. After that first choice almost every column is a single solid segment. The specimen has memorised its corpus: once a branch is taken, the rest of the line follows. Different random numbers u



HOW TO READ

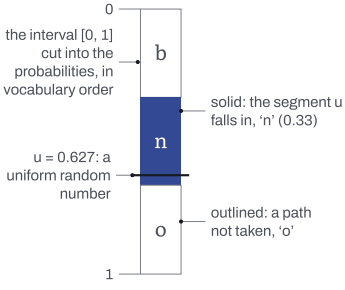


Figure 9.1 The trained specimen writes twenty-two characters after the prompt "␣". Each column is the distribution of the next character, stacked along [0,1] in vocabulary order; the tick is the uniform random number u, and the segment it falls in (solid) is the character written. Drag the temperature, draw new random numbers, or play the writing from the start. Under the text: where the specimen's contexts leave the set it was trained on.

choose different first characters, and each choice is followed by the rest of the corpus from that point on.

Then the text runs past the last "be" of the corpus. From there on, every context the specimen sees is one it never met in

training; the bracket under the text marks the place. The columns stay just as solid. The specimen is exactly as sure of what it never saw as of what it learned, because nothing in training ever asked it to be unsure: softmax always returns a distribution, and away from the data that distribution is whatever the parameters happen to give. Confidence is not knowledge.

Drag the temperature up to 2 and the outlined segments widen. Branches that were negligible become possible, the text leaves the corpus earlier, and it decays into strings like those a careless copyist might produce. Drag it to the far left and every column is a single segment: greedy decoding, the same text whatever u.

9.4 A real model

The specimen can only recite. To see what next-character prediction can do, the same code trains a larger model on *Tiny Shakespeare*: about a million characters from Shakespeare's plays, $V = 65$ distinct characters (letters in both cases, punctuation, space, newline, and one digit, 3). The model has $D = 128$, $H = 4$ heads, $L = 4$ blocks and a context of $T = 256$ characters, and its 810,049 parameters are trained in Julia in Float32, with AdamW, a warmup-and-cosine schedule and clipping (Chapter 8). Each step reads 32 windows of 256 characters. The last tenth of the text is held out: the model never trains on it, so its loss there measures prediction rather than memory.

The loss starts at $\log 65$, where it must: the untrained model's logits are nearly equal, and its first sample is every character of the vocabulary in random order, capitals, punctuation and the one digit alike. Within a hundred steps it has learned which characters are common: spaces, newlines, lower-case vowels. By a few hundred steps the strings have the shape of words and lines. By a thousand the text has the layout of a play, with speaker names in capitals, a colon and a line break, and lines of roughly the length of verse. By the end, most words are real words.

A loss in nats is easier to feel through its exponential. If a model's loss is $\mathscr{L}$, then $e^{\mathscr{L}}$, the *perplexity*, is the number of equally likely characters that would cost the same surprise: 65 at the start, and, at the end, a handful (read it off the figure). Two curves diverge as training goes on. The loss on the training text keeps falling; the loss on held-out text falls more slowly and then

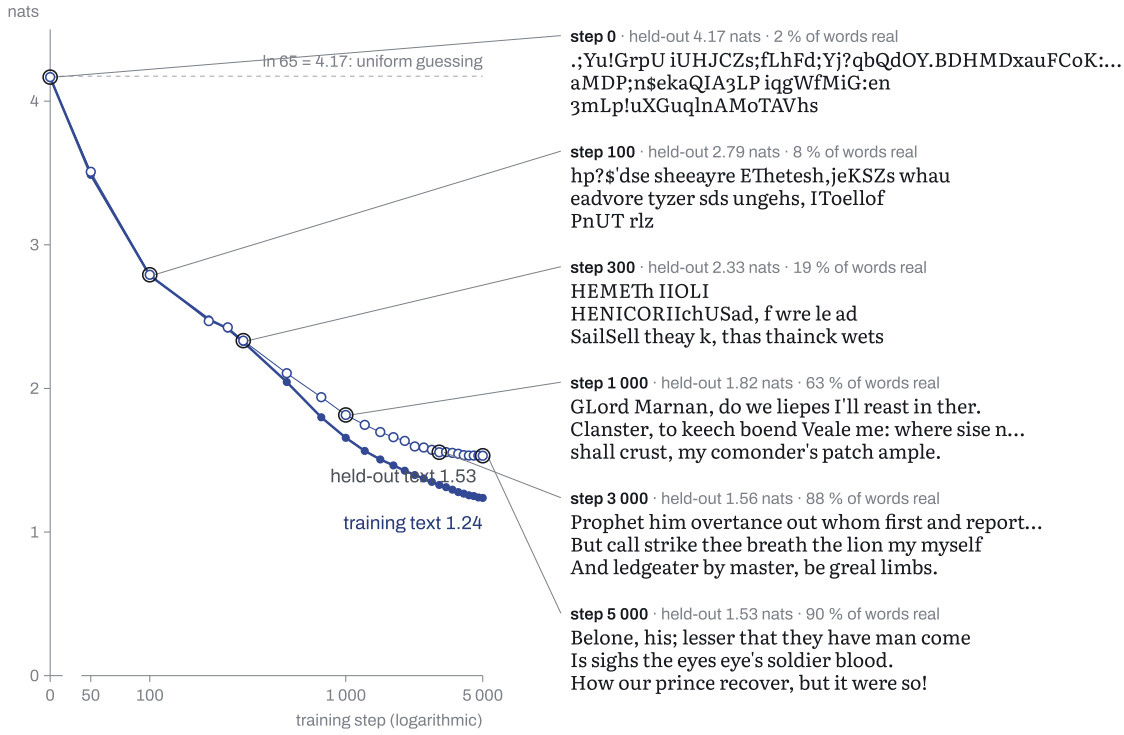


Figure 9.2 The Tiny Shakespeare model in training. Left: its loss, the mean surprise per character, on its training text (solid) and on held-out text it never saw (open circles), with $\log 65$, uniform guessing, for reference; the step axis is logarithmic. Right: what the model wrote at $\tau = 1$ at each saved checkpoint, starting from a newline, with the share of its words that occur in the training text.

flattens. The gap between them is what the model has memorised about its particular million characters rather than learned about Shakespeare's.

Figure 9.3 shows the dial at work. At $\tau = \frac{1}{2}$ every word is real, and the text is safe and repetitive, leaning on the commonest words. At $\tau = 1$ a few invented words appear. At $\tau = 1.5$ a third of the words are invented, and spelling decays.

Here, honestly, is what the model has learned. It spells common words. It knows the format of a play: who speaks is set in capitals and followed by a colon and a line break, and speeches are broken into lines of verse length. It knows where capitals and punctuation go. It strings two or three words together with a plausible rhythm. And here is what it has not learned. It does not know what any word means. Its sentences have no syntax

τ = 0.5	τ = 0.8	τ = 1 the model's own	τ = 1.5
81 of 81 words real (100 %)	72 of 77 words real (94 %)	63 of 70 words real (90 %)	41 of 63 words real (65 %)
I am the man and fear of little and heart. MENENIUS: If it not there, how the king his complexions, That should be the buried of my heart And so her brothers, which the song that the seas And the time of my father's death, and my earth To know the fight is the dead and grown and by;	See when this champion of all and the encouracis good the world, when it shames to stay, Or he the sides ground and this and my sorrow. What is thou art report, then, my souls, Nell we present with the dry wooers in hart As body the very oath piled Triumphy: what are tale? GLOUCESTER: The trumper, sentency and me the sigh	Belone, his; lesser that they have man come Is sighs the eyes eye's soldier blood. How our prince recover, but it were so! HASTINGS: Hold, as you To-daughter stands do you, cry buy two so offence Tempers me? But is a lingering dash'd. PETRUCHIO: Who must sir and so believe, the fruiter torrow, with	Decomesses and Richard!ing it, Lervant: by fools'Farp's Duke home O! 'coupme of law-truly:' Her which wonlves liquel Ratclass' errish; fest, takin? Within bloots glasdace! But fire: savouch, unnever, though his souls!--I'll aginly king, when it hay, I caught forgived, one doing stranged word: this old. QUEEN MARGARET:

Step 5000. Every sample starts from a newline; underlined with dots: words that never occur in the training text.

Figure 9.3 The trained model writing from the same prompt at four temperatures. Words that never occur in the training text are underlined with dots; the bar above each column is the share of words that do.

beyond a few words. The speakers do not speak to each other, nothing happens, and names are invented as freely as words. Every one of these is a statement about next-character statistics: what can be predicted from the last few characters, it predicts; what needs more than that, it does not.

9.5 Inside the real model

What do its sixteen heads read? Figure 9.4 (p. 125) runs the trained model on a passage of the held-out text in which one line

is spoken twice. Above the passage are arcs for one reader in one head, as in Chapter 5; beneath it, the surprise of each character. Below are all sixteen attention patterns, drawn as lower triangles, as in Chapter 5, with three habits measured for each head: how much weight it puts on the reader itself, on the character before it, and on the character a copying model would read.

The first layer spreads its attention widely. In the later layers many heads put a large share of their weight, in several close to half, on the reader itself and on the character before it; their dots crowd the diagonal and the line just below it. That is how a model whose maps act column by column (Chapter 6) gets to see pairs and triples of characters at all: a head that reads the previous character copies it into the current column, and the next layer can then react to the pair. One head of the third layer looks different: its dots stand in vertical stripes, because whatever the reader, it puts much of its weight on line breaks and on the capitals that begin lines. A model that writes speeches in lines has use for knowing where the current line began.

Heads that do this, called *induction heads*, are well documented in larger transformers trained for longer, where they appear fairly suddenly during training (Olsson et al., *In-context learning and induction heads*, 2022). The mechanism needs two heads in sequence: one that reads the previous character, and one that looks for an earlier position whose previous character matches the current one. This model has several heads of the first kind and none of the second.

The repeated line is a test. A model that noticed the repetition could predict the second "Good morrow, neighbour" almost perfectly: read where the same characters came before, and copy what followed them. The *copy* number measures exactly that, head by head. It is at most a hundredth for every head of this model, and the surprise per character of the repeated line is the same the second time as the first. Within the 5000 steps of its training, this model has not learned to copy from its own context.

The two models of this book sit at the ends of the same road. The specimen has learned its eighteen characters perfectly and knows nothing else. The Shakespeare model has learned the statistics of a million characters well enough to imitate their surface, and nothing of what they say. Everything in between, the larger contexts, the more layers, the longer training, the more data, is the subject of the Epilogue.

Exercises

EXERCISE 9.1

Let u be uniform on [0,1) and $F_a = p_1 + \cdots + p_a$. Show that the first a with $F_a > u$ is a with probability p_a. What changes if the characters are stacked in a different order?

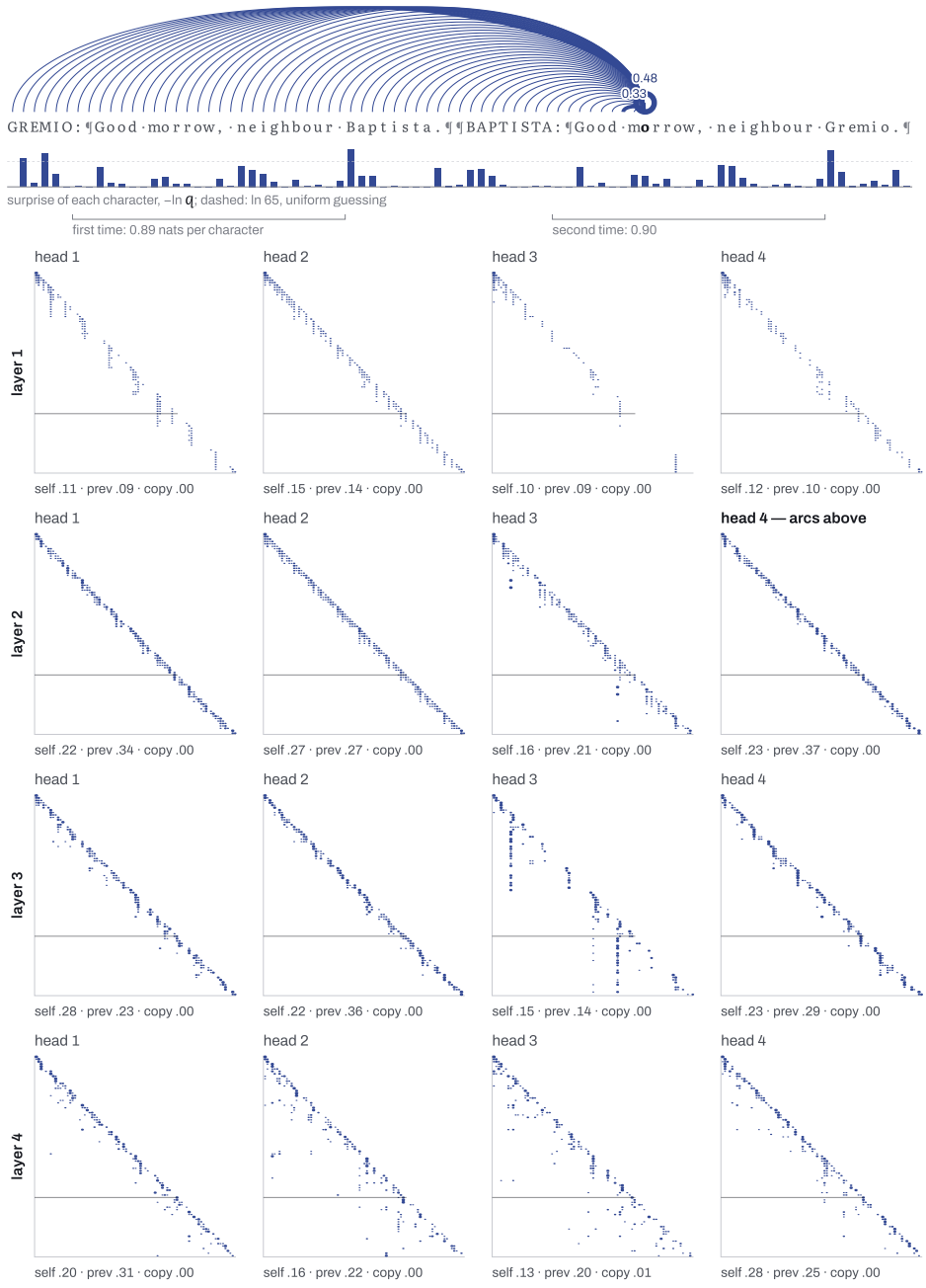


Figure 9.4 The trained model reading a passage of held-out text that contains a repeated line. Top: the passage, with each character's surprise beneath it; arcs show what the chosen reader reads in the chosen head. Below: every head of every layer, rows readers and columns sources, dot area the weight. Under each head, its mean weight on the reader itself (self), on the previous character (prev), and, over the repeated line, on the character that followed the same character the first time (copy). Point at a character, or click a head.

EXERCISE 9.2

Construct a model on two characters $\{x, y\}$ and texts of length two in which greedy decoding produces a text that is not the most probable text of length two.

EXERCISE 9.3

In Figure 9.1 (p. 120) at $\tau = 1$, how many different texts can the specimen write from the prompt "␣" before it leaves its corpus? Use "draw again" to check. Then raise τ to 2 and count the columns that are no longer a single segment.

EXERCISE 9.4

Show that $e^{H(p)} = V$ for the uniform distribution on V outcomes and $e^{H(p)} = 1$ for a one-hot p. Read the final held-out loss of the Shakespeare model off Figure 9.2 (p. 122) and convert it to a perplexity. Between how many characters is the model, on average, still choosing?

EXERCISE 9.5

Why is the loss on held-out text higher than on the training text, and why does the gap grow? What would you expect if training went on for ten times as long with the same data?

EXERCISE 9.6

In Figure 9.4 (p. 125), pick a head with a large *prev*. Point at characters in the middle of a word and at the first character after a space. What does each read? Why might the model want the previous character to be available in every column?

Epilogue

What this book left out changes how much is computed, on what hardware and in how many bits, far more than it changes what is computed: a large language model is still a composition of maps between named spaces, pushed forward to predict and pulled back to learn.

MATHS	PHYSICS	ML
an alphabet of strings $\mathscr{A} \subset \Sigma^*$ over bytes Σ	*a coarse-graining of the text*	"tokeniser vocabulary, subwords"
repeated merging of the most frequent adjacent pair	*a renormalisation step, run greedily*	"byte-pair encoding, BPE"
floating-point numbers with fewer bits of mantissa	*a coarser grid of representable values*	"mixed precision, bf16"
rotating q_i and k_j by $R(i)$ and $R(j)$ before $q_i \cdot k_j$	*a phase convention depending only on $i - j$*	"rotary position embedding, RoPE"
the stored columns k_j, v_j of the past, reused	*a memo*	"KV cache"
minimising a second loss, starting from the first minimiser	*relaxing from a prepared state*	"fine-tuning"

E.1 What we left out

THE SPECIMEN IS COMPLETE IN ITS MAPS AND SMALL IN everything else. The distance from it to the models in the news is real, and it lies in a few directions that can be named.

Tokens instead of characters

Our alphabet is the set of characters of the text. Production models use an alphabet of *tokens*: strings of bytes chosen once, before training, so that common words are one token and rare words a few. The usual construction is *byte-pair encoding*. Start from the 256 byte values; count every adjacent pair of symbols in a large sample of text; add the most frequent pair to the alphabet as a new symbol and rewrite the sample with it; repeat until the alphabet has the desired size, typically tens of thousands to a few hundred thousand.[1] The result is a finite alphabet $\mathscr{A}$, and nothing in this book depends on what its elements are: V grows, E and W_{out} get more columns and rows, and every map is unchanged.

What changes is the text the model sees. A token covers about four characters of English on average, so a context of T tokens holds about four times as much text, and the chain rule of Chapter 1 is applied to fewer, larger symbols. The price is that the model never sees the letters inside a token, which is why large models can be oddly bad at spelling and at counting characters.

[1] The algorithm was introduced for data compression by Gage (1994) and adapted to language by Sennrich, Haddow and Birch (2016).

Scale

The specimen has about a thousand parameters, and the Shakespeare model of Chapter 9 a little under a million. Published language models have had from hundreds of millions to hundreds of billions.

Size has a second axis, the data. The specimen sees the same 80 predictions at every step. A large model sees trillions of tokens, most of them only once. Over many orders of magnitude, the loss reached by such models falls smoothly, close to a power law in the number of parameters and in the number of tokens trained on,[2] and the practice of the field is largely the practice of exploiting that regularity. None of it requires a new map; all of it requires more of the same ones.

[2] Kaplan et al., *Scaling laws for neural language models* (2020); Hoffmann et al., *Training compute-optimal large language models* (2022).

Hardware

Almost all the arithmetic of the model is matrix products: the four maps of attention, the two of the MLP and the readout, in both directions. A graphics processor performs such products thousands of times faster than the loops of our Julia, and production code is organised around keeping it busy: long batches, fused kernels, and parameters and activations split across many devices.

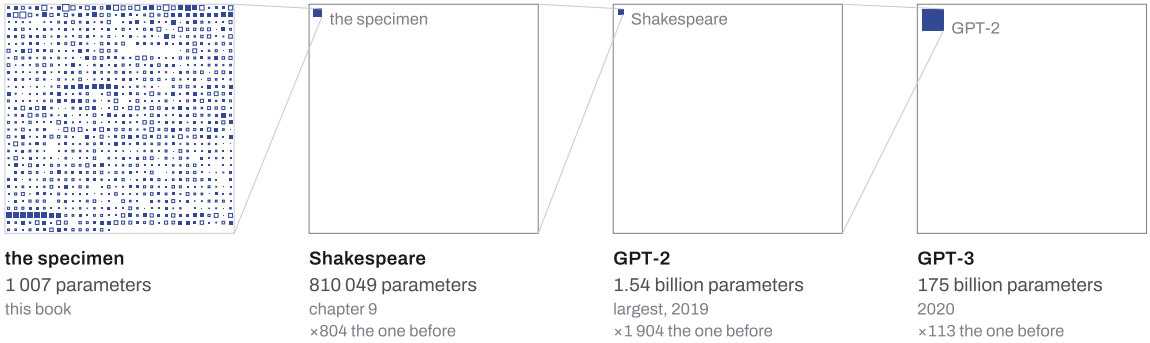


Figure E.1 How big is big. Each square has area proportional to a parameter count, and each is drawn inside the next as a single mark. From the left: the specimen; the Shakespeare model, trained on the works of Shakespeare; GPT-2, the largest of the 2019 models, with 1.5 billion parameters (Radford et al., 2019); and GPT-3, with 175 billion (Brown et al., 2020). These two published counts are the only numbers in the book that were not computed from our own models.

Two costs we never felt dominate at scale. Attention computes a $T \times T$ score matrix for every head and every window, so its cost grows with the square of the context; fast implementations compute the masked softmax of Chapter 5 in tiles, never storing the scores, and never computing the upper triangle at all.[3] And when a model writes, each new character needs the keys and values of every earlier position; they are stored once and reused, in a KV *cache*, instead of recomputing the whole window at every step as `generate` does in Chapter 9.

[3] Dao et al., *FlashAttention* (2022).

Precision

The specimen computes in 64-bit floating point, so that the Julia and the JavaScript can agree to twelve decimal places and every figure is exact. The Shakespeare model trains in 32 bits. Large models train with 16-bit numbers for most of the arithmetic, keeping a 32-bit copy of the parameters for the update and, in the narrowest formats, rescaling the loss so that small gradients survive the coarser grid.[4] They are often served with fewer bits still. The maps are the same; the rounding is not, and a good part of the engineering is in keeping it harmless.

[4] Micikevicius et al., *Mixed precision training* (2018).

Other architectures

Our block is the *pre-norm* transformer: LayerNorm before each branch, the residual stream left untouched between them. The original transformer of 2017 normalised after the sum; the pre-norm order trains more reliably in deep stacks.[5] Since then the parts have changed while the skeleton has not:

- Normalisation without centring, which divides by the root mean square only (RMSNorm).
- Positions as rotations of queries and keys (RoPE), which is the rotation property of Chapter 2 made into the mechanism.[6]
- A gated MLP, in which the output of the nonlinearity is multiplied, entry by entry, by a second linear image of the input.[7]
- Several query heads sharing one key and value head, to shrink the cache; and *mixtures of experts*, in which each token passes through only a few of many MLPs.
- The embedding and the readout as one matrix, $W_{\text{out}} = E^{\mathsf{T}}$. Ours are independent.

Further away are models whose mixing step is not attention at all, such as the state-space models, which replace the $T \times T$ kernel by a recurrence that is linear in its state.[8]

[5] Vaswani et al., *Attention is all you need* (2017); Xiong et al., *On layer normalization in the transformer architecture* (2020).

[6] Su et al., *RoFormer* (2021).

[7] Shazeer, *GLU variants improve transformer* (2020).

[8] Gu and Dao, *Mamba* (2023).

After the first loss

A model trained only to predict the next token of a large text continues text; it does not answer questions. The models one talks to have been trained further: on curated dialogues, with the same loss, and then against a learned score of how well a human would rate the answer.[9] These stages change the data and the objective. The pullbacks of Chapter 7, and the optimiser of Chapter 8, are used unchanged.

[9] Ouyang et al., *Training language models to follow instructions with human feedback* (2022).

E.2 A dictionary of jargon

Every term of the field defined in this book, in alphabetical order, with its names in mathematics and in physics and the chapter that defines it. The dictionary is assembled when the book is built, from the names tables at the head of each chapter; it is exactly as complete as they are.

A

"**Adam**" $\theta \leftarrow \theta - \eta G\hat{m}$, G diagonal · *a coordinate-dependent mobility* → 8 Learning, p. 98

"**AdamW**" $\theta \leftarrow (1-\eta\lambda)\theta - \cdots$ · *friction towards the origin* → 8 Learning, p. 98

"**architecture**" a map f_θ from inputs to outputs · *a response function* → 0 Prologue, p. 7

"**attention output**" $o_i = \sum_j A_{ij} v_j$, a convex combination · *expectation value* $\langle v \rangle_i$ → 5 Attention, p. 58

"**attention pattern**" row-stochastic, lower-triangular $A \in \mathbb{R}^{T\times T}$ · *causal transition kernel* → 5 Attention, p. 58

"**attention score**" $S_{ij} = q_i \cdot k_j / \sqrt{d}$ · *negative energy* → 5 Attention, p. 58

"**autoregressive generation**" $x_{1:t} \mapsto x_{t+1}$, repeated · *a Markov chain on contexts* → 9 Speaking, p. 117

"**autoregressive model**" $P(x) = \prod_t q(x_{t+1} \mid x_{\leq t})$ · *a chain of conditional experiments* → 1 The task, p. 14

B

"**backpropagation**" applying $Df^\top$ in reverse order · *the adjoint equation* → 0 Prologue, p. 7

"**backpropagation through** f" $\bar{x} = Df(x)^\top \bar{y}$ · *the adjoint map* → 7 Pulling back, p. 82

"**backward pass**" 1 applying $Df^\top$ in reverse order · *the adjoint equation* → 0 Prologue, p. 7
2 $(f_k \circ \cdots \circ f_1)^* = f_1^* \circ \cdots \circ f_k^*$ · *the adjoint method* → 7 Pulling back, p. 82

"**bf16**" floating-point numbers with fewer bits of mantissa · *a coarser grid of representable values* → Epilogue, p. 127

"**BPE**" repeated merging of the most frequent adjacent pair · *a renormalisation step, run greedily* → Epilogue, p. 127

"**byte-pair encoding**" repeated merging of the most frequent adjacent pair · *a renormalisation step, run greedily* → Epilogue, p. 127

C

"**causal mask**" $A_{ij} = 0$ for $j > i$ · *causality* → 5 Attention, p. 58

"**centring**" $\Pi = I - \mathbf{1}\mathbf{1}^\top / D$ · *projection out of the uniform mode* → 6 The block, p. 68

"**context**" the prefix $x_{\leq t} = (x_1, \ldots, x_t)$ · *the past* → 1 The task, p. 14

"**context length**" the largest t the map accepts, T · *the length of the memory* → 1 The task, p. 14

"**context window**" 1 the largest t the map accepts, T · *the length of the memory* → 1 The task, p. 14 2 the last T characters · *a finite memory* → 9 Speaking, p. 117

"**cross-entropy**" $H(p,q) = -\sum_a p_a \log q_a$ · *mean surprisal under* p → 4 Measuring surprise, p. 46

"**cross-entropy loss**" $\mathscr{L}(\theta) = \frac{1}{N}\sum_n -\log q_\theta(y_n \mid c_n)$ · *mean surprisal* → 4 Measuring surprise, p. 46

D

"d_{model}" the dimension D of the columns · *the number of modes* → 2 Symbols become vectors, p. 24

E

"**embedding lookup**" $x \mapsto E[:, x] = EC$, with C one-hot · *evaluating a linear map on basis states* → 2 Symbols become vectors, p. 24

"**embedding matrix**" $E \in \mathbb{R}^{D\times V}$, column a is Ee_a · *a coupling of each symbol to* D *modes* → 2 Symbols become vectors, p. 24

"**entropy**" $H(p) = -\sum_a p_a \log p_a$ · *Gibbs entropy* → 4 Measuring surprise, p. 46

F

"**feed-forward layer**" $y = \sum_k r_k (W_2)_{:,k} + b_2$, $r = \text{relu}(W_1 \hat{f} + b_1)$ · *a sum of switched sources* → 6 The block, p. 68

"**fine-tuning**" minimising a second loss, starting from the first minimiser · *relaxing from a prepared state* → Epilogue, p. 127

"**forward pass**" evaluating f_θ, value by value · *integrating forward* → 0 Prologue, p. 7

"**forward-mode derivative**" $Df(x)$, pushing dx forward to dy · *linear response* → 7 Pulling back, p. 82

G

"**"grad"**" $\bar{x} = \partial \ell / \partial x$, a covector · *a generalised force, conjugate to* x → 7 Pulling back, p. 82

"**gradient**" 1 the covector $\bar{x} = \partial \ell / \partial x$ · *the linear response, the conjugate variable* → 0 Prologue, p. 7 2 $\bar{x} = \partial \ell / \partial x$, a covector · *a generalised force, conjugate to* x → 7 Pulling back, p. 82

"**gradient accumulation**" $\Delta^*(\bar{u}, \bar{v}) = \bar{u} + \bar{v}$ · *responses superpose* → 7 Pulling back, p. 82

"**gradient check**" $\big(\ell(\theta + he_i) - \ell(\theta - he_i)\big)/2h$ · *a symmetric perturbation experiment* → 7 Pulling back, p. 82

"**gradient clipping**" projection onto $\{\|g\| \le c\}$ · *a bound on the force* → 8 Learning, p. 98

"**gradient descent**" $\theta_{k+1} = \theta_k - \eta \nabla \ell(\theta_k)$ · *an explicit Euler step* → 8 Learning, p. 98

"**gradient flow**" $\dot{\theta} = -\nabla \ell(\theta)$ · *overdamped motion in a potential* → 8 Learning, p. 98

"**gradient of the loss with respect to the logits**" $\bar{Z} = (P - Y)/N$ · *predicted minus observed occupation* → 4 Measuring surprise, p. 46

"**greedy choice**" $\arg\max_a z_a$ · *ground state* → 3 Energies and probabilities, p. 34

"**greedy decoding**" $\arg\max_a z_a$ · *ground state,* $\tau \to 0$ → 9 Speaking, p. 117

I

"**inference**" evaluating f_θ, value by value · *integrating forward* → 0 Prologue, p. 7

"**input embeddings**" $X_0 = E[:, x] + P \in \mathbb{R}^{D \times T}$ · *the initial condition* → 2 Symbols become vectors, p. 24

J

"**Jacobian**" $Df(x)$, pushing dx forward to dy · *linear response* → 7 Pulling back, p. 82

K

"**KL divergence**" $\mathrm{KL}(p \,\|\, q)$ · *relative entropy* → 4 Measuring surprise, p. 46

"**KV cache**" the stored columns k_j, v_j of the past, reused · *a memo* → Epilogue, p. 127

L

"**layer normalisation**" $\hat{x} = \Pi x / \sigma$, $\|\hat{x}\| \lesssim \sqrt{D}$ · *projection to (just inside) a sphere* → 6 The block, p. 68

"**LayerNorm**" $\hat{x} = \Pi x / \sigma$, $\|\hat{x}\| \lesssim \sqrt{D}$ · *projection to (just inside) a sphere* → 6 The block, p. 68

"**learned gain and bias**" $y = \gamma \odot \hat{x} + \beta$ · *a diagonal affine map* → 6 The block, p. 68

"**learning rate**" η · *the time step* → 8 Learning, p. 98

"**likelihood**" $\prod_n q_\theta(y_n \mid c_n)$ · *likelihood* → 4 Measuring surprise, p. 46

"**logit**" $S_{ij} = q_i \cdot k_j / \sqrt{d}$ · *negative energy* → 5 Attention, p. 58

"**logits**" $z \in \mathbb{R}^V$ · *negative energies* $-E_a$ → 3 Energies and probabilities, p. 34

"**log-sum-exp**" $\mathrm{lse}(z) = \log \sum_a e^{z_a}$ · $\log \mathcal{Z} = -F$ *at* $\tau = 1$ → 3 Energies and probabilities, p. 34

"**loss**" 1 $\ell : \mathbb{R}^P \to \mathbb{R}$, to be minimised · *an energy, a cost* → 0 Prologue, p. 7 2 $\mathscr{L}(\theta) = \frac{1}{N} \sum_n -\log q_\theta(y_n \mid c_n)$ · *mean surprisal* → 4 Measuring surprise, p. 46

M

"**mean subtraction**" $\Pi = I - \mathbf{1}\mathbf{1}^\top / D$ · *projection out of the uniform mode* → 6 The block, p. 68

"**minibatch gradient**" $\hat{g}$ with $\mathbb{E}\hat{g} = \nabla \ell$ · *a noisy measurement of the force* → 8 Learning, p. 98

"**mixed precision**" floating-point numbers with fewer bits of mantissa · *a coarser grid of representable values* → Epilogue, p. 127

"**MLP**" $y = \sum_k r_k (W_2)_{:,k} + b_2$, $r = \mathrm{relu}(W_1 \hat{f} + b_1)$ · *a sum of switched sources* → 6 The block, p. 68

"**model**" 1 a map f_θ from inputs to outputs · *a response function* → 0 Prologue, p. 7 2 $\theta \in \mathbb{R}^P$ · *a point in configuration space* → 6 The block, p. 68

"**model dimension**" the dimension D of the columns · *the number of modes* → 2 Symbols become vectors, p. 24

"**model size**" $p = \dim \theta$ · *the number of degrees of freedom* → 0 Prologue, p. 7

"**multi-head attention**" $\mathbb{R}^D = \mathbb{R}^d \oplus \cdots \oplus \mathbb{R}^d$ · *direct sum of subsystems* → 5 Attention, p. 58

N

"**negative log-likelihood**" $-\log q_y$ · *surprisal*
→ 4 Measuring surprise, p. 46

"**network**" a map f_θ from inputs to outputs · *a response function* → 0 Prologue, p. 7

"**next-token prediction**" $q(\,\cdot \mid x_{\le t}) \in \Delta^{V-1}$ · *a conditional law, a transition kernel* → 1 The task, p. 14

"**normaliser**" $\sum_a e^{z_a}$ · *partition function* $\mathcal{Z}$
→ 3 Energies and probabilities, p. 34

"**numerical gradient**" $\big(\,\ell(\theta + he_i) - \ell(\theta - he_i)\,\big)/2h$ · *a symmetric perturbation experiment* → 7 Pulling back, p. 82

O

"**objective**" $\ell : \mathbb{R}^p \to \mathbb{R}$, to be minimised · *an energy, a cost* → 0 Prologue, p. 7

"**one-hot vector**" a standard basis vector $e_a \in \mathbb{R}^V$ · *a pure state, an occupation-number basis vector*
→ 2 Symbols become vectors, p. 24

P

"**parameter count**" $p = \dim\theta$ · *the number of degrees of freedom* → 0 Prologue, p. 7

"**parameters**" a point $\theta \in \mathbb{R}^p$ · *the coordinates of a configuration* → 0 Prologue, p. 7

"**perplexity**" $e^{\mathcal{L}}$ · *effective number of states* → 9 Speaking, p. 117

"**per-token loss**" 1 $-\log q(x_{t+1} \mid x_{\le t})$ · *surprisal*
→ 1 The task, p. 14 2 $-\log q_y$ · *surprisal*
→ 4 Measuring surprise, p. 46

"**preconditioning**" $\theta \leftarrow \theta - \eta\, G\hat{m}$, G diagonal · *a coordinate-dependent mobility* → 8 Learning, p. 98

"**probabilities**" $\mathrm{softmax}(z) \in \Delta^{V-1}$ · *Gibbs distribution*
→ 3 Energies and probabilities, p. 34

"**prompt**" 1 the prefix $x_{\le t} = (x_1, \dots, x_t)$ · *the past* → 1 The task, p. 14 2 $x_{1:k}$, fixed · *initial condition* → 9 Speaking, p. 117

R

"**random baseline**" $\log V$ · *entropy of the uniform distribution* → 4 Measuring surprise, p. 46

"**residual connection**" $x_{k+1} = x_k + f_k(x_k)$ · *an explicit Euler step* → 6 The block, p. 68

"**residual stream**" $X \in \mathbb{R}^{D\times N}$, updated by $X \leftarrow X + f(X)$ · *the state, evolved in depth* → 6 The block, p. 68

"**residual stream at layer 0**" $X_0 = E[:, x] + P \in \mathbb{R}^{D\times T}$ · *the initial condition* → 2 Symbols become vectors, p. 24

"**reverse-mode differentiation**" $(f_k \circ \cdots \circ f_1)^* = f_1^* \circ \cdots \circ f_k^*$ · *the adjoint method* → 7 Pulling back, p. 82

"**RoPE**" rotating q_i and k_j by $R(i)$ and $R(j)$ before $q_i \cdot k_j$ · *a phase convention depending only on* $i - j$ → Epilogue, p. 127

"**rotary position embedding**" rotating q_i and k_j by $R(i)$ and $R(j)$ before $q_i \cdot k_j$ · *a phase convention depending only on* $i - j$ → Epilogue, p. 127

S

"**sampling**" $a = \min\{a : \; p_1 + \cdots + p_a > u\}$
, $u \sim U[0,1)$ · *drawing a state from the Gibbs distribution* → 9 Speaking, p. 117

"**"scale and shift"**" $y = \gamma \odot \hat{x} + \beta$ · *a diagonal affine map* → 6 The block, p. 68

"**scale factor**" $\sqrt{d}$ · *temperature* → 5 Attention, p. 58

"**sequence**" $x = (x_1, \dots, x_m) \in \mathcal{A}^m$ · *a configuration, a history* → 1 The task, p. 14

"**sharpness**" $\lambda_{\max}$ of the Hessian · *the stiffest mode, the curvature scale* → 8 Learning, p. 98

"**shift invariance**" $z \mapsto z + c\mathbf{1}$ · *choice of the zero of energy*
→ 3 Energies and probabilities, p. 34

"**sinusoidal positional encoding**" $P_t = \big(\sin\omega_r t, \cos\omega_r t\big)_r$, fixed · *phases of* $D/2$ *oscillators*
→ 2 Symbols become vectors, p. 24

"**skip connection**" $x_{k+1} = x_k + f_k(x_k)$ · *an explicit Euler step* → 6 The block, p. 68

"**softmax output**" $\mathrm{softmax}(z) \in \Delta^{V-1}$ · *Gibbs distribution*
→ 3 Energies and probabilities, p. 34

"**step size**" η · *the time step* → 8 Learning, p. 98

"**stochastic gradient**" $\hat{g}$ with $\mathbb{E}\hat{g} = \nabla\ell$ · *a noisy measurement of the force* → 8 Learning, p. 98

"**subwords**" an alphabet of strings $\mathcal{A} \subset \Sigma^*$ over bytes Σ · *a coarse-graining of the text* → Epilogue, p. 127

T

"**temperature**" $\tau = 1/\beta$ · *temperature* → 3 Energies and probabilities, p. 34

"**temperature sampling**" $\mathrm{softmax}(z/\tau)$, $\tau \neq 1$ · *Gibbs distribution at another temperature* → 9 Speaking, p. 117

"**text**" $x = (x_1, \ldots, x_m) \in \mathscr{A}^m$ · *a configuration, a history* → 1 The task, p. 14

"**token**" $a \in \{1, \ldots, V\}$, the index of a symbol · *a label of a state* → 1 The task, p. 14

"**token embedding**" $E \in \mathbb{R}^{D \times V}$, column a is Ee_a · *a coupling of each symbol to D modes* → 2 Symbols become vectors, p. 24

"**token id**" $a \in \{1, \ldots, V\}$, the index of a symbol · *a label of a state* → 1 The task, p. 14

"**tokeniser vocabulary**" an alphabet of strings $\mathscr{A} \subset \Sigma^*$ over bytes Σ · *a coarse-graining of the text* → Epilogue, p. 127

"**training set**" the pairs $(x_{\leq t}, x_{t+1})$ we fit · *the sample* → 1 The task, p. 14

"**a training step**" $\theta_{k+1} = \theta_k - \eta \nabla \ell(\theta_k)$ · *an explicit Euler step* → 8 Learning, p. 98

U

"**an update**" $\theta_{k+1} = \theta_k - \eta \nabla \ell(\theta_k)$ · *an explicit Euler step* → 8 Learning, p. 98

V

"**vector–Jacobian product**" $\bar{x} = Df(x)^\top \bar{y}$ · *the adjoint map* → 7 Pulling back, p. 82

"**vocabulary**" a finite set $\mathscr{A}$ with $V = |\mathscr{A}|$ elements · *the single-site state space* → 1 The task, p. 14

W

"**weight decay**" $\theta \leftarrow (1 - \eta\lambda)\theta - \cdots$ · *friction towards the origin* → 8 Learning, p. 98

"**weights**" 1 a point $\theta \in \mathbb{R}^p$ · *the coordinates of a configuration* → 0 Prologue, p. 7 2 $\theta \in \mathbb{R}^p$ · *a point in configuration space* → 6 The block, p. 68

"**width**" the dimension D of the columns · *the number of modes* → 2 Symbols become vectors, p. 24